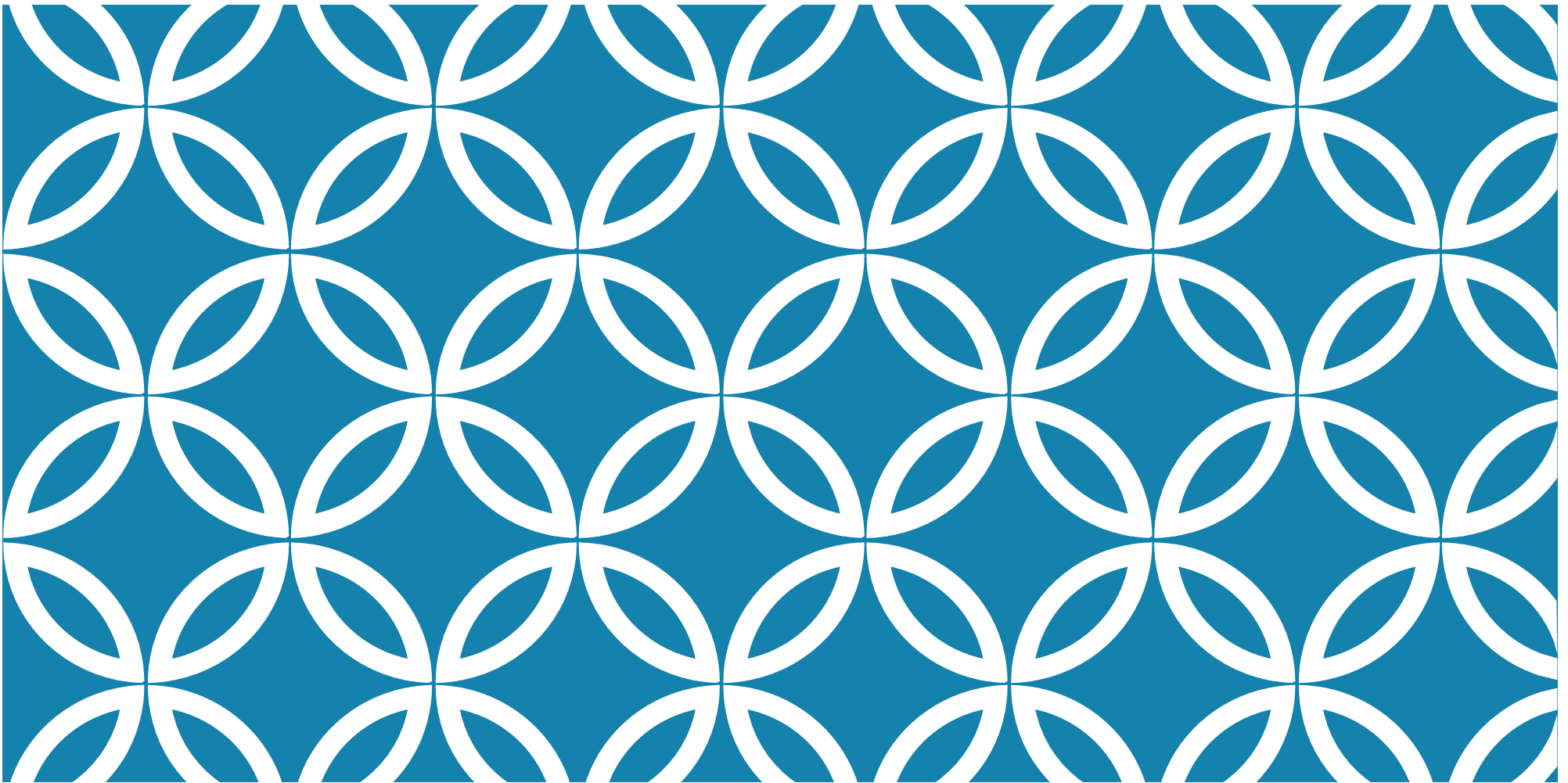
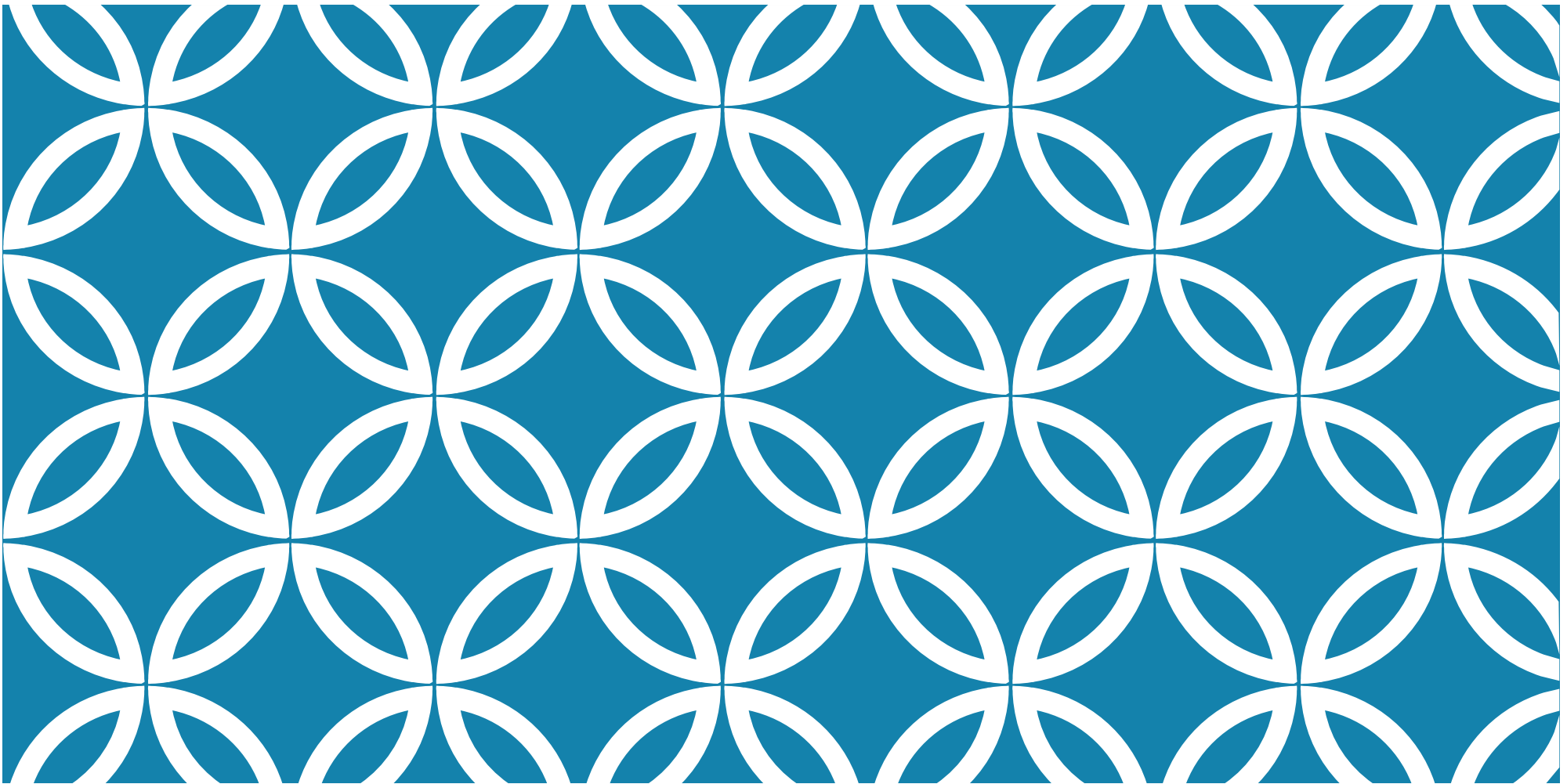




P00- JAVA ET SPRINT BOOT

Par : Evaris Fomekong
Année : 2026



LECTURE 1 : FONDAMENTAUX AVANCÉS DE JAVA ET POO POUR SPRING

- Révision approfondie des concepts POO : encapsulation, héritage, polymorphisme et abstraction en Java, - Interfaces, classes abstraites et leur utilisation dans la conception d'architectures modulai

INTRODUCTION : POO ET SPRING BOOT (PARTIE 1/2)

Les fondamentaux de la Programmation Orientée Objet constituent le socle architectural de Spring Boot. La maîtrise approfondie de l'encapsulation, de l'héritage, du polymorphisme et de l'abstraction est essentielle pour concevoir des applications Spring modulaires, maintenables et évolutives. Ces principes permettent de tirer pleinement parti des mécanismes d'injection de dépendances et de l'architecture en couches du framework.

INTRODUCTION : POO ET SPRING BOOT (PARTIE 2/2)

Révision approfondie des 4 piliers de la POO en Java

Interfaces et classes abstraites pour architectures modulaires

Collections Java et manipulation professionnelle

Application directe aux patterns Spring Boot

ENCAPSULATION : PRINCIPE ET APPLICATION (PARTIE 1/2)

L'encapsulation protège l'intégrité des données en masquant les détails d'implémentation et en contrôlant l'accès via des méthodes publiques. En Java, elle repose sur les modificateurs d'accès (`private`, `protected`, `public`) et les accesseurs (`getters/setters`). Dans Spring Boot, l'encapsulation est cruciale pour les entités JPA et les DTOs.

ENCAPSULATION : PRINCIPE ET APPLICATION (PARTIE 2/2)

Encapsulation dans une entité JPA Spring Boot (java)

```
1. @Entity
2. public class Utilisateur {
3.     @Id
4.     @GeneratedValue(strategy = GenerationType.IDENTITY)
5.     private Long id;
6.
7.     private String email;
8.     private String motDePasse;
9.
10.    // Constructeur par défaut requis par JPA
11.    public Utilisateur() {}
12.
13.    // Getters et Setters avec validation
14.    public String getEmail() {
15.        return email;
16.    }
17.
18.    public void setEmail(String email) {
19.        if (email == null || !email.contains("@")) {
20.            throw new IllegalArgumentException("Email invalide");
```

ENCAPSULATION : PRINCIPE ET APPLICATION (PARTIE 2/2) [A]

Encapsulation dans une entité JPA Spring Boot (java) - Suite

```
21.     }
22.     this.email = email;
23. }
24.
25. public void setMotDePasse(String motDePasse) {
26.     // Encapsulation de la logique de validation
27.     if (motDePasse.length() < 8) {
28.         throw new IllegalArgumentException("Mot de passe trop court");
29.     }
30.     this.motDePasse = motDePasse;
31. }
32. }
```

HÉRITAGE : RÉUTILISATION ET SPÉCIALISATION (PARTIE 1/3)

L'héritage permet de créer des hiérarchies de classes favorisant la réutilisation du code et la spécialisation progressive. En Java, une classe enfant hérite des attributs et méthodes de sa classe parente via le mot-clé 'extends'. Spring Boot exploite l'héritage pour structurer les couches applicatives (repositories, services, controllers).

HÉRITAGE : RÉUTILISATION ET SPÉCIALISATION (PARTIE 2/3)

Hiérarchie de classes avec héritage (java)

```
1. // Classe parente abstraite
2. public abstract class Personne {
3.     protected String nom;
4.     protected String prenom;
5.     protected LocalDate dateNaissance;
6.
7.     public abstract String getRole();
8.
9.     public int calculerAge() {
10.         return Period.between(dateNaissance, LocalDate.now()).getYears();
11.     }
12. }
13.
14. // Classes spécialisées
15. @Entity
16. public class Etudiant extends Personne {
17.     private String numeroEtudiant;
18.     private String promotion;
19.
20.     @Override
```

HÉRITAGE : RÉUTILISATION ET SPÉCIALISATION (PARTIE 2/3) [A]

Hiérarchie de classes avec héritage (java) - Suite

```
21.     public String getRole() {
22.         return "ETUDIANT";
23.     }
24. }
25.
26. @Entity
27. public class Professeur extends Personne {
28.     private String departement;
29.     private List<String> specialites;
30.
31.     @Override
32.     public String getRole() {
33.         return "PROFESSEUR";
34.     }
35. }
```

HÉRITAGE : RÉUTILISATION ET SPÉCIALISATION (PARTIE 3/3)

Favorise la réutilisation du code commun (attributs et méthodes)

Permet la spécialisation progressive des comportements

Facilite la maintenance en centralisant la logique partagée

Attention : éviter l'héritage trop profond (préférer la composition)

POLYMORPHISME : FLEXIBILITÉ ET EXTENSIBILITÉ (PARTIE 1/2)

Le polymorphisme permet à un objet de prendre plusieurs formes, autorisant l'utilisation d'une référence de type parent pour manipuler des objets de types enfants. Il se manifeste par la surcharge (même méthode, signatures différentes) et la redéfinition (override). Spring Boot exploite massivement le polymorphisme pour l'injection de dépendances et les stratégies de traitement.

POLYMORPHISME : FLEXIBILITÉ ET EXTENSIBILITÉ (PARTIE 2/2)

Polymorphisme avec injection de dépendances Spring (java)

```
1. // Interface de stratégie
2. public interface ServicePaielement {
3.     void traiterPaielement(double montant);
4.     boolean validerTransaction(String reference);
5. }
6.
7. // Implémentations polymorphiques
8. @Service
9. @Qualifier("carteBancaire")
10. public class PaiementCarteBancaire implements ServicePaielement {
11.     @Override
12.     public void traiterPaielement(double montant) {
13.         // Logique spécifique carte bancaire
14.         System.out.println("Paielement CB: " + montant + "€");
15.     }
16.
17.     @Override
18.     public boolean validerTransaction(String reference) {
19.         return reference.startsWith("CB-");
20.     }
}
```

POLYMORPHISME : FLEXIBILITÉ ET EXTENSIBILITÉ (PARTIE 2/2) [A]

Polymorphisme avec injection de dépendances Spring (java) - Suite

```
21. }
22.
23. @Service
24. @Qualifier("paypal")
25. public class PaiementPayPal implements ServicePaiement {
26.     @Override
27.     public void traiterPaiement(double montant) {
28.         // Logique spécifique PayPal
29.         System.out.println("Paiement PayPal: " + montant + "€");
30.     }
31.
32.     @Override
33.     public boolean validerTransaction(String reference) {
34.         return reference.startsWith("PP-");
35.     }
36. }
37.
38. // Utilisation polymorphique dans un controller
39. @RestController
40. public class PaiementController {
```

POLYMORPHISME : FLEXIBILITÉ ET EXTENSIBILITÉ (PARTIE 2/2) [B]

Polymorphisme avec injection de dépendances Spring (java) - Suite

```
41.     @Autowired
42.     @Qualifier("carteBancaire")
43.     private ServicePaie ment servicePaie ment;
44.
45.     @PostMapping("/payer")
46.     public ResponseEntity<String> effectuerPaie ment (@RequestParam
double montant) {
47.         servicePaie ment.traite rPaie ment (montant);
48.         return ResponseEntity.ok("Paie ment effectue ");
49.     }
50. }
```

ABSTRACTION : SIMPLIFICATION ET CONTRATS (PARTIE 1/2)

L'abstraction consiste à exposer uniquement les fonctionnalités essentielles en cachant les détails complexes d'implémentation. Elle se concrétise via les interfaces et classes abstraites qui définissent des contrats. Dans Spring Boot, l'abstraction structure l'architecture en couches et facilite les tests unitaires via le mocking.

ABSTRACTION : SIMPLIFICATION ET CONTRATS (PARTIE 2/2)

Comparaison Interface vs Classe Abstraite en Java

Caractéristique	Interface	Classe Abstraite
Héritage multiple	Oui (implements multiples)	Non (extends unique)
Méthodes concrètes	Oui (depuis Java 8: default)	Oui
Attributs d'instance	Non (seulement constantes)	Oui
Constructeur	Non	Oui
Usage Spring Boot	Contrats de services, repositories	Classes de base partagées
Niveau d'abstraction	100% (contrat pur)	Partiel (code partagé)

INTERFACES : CONTRATS ET ARCHITECTURE MODULAIRE (PARTIE 1/3)

Les interfaces définissent des contrats comportementaux sans implémentation, favorisant le découplage et la testabilité. Spring Boot utilise massivement les interfaces pour structurer les couches applicatives, notamment avec les repositories, services et les API REST. Elles permettent l'injection de dépendances et facilitent le remplacement d'implémentations.

INTERFACES : CONTRATS ET ARCHITECTURE MODULAIRE (PARTIE 2/3)

Architecture en couches avec interfaces Spring Boot (java)

```
1. // Interface de repository (couche d'accès aux données)
2. public interface UtilisateurRepository extends JpaRepository<Utilisateur, Long> {
3.     Optional<Utilisateur> findByEmail(String email);
4.     List<Utilisateur> findByNomContaining(String nom);
5.     boolean existsByEmail(String email);
6. }
7.
8. // Interface de service (couche métier)
9. public interface UtilisateurService {
10.     UtilisateurDTO creerUtilisateur(UtilisateurDTO dto);
11.     UtilisateurDTO modifierUtilisateur(Long id, UtilisateurDTO dto);
12.     void supprimerUtilisateur(Long id);
13.     Optional<UtilisateurDTO> rechercherParEmail(String email);
14.     List<UtilisateurDTO> listerTous();
15. }
16.
17. // Implémentation du service
18. @Service
19. public class UtilisateurServiceImpl implements UtilisateurService {
20.     @Autowired
```

INTERFACES : CONTRATS ET ARCHITECTURE MODULAIRE (PARTIE 2/3) [A]

Architecture en couches avec interfaces Spring Boot (java) - Suite

```
21.     private UtilisateurRepository repository;
22.
23.     @Autowired
24.     private ModelMapper mapper;
25.
26.     @Override
27.     @Transactional
28.     public UtilisateurDTO creerUtilisateur(UtilisateurDTO dto) {
29.         if (repository.existsByEmail(dto.getEmail())) {
30.             throw new BusinessException("Email déjà utilisé");
31.         }
32.         Utilisateur entite = mapper.map(dto, Utilisateur.class);
33.         Utilisateur sauvegarde = repository.save(entite);
34.         return mapper.map(sauvegarde, UtilisateurDTO.class);
35.     }
36.
37.     @Override
38.     public Optional<UtilisateurDTO> rechercherParEmail(String email) {
39.         return repository.findByEmail(email)
40.             .map(u -> mapper.map(u, UtilisateurDTO.class));
```

INTERFACES : CONTRATS ET ARCHITECTURE MODULAIRE (PARTIE 2/3) [B]

Architecture en couches avec interfaces Spring Boot (java) - Suite

```
41.      }  
42.  
43.      // Autres implémentations...  
44. }
```

INTERFACES : CONTRATS ET ARCHITECTURE MODULAIRE (PARTIE 3/3)

Séparation claire entre contrat (interface) et implémentation

Facilite les tests unitaires avec des mocks

Permet l'injection de dépendances Spring

Supporte plusieurs implémentations d'un même contrat

Favorise le principe SOLID (Dependency Inversion)

CLASSES ABSTRAITES : CODE PARTAGÉ ET TEMPLATE METHOD (PARTIE 1/2)

Les classes abstraites permettent de factoriser du code commun tout en imposant l'implémentation de certaines méthodes aux classes filles. Elles sont idéales pour le pattern Template Method où le squelette d'un algorithme est défini dans la classe parente, et les étapes spécifiques sont implémentées par les enfants. Dans Spring Boot, elles servent à créer des classes de base pour les services ou entités.

CLASSES ABSTRAITES : CODE PARTAGÉ ET TEMPLATE METHOD (PARTIE 2/2)

Pattern Template Method avec classe abstraite (java)

```
1. // Classe abstraite définissant un template de traitement
2. public abstract class TraitementDocument {
3.
4.     // Template Method (méthode finale)
5.     public final ResultatTraitement traiter(Document document) {
6.         // Étapes communes
7.         validerDocument(document);
8.
9.         // Étape spécifique (abstraite)
10.        String contenuExtrait = extraireContenu(document);
11.
12.        // Traitement commun
13.        String contenuNettoye = nettoyerContenu(contenuExtrait);
14.
15.        // Étape spécifique (abstraite)
16.        analyserContenu(contenuNettoye);
17.
18.        // Finalisation commune
19.        return genererResultat(contenuNettoye);
20.    }
```


CLASSES ABSTRAITES : CODE PARTAGÉ ET TEMPLATE METHOD (PARTIE 2/2) [A]

Pattern Template Method avec classe abstraite (java) - Suite

```
21.  
22.     // Méthodes concrètes partagées  
23.     protected void validerDocument(Document doc) {  
24.         if (doc == null || doc.getTaille() == 0) {  
25.             throw new IllegalArgumentException("Document invalide");  
26.         }  
27.     }  
28.  
29.     protected String nettoyerContenu(String contenu) {  
30.         return contenu.trim().replaceAll("\\s+", " ");  
31.     }  
32.  
33.     protected ResultatTraitement genererResultat(String contenu) {  
34.         return new ResultatTraitement(contenu, LocalDateTime.now());  
35.     }  
36.  
37.     // Méthodes abstraites à implémenter  
38.     protected abstract String extraireContenu(Document document);  
39.     protected abstract void analyserContenu(String contenu);  
40. }
```

CLASSES ABSTRAITES : CODE PARTAGÉ ET TEMPLATE METHOD (PARTIE 2/2) [B]

Pattern Template Method avec classe abstraite (java) - Suite

```
41.  
42. // Implémentations concrètes  
43. @Component  
44. public class TraitementPDF extends TraitementDocument {  
45.     @Override  
46.     protected String extraireContenu(Document document) {  
47.         // Logique spécifique PDF  
48.         return "Contenu extrait du PDF";  
49.     }  
50.  
51.     @Override  
52.     protected void analyserContenu(String contenu) {  
53.         // Analyse spécifique PDF  
54.         System.out.println("Analyse PDF: " + contenu);  
55.     }  
56. }  
57.  
58. @Component  
59. public class TraitementWord extends TraitementDocument {  
60.     @Override
```

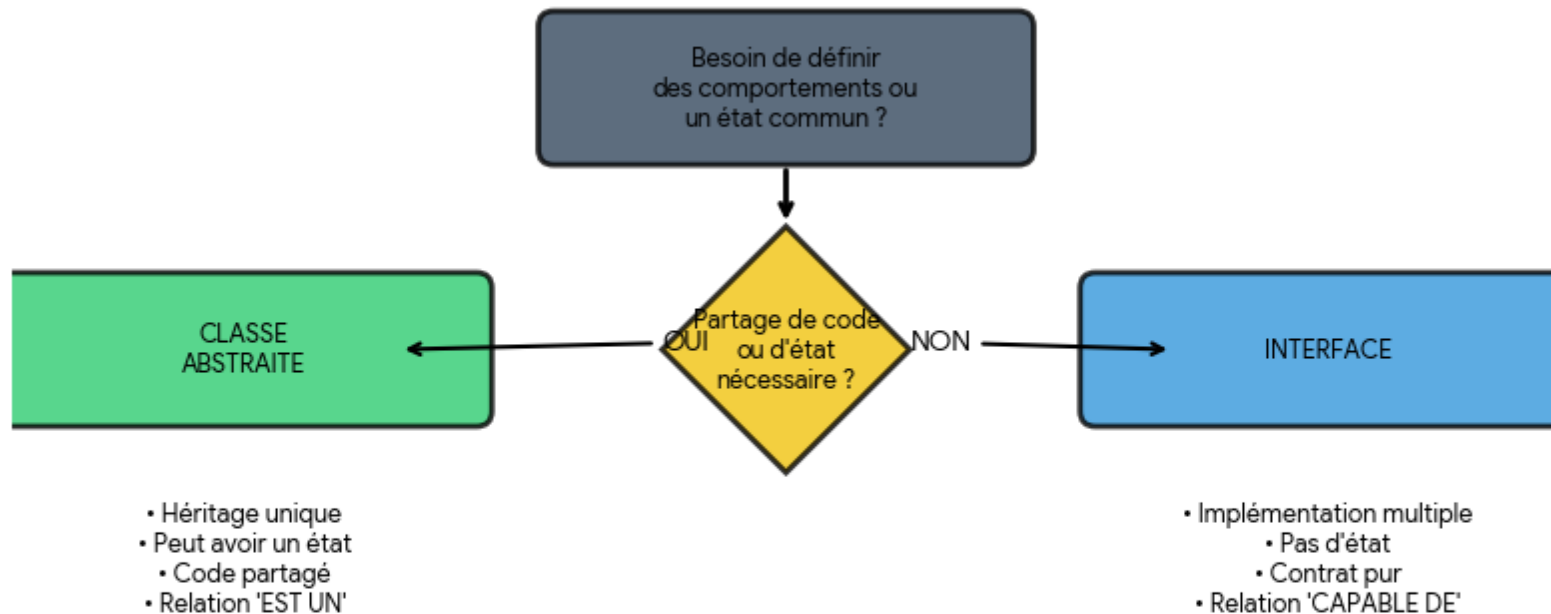
CLASSES ABSTRAITES : CODE PARTAGÉ ET TEMPLATE METHOD (PARTIE 2/2) [C]

Pattern Template Method avec classe abstraite (java) - Suite

```
61.     protected String extraireContenu(Document document) {
62.         // Logique spécifique Word
63.         return "Contenu extrait du Word";
64.     }
65.
66.     @Override
67.     protected void analyserContenu(String contenu) {
68.         // Analyse spécifique Word
69.         System.out.println("Analyse Word: " + contenu);
70.     }
71. }
```

INTERFACES VS CLASSES ABSTRAITES : QUAND UTILISER QUOI ? (PARTIE 1/3)

Interfaces vs Classes Abstraites : Quand Utiliser Quoi ?



INTERFACES VS CLASSES ABSTRAITES : QUAND UTILISER QUOI ? (PARTIE 2/3)

Privilégier les interfaces pour :

- Définir des contrats de services (découplage maximal)
- Permettre l'héritage multiple de comportements
- Faciliter les tests avec mocks
- Respecter le principe SOLID (Dependency Inversion)

Utiliser les classes abstraites pour :

- Partager du code commun entre classes liées
- Implémenter le pattern Template Method
- Définir des classes de base avec état partagé
- Fournir des implémentations par défaut complexes

INTERFACES VS CLASSES ABSTRAITES : QUAND UTILISER QUOI ? (PARTIE 3/3)

Dans Spring Boot, la recommandation générale est de privilégier les interfaces pour les couches de services et repositories afin de maximiser la flexibilité et la testabilité. Les classes abstraites sont réservées aux cas où du code métier significatif doit être partagé entre plusieurs implémentations apparentées.

COLLECTIONS JAVA : VUE D'ENSEMBLE (PARTIE 1/4)

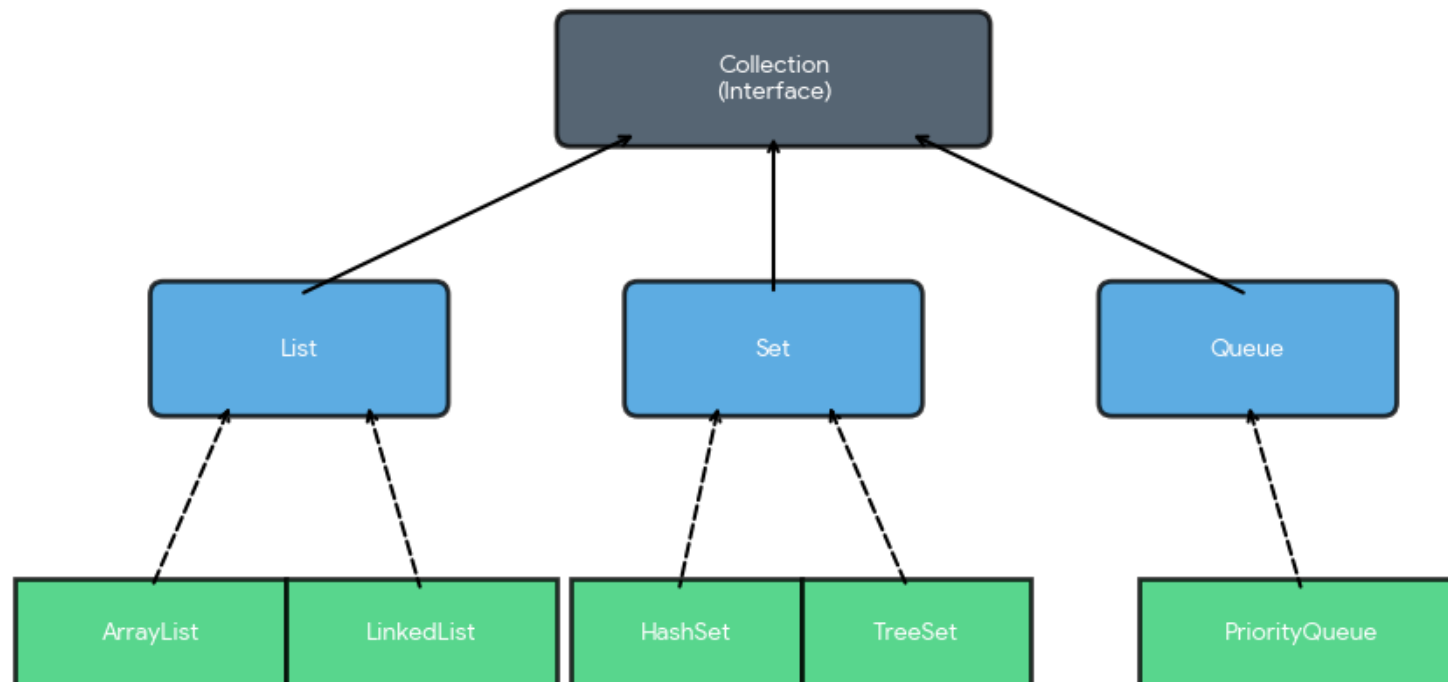
Framework Collections Java

COLLECTIONS JAVA : VUE D'ENSEMBLE (PARTIE 2/4)

Le Java Collections Framework fournit des structures de données optimisées et réutilisables pour stocker et manipuler des groupes d'objets. Les trois interfaces principales sont List (collections ordonnées), Set (collections sans doublons) et Map (associations clé-valeur). Dans Spring Boot, ces collections sont omniprésentes : listes d'entités, ensembles de rôles, maps de configuration, etc.

COLLECTIONS JAVA : VUE D'ENSEMBLE (PARTIE 3/4)

Collections Java : Vue d'Ensemble



Légende : ■ Interface

■ Classe concrète

-.-> Implémente

—> Hérite

COLLECTIONS JAVA : VUE D'ENSEMBLE (PARTIE 4/4)

Comparaison des principales interfaces de collections

Interface	Ordre	Doublons	Null autorisé	Cas d'usage Spring Boot
List	Oui (index)	Oui	Oui	Liste d'entités, résultats de requêtes
Set	Non (sauf LinkedHashSet)	Non	Oui (1 seul)	Rôles utilisateur, tags uniques
Map	Non (sauf LinkedHashMap)	Clés uniques	Oui	Configuration, cache, indexation

LIST : COLLECTIONS ORDONNÉES (PARTIE 1/3)

L'interface `List` représente une séquence ordonnée d'éléments accessibles par index. Les implémentations principales sont `ArrayList` (tableau dynamique, accès rapide), `LinkedList` (liste chaînée, insertion/suppression rapides) et `Vector` (thread-safe mais obsolète). Dans Spring Boot, `List` est utilisée pour les résultats de requêtes JPA, les collections d'entités liées, et les réponses d'API REST.

LIST : COLLECTIONS ORDONNÉES (PARTIE 2/3)

Manipulation de List dans un service Spring Boot (java)

```
1. // Utilisation de List dans un service Spring Boot
2. @Service
3. public class ProduitService {
4.     @Autowired
5.     private ProduitRepository repository;
6.
7.     // Retour de liste depuis repository
8.     public List<ProduitDTO> rechercherParCategorie(String categorie) {
9.         List<Produit> produits = repository.findByCategorie(categorie);
10.
11.         // Transformation avec Stream API
12.         return produits.stream()
13.             .map(this::convertirEnDTO)
14.             .collect(Collectors.toList());
15.     }
16.
17.     // Manipulation de liste avec opérations courantes
18.     public List<ProduitDTO> filtrerEtTrier(List<ProduitDTO> produits,
double prixMin) {
19.         List<ProduitDTO> resultat = new ArrayList<>();
20.
```

LIST : COLLECTIONS ORDONNÉES (PARTIE 2/3) [A]

Manipulation de List dans un service Spring Boot (java) - Suite

```
21.         // Filtrage
22.         for (ProduitDTO p : produits) {
23.             if (p.getPrix() >= prixMin) {
24.                 resultat.add(p);
25.             }
26.         }
27.
28.         // Tri avec Comparator
29.         resultat.sort(Comparator.comparing(ProduitDTO::getPrix).reversed());
30.
31.         return resultat;
32.     }
33.
34.     // Utilisation moderne avec Stream API
35.     public List<String> extraireNomsProduits(List<ProduitDTO> produits) {
36.         return produits.stream()
37.             .map(ProduitDTO::getNom)
38.             .sorted()
39.             .distinct()
40.             .collect(Collectors.toList());
```

LIST : COLLECTIONS ORDONNÉES (PARTIE 2/3) [B]

Manipulation de List dans un service Spring Boot (java) - Suite

```
41.     }
42.
43.     // Pagination avec subList
44.     public List<ProduitDTO> paginer(List<ProduitDTO> produits, int page,
    int taille) {
45.         int debut = page * taille;
46.         int fin = Math.min(debut + taille, produits.size());
47.
48.         if (debut >= produits.size()) {
49.             return Collections.emptyList();
50.         }
51.
52.         return produits.subList(debut, fin);
53.     }
54.
55.     private ProduitDTO convertirEnDTO(Produit p) {
56.         return new ProduitDTO(p.getId(), p.getNom(), p.getPrix());
57.     }
58. }
```

LIST : COLLECTIONS ORDONNÉES (PARTIE 3/3)

ArrayList : choix par défaut (accès $O(1)$, insertion $O(n)$)

LinkedList : pour insertions/suppressions fréquentes en milieu de liste

Méthodes essentielles : `add()`, `get()`, `remove()`, `contains()`, `size()`

Stream API pour transformations fonctionnelles (`map`, `filter`, `collect`)

Pagination avec `subList()` pour grandes collections

SET : COLLECTIONS SANS DOUBLONS

(PARTIE 1/2)

L'interface Set garantit l'unicité des éléments et ne permet pas de doublons. Les implémentations principales sont HashSet (basé sur table de hachage, non ordonné, $O(1)$ pour add/contains), TreeSet (arbre rouge-noir, trié naturellement, $O(\log n)$) et LinkedHashSet (maintient l'ordre d'insertion). Dans Spring Boot, Set est utilisé pour les relations Many-to-Many, les rôles utilisateurs, et les tags.

SET : COLLECTIONS SANS DOUBLONS (PARTIE 2/2)

Exemple de Code (Code)

```
1. // Entité avec relation Many-to-Many utilisant Set
2. @Entity
3. public class Utilisateur {
4.     @Id
5.     @GeneratedValue(strategy = GenerationType.IDENTITY)
6.     private Long id;
7.
8.     private String username;
9.
10.    // Set pour éviter les doublons de rôles
11.    @ManyToMany(fetch = FetchType.EAGER)
12.    @JoinTable(
13.        name = "utilisateur_roles",
14.        joinColumns = @JoinColumn(name = "utilisateur_id"),
15.        inverseJoinColumns = @JoinColumn(name = "role_id")
16.    )
17.    private Set<Role> roles = new HashSet<>();
18.
19.    // Méthodes utilitaires pour gérer les rôles
20.    public void ajouterRole(Role role) {
```

SET : COLLECTIONS SANS DOUBLONS (PARTIE 2/2) [A]

Exemple de Code (Code) - Suite

```
21.         this.roles.add(role); // Pas de doublon grâce au Set
22.     }
23.
24.     public void retirerRole(Role role) {
25.         this.roles.remove(role);
26.     }
27.
28.     public boolean aRole(String nomRole) {
29.         return roles.stream()
30.             .anyMatch(r -> r.getNom().equals(nomRole));
31.     }
32. }
33.
34. // Service utilisant Set pour opérations ensemblistes
35. @Service
36. public class UtilisateurService {
37.
38.     public Set<String> extrairePermissions(Utilisateur utilisateur)
```

GESTION DES EXCEPTIONS EN JAVA

(PARTIE 1/2)

La gestion des exceptions est un mécanisme fondamental en Java permettant de gérer les erreurs de manière structurée et prévisible. Java distingue deux types d'exceptions : les exceptions vérifiées (checked) qui doivent être déclarées ou capturées, et les exceptions non vérifiées (unchecked) héritant de `RuntimeException`.

GESTION DES EXCEPTIONS EN JAVA

(PARTIE 2/2)

Exceptions vérifiées (Checked) : IOException, SQLException

- Doivent être déclarées avec throws ou capturées avec try-catch
- Vérifiées à la compilation

Exceptions non vérifiées (Unchecked) : NullPointerException, IllegalArgumentException

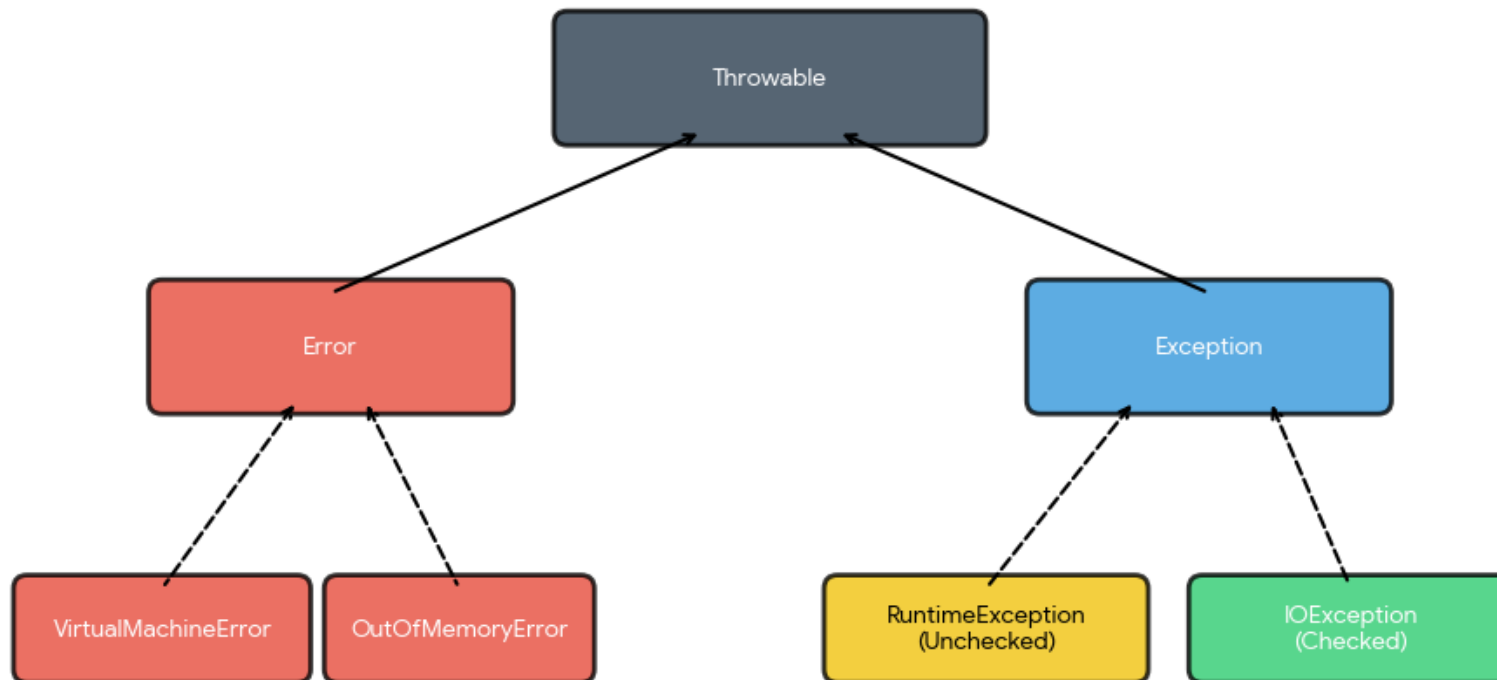
- Héritent de RuntimeException
- Non vérifiées à la compilation

Errors : OutOfMemoryError, StackOverflowError

- Erreurs système graves, généralement non récupérables

HIÉRARCHIE DES EXCEPTIONS JAVA (PARTIE 1/2)

Hiérarchie des Exceptions Java



Légende : ■ Checked (Vérifiée)

■ Unchecked (Non vérifiée)

—▷ Hérite de

HIÉRARCHIE DES EXCEPTIONS JAVA (PARTIE 2/2)

Comparaison : Checked vs Unchecked Exceptions

Critère	Checked Exceptions	Unchecked Exceptions
Classe parente	Exception	RuntimeException
Vérification	À la compilation	À l'exécution
Obligation	Déclaration ou capture obligatoire	Optionnelle
Cas d'usage	Erreurs récupérables externes	Erreurs de programmation
Exemples	IOException, SQLException	NullPointerException, IllegalArgumentException

CRÉATION D'EXCEPTIONS PERSONNALISÉES (PARTIE 1/2)

Les exceptions personnalisées permettent de créer des types d'erreurs spécifiques au domaine métier de l'application. Dans Spring Boot, elles sont essentielles pour implémenter une gestion d'erreurs cohérente et faciliter la communication entre les couches applicatives.

CRÉATION D'EXCEPTIONS PERSONNALISÉES (PARTIE 2/2)

Exception personnalisée pour ressource introuvable (java)

```
1. // Exception métier personnalisée (unchecked)
2. public class ResourceNotFoundException extends RuntimeException {
3.     private String resourceName;
4.     private String fieldName;
5.     private Object fieldValue;
6.
7.     public ResourceNotFoundException(String resourceName,
8.                                     String fieldName,
9.                                     Object fieldValue) {
10.         super(String.format("%s non trouvé avec %s : '%s'",
11.                             resourceName, fieldName, fieldValue));
12.         this.resourceName = resourceName;
13.         this.fieldName = fieldName;
14.         this.fieldValue = fieldValue;
15.     }
16.
17.     // Getters
18.     public String getResourceName() { return resourceName; }
19.     public String getFieldName() { return fieldName; }
20.     public Object getFieldValue() { return fieldValue; }
```


CRÉATION D'EXCEPTIONS PERSONNALISÉES (PARTIE 2/2) [A]

Exception personnalisée pour ressource introuvable (java) - Suite

```
21. }
```

EXCEPTIONS MÉTIER POUR SPRING BOOT (PARTIE 1/2)

Exceptions métier courantes (java)

```
1. // Exception pour validation métier
2. public class BusinessValidationException extends RuntimeException {
3.     private List<String> errors;
4.
5.     public BusinessValidationException(String message) {
6.         super(message);
7.         this.errors = new ArrayList<>();
8.     }
9.
10.    public BusinessValidationException(List<String> errors) {
11.        super("Erreurs de validation métier");
12.        this.errors = errors;
13.    }
14.
15.    public void addError(String error) {
16.        this.errors.add(error);
17.    }
18.
19.    public List<String> getErrors() { return errors; }
20. }
```

EXCEPTIONS MÉTIER POUR SPRING BOOT (PARTIE 1/2) [A]

Exceptions métier courantes (java) - Suite

```
21.  
22. // Exception pour conflit de données  
23. public class DataConflictException extends RuntimeException {  
24.     public DataConflictException(String message) {  
25.         super(message);  
26.     }  
27. }
```

EXCEPTIONS MÉTIER POUR SPRING BOOT (PARTIE 2/2)

Bonnes pratiques pour exceptions personnalisées :

Hériter de `RuntimeException` pour plus de flexibilité

Nommer explicitement selon le contexte métier

Inclure des attributs pour contexte enrichi

Fournir plusieurs constructeurs pour différents cas d'usage

Documenter avec `JavaDoc` quand et pourquoi les lever

BONNES PRATIQUES DE GESTION DES EXCEPTIONS (PARTIE 1/2)

Bonnes et mauvaises pratiques

À FAIRE ✓	À ÉVITER ✗
Capter des exceptions spécifiques	Capter Exception générique
Logger avant de relancer	Avaler les exceptions (catch vide)
Utiliser try-with-resources	Oublier de fermer les ressources
Créer des exceptions métier parlantes	Utiliser des messages génériques
Gérer centralement avec @ControllerAdvice	Dupliquer la gestion dans chaque méthode
Inclure le contexte d'erreur	Exposer des détails techniques sensibles

BONNES PRATIQUES DE GESTION DES EXCEPTIONS (PARTIE 2/2)

Exemples de bonnes et mauvaises pratiques (java)

```
1. // ✓ BONNE PRATIQUE : Try-with-resources
2. public void lireFichier(String chemin) throws IOException {
3.     try (BufferedReader reader = new BufferedReader(
4.         new FileReader(chemin))) {
5.         String ligne = reader.readLine();
6.         // Traitement...
7.     } // Fermeture automatique
8. }
9.
10. // ✗ MAUVAISE PRATIQUE : Catch vide
11. try {
12.     riskyOperation();
13. } catch (Exception e) {
14.     // Ne rien faire - JAMAIS FAIRE ÇA!
15. }
16.
17. // ✓ BONNE PRATIQUE : Log et relance
18. try {
19.     businessOperation();
20. } catch (DataAccessException e) {
```

BONNES PRATIQUES DE GESTION DES EXCEPTIONS (PARTIE 2/2) [A]

Exemples de bonnes et mauvaises pratiques (java) - Suite

```
21.     logger.error("Erreur d'accès données: {}", e.getMessage());
22.     throw new BusinessException("Opération impossible", e);
23. }
```

GESTION CENTRALISÉE AVEC `@ControllerAdvice` (PARTIE 1/2)

Spring Boot offre `@ControllerAdvice` pour centraliser la gestion des exceptions dans les API REST. Cette approche permet de séparer la logique métier de la gestion d'erreurs et de standardiser les réponses HTTP.

GESTION CENTRALISÉE AVEC @CONTROLLERADVICE (PARTIE 2/2)

Gestionnaire global d'exceptions Spring (java)

```
1. @ControllerAdvice
2. public class GlobalExceptionHandler {
3.
4.     @ExceptionHandler(ResourceNotFoundException.class)
5.     public ResponseEntity<ErrorResponse> handleResourceNotFound(
6.         ResourceNotFoundException ex) {
7.         ErrorResponse error = new ErrorResponse(
8.             HttpStatus.NOT_FOUND.value(),
9.             ex.getMessage(),
10.            LocalDateTime.now()
11.        );
12.        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
13.    }
14.
15.    @ExceptionHandler(BusinessValidationException.class)
16.    public ResponseEntity<ErrorResponse> handleValidationError(
17.        BusinessValidationException ex) {
18.        ErrorResponse error = new ErrorResponse(
19.            HttpStatus.BAD_REQUEST.value(),
20.            "Erreurs de validation",
```

GESTION CENTRALISÉE AVEC @CONTROLLERADVICE (PARTIE 2/2) [A]

Gestionnaire global d'exceptions Spring (java) - Suite

```
21.         LocalDateTime.now(),
22.         ex.getErrors()
23.     );
24.     return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
25. }
26.
27. @ExceptionHandler(Exception.class)
28. public ResponseEntity<ErrorResponse> handleGlobalException(
29.     Exception ex) {
30.     ErrorResponse error = new ErrorResponse(
31.         HttpStatus.INTERNAL_SERVER_ERROR.value(),
32.         "Erreur interne du serveur",
33.         LocalDateTime.now()
34.     );
35.     return new ResponseEntity<>(error,
36.                                 HttpStatus.INTERNAL_SERVER_ERROR);
37. }
38. }
```

STRUCTURE DE RÉPONSE D'ERREUR STANDARDISÉE (PARTIE 1/2)

Classe de réponse d'erreur standardisée (java)

```
1. public class ErrorResponse {
2.     private int status;
3.     private String message;
4.     private LocalDateTime timestamp;
5.     private List<String> errors;
6.
7.     // Constructeur pour erreur simple
8.     public ErrorResponse(int status, String message,
9.         LocalDateTime timestamp) {
10.         this.status = status;
11.         this.message = message;
12.         this.timestamp = timestamp;
13.         this.errors = new ArrayList<>();
14.     }
15.
16.     // Constructeur pour erreurs multiples
17.     public ErrorResponse(int status, String message,
18.         LocalDateTime timestamp, List<String> errors) {
19.         this.status = status;
20.         this.message = message;
```

STRUCTURE DE RÉPONSE D'ERREUR STANDARDISÉE (PARTIE 1/2) [A]

Classe de réponse d'erreur standardisée (java) - Suite

```
21.         this.timestamp = timestamp;
22.         this.errors = errors;
23.     }
24.
25.     // Getters et setters
26.     public int getStatus() { return status; }
27.     public String getMessage() { return message; }
28.     public LocalDateTime getTimestamp() { return timestamp; }
29.     public List<String> getErrors() { return errors; }
30. }
```

STRUCTURE DE RÉPONSE D'ERREUR STANDARDISÉE (PARTIE 2/2)

Avantages de la gestion centralisée :

Cohérence des réponses d'erreur dans toute l'API

Séparation des préoccupations (logique métier vs gestion erreurs)

Facilité de maintenance et d'évolution

Logging centralisé des erreurs

Possibilité d'internationalisation des messages

Contrôle des informations exposées au client

INTRODUCTION AUX ANNOTATIONS JAVA (PARTIE 1/3)

Les annotations Java sont des métadonnées ajoutées au code source qui fournissent des informations supplémentaires au compilateur, aux outils de développement ou au runtime. Introduites en Java 5, elles sont devenues fondamentales dans les frameworks modernes comme Spring Boot, permettant une configuration déclarative et réduisant considérablement le code boilerplate.

INTRODUCTION AUX ANNOTATIONS JAVA (PARTIE 2/3)

Rôles principaux des annotations :

Configuration déclarative : Remplacer les fichiers XML de configuration

Injection de dépendances : Gérer automatiquement les dépendances entre composants

Validation : Définir des règles de validation sur les données

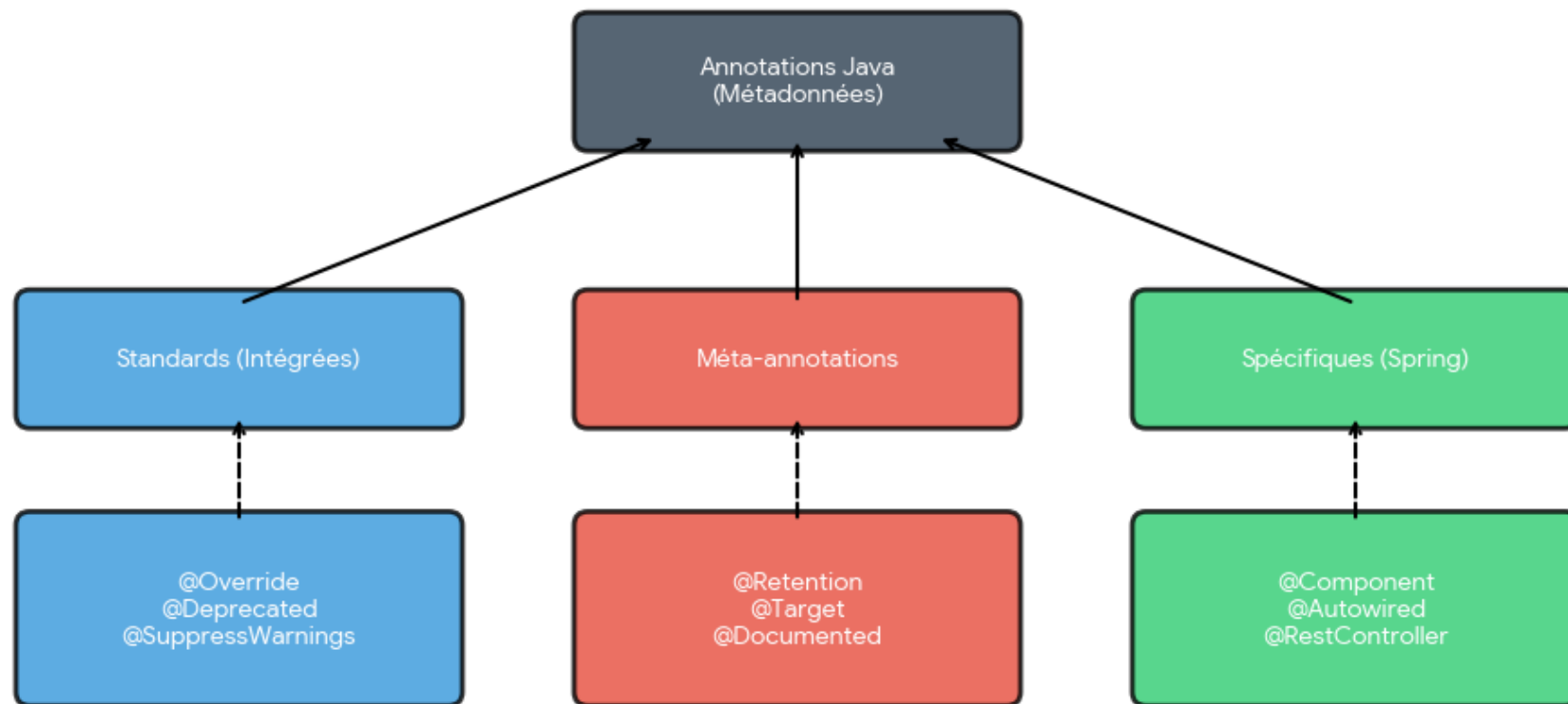
Mapping : Lier des classes aux tables de base de données (JPA)

Routing HTTP : Définir les endpoints REST dans les contrôleurs

Aspect-Oriented Programming : Ajouter des comportements transversaux

INTRODUCTION AUX ANNOTATIONS JAVA (PARTIE 3/3)

Introduction aux Annotations Java



ANNOTATIONS JAVA STANDARDS VS SPRING (PARTIE 1/2)

Catégories d'annotations

Catégorie	Annotations Java Standard	Annotations Spring/Spring Boot
Métadonnées	@Override, @Deprecated, @SuppressWarnings	@Component, @Bean, @Configuration
Injection	@Resource, @Inject (JSR-330)	@Autowired, @Qualifier, @Value
Persistence	@Entity, @Table, @Column (JPA)	@Transactional, @Repository
Validation	@NotNull, @Size, @Email (Bean Validation)	@Validated
Web/REST	@Path (JAX-RS)	@RestController, @GetMapping, @PostMapping
Configuration	-	@SpringBootApplication, @EnableAutoConfiguration

ANNOTATIONS JAVA STANDARDS VS SPRING (PARTIE 2/2)

Spring Boot combine des annotations Java standards (JPA, Bean Validation) avec ses propres annotations pour créer un écosystème cohérent. Les annotations standards garantissent la portabilité, tandis que les annotations Spring apportent des fonctionnalités spécifiques au framework.

ANNOTATIONS DE STÉRÉOTYPE SPRING

Annotations de stéréotype Spring (java)

```
1. // @Component : Composant Spring générique
2. @Component
3. public class EmailService {
4.     public void sendEmail(String to, String message) {
5.         // Logique d'envoi
6.     }
7. }
8.
9. // @Service : Composant de logique métier
10. @Service
11. public class UserService {
12.     @Autowired
13.     private UserRepository userRepository;
14.
15.     public User findById(Long id) {
16.         return userRepository.findById(id)
17.             .orElseThrow(() -> new ResourceNotFoundException(
18.                 "User", "id", id));
19.     }
20. }
```

ANNOTATIONS DE STÉRÉOTYPE SPRING

[A]

Annotations de stéréotype Spring (java) - Suite

```
21.
22. // @Repository : Composant d'accès aux données
23. @Repository
24. public interface UserRepository extends JpaRepository<User, Long> {
25.     Optional<User> findByEmail(String email);
26. }
27.
28. // @Controller : Composant de présentation (retourne des vues)
29. @Controller
30. public class HomeController {
31.     @GetMapping("/")
32.     public String home(Model model) {
33.         return "home"; // Nom de la vue Thymeleaf
34.     }
35. }
36.
37. // @RestController : Contrôleur REST (retourne des données JSON)
38. @RestController
39. @RequestMapping("/api/users")
40. public class UserRestController {
```

ANNOTATIONS DE STÉRÉOTYPE SPRING

[B]

Annotations de stéréotype Spring (java) - Suite

```
41.     @Autowired
42.     private UserService userService;
43.
44.     @GetMapping("/{id}")
45.     public ResponseEntity<User> getUser(@PathVariable Long id) {
46.         return ResponseEntity.ok(userService.findById(id));
47.     }
48. }
```

ANNOTATIONS D'INJECTION DE DÉPENDANCES

Injection de dépendances avec annotations (java)

```
1. // Injection par constructeur (RECOMMANDÉE)
2. @Service
3. public class OrderService {
4.     private final UserService userService;
5.     private final ProductService productService;
6.
7.     @Autowired // Optionnel depuis Spring 4.3 si un seul constructeur
8.     public OrderService(UserService userService,
9.         ProductService productService) {
10.         this.userService = userService;
11.         this.productService = productService;
12.     }
13. }
14.
15. // Injection par setter
16. @Service
17. public class PaymentService {
18.     private EmailService emailService;
19.
20.     @Autowired
```

ANNOTATIONS D'INJECTION DE DÉPENDANCES [A]

Injection de dépendances avec annotations (java) - Suite

```
21.     public void setEmailService(EmailService emailService) {
22.         this.emailService = emailService;
23.     }
24. }
25.
26. // Injection par champ (DÉCONSEILLÉE)
27. @Service
28. public class NotificationService {
29.     @Autowired
30.     private EmailService emailService; // Difficile à tester
31. }
32.
33. // @Qualifier pour résoudre l'ambiguïté
34. @Service
35. public class ReportService {
36.     private final DataSource dataSource;
37.
38.     @Autowired
39.     public ReportService(@Qualifier("primaryDataSource")
40.         DataSource dataSource) {
```

ANNOTATIONS D'INJECTION DE DÉPENDANCES [B]

Injection de dépendances avec annotations (java) - Suite

```
41.         this.dataSource = dataSource;
42.     }
43. }
44.
45. // @Value pour injecter des propriétés
46. @Component
47. public class AppConfig {
48.     @Value("${app.name}")
49.     private String appName;
50.
51.     @Value("${app.max.users:100}") // Valeur par défaut : 100
52.     private int maxUsers;
53. }
```


ANNOTATIONS DE MAPPING HTTP (PARTIE 1/2)

Annotations de mapping REST (java)

```
1. @RestController
2. @RequestMapping("/api/products")
3. public class ProductController {
4.
5.     @Autowired
6.     private ProductService productService;
7.
8.     // GET /api/products
9.     @GetMapping
10.    public List<Product> getAllProducts() {
11.        return productService.findAll();
12.    }
13.
14.    // GET /api/products/{id}
15.    @GetMapping("/{id}")
16.    public ResponseEntity<Product> getProductById(
17.        @PathVariable Long id) {
18.        Product product = productService.findById(id);
19.        return ResponseEntity.ok(product);
20.    }
```

ANNOTATIONS DE MAPPING HTTP (PARTIE 1/2) [A]

Annotations de mapping REST (java) - Suite

```
21.  
22.     // POST /api/products  
23.     @PostMapping  
24.     public ResponseEntity<Product> createProduct(  
25.         @RequestBody @Valid Product product) {  
26.         Product saved = productService.save(product);  
27.         return ResponseEntity.status(HttpStatus.CREATED).body(saved);  
28.     }  
29.  
30.     // PUT /api/products/{id}  
31.     @PutMapping("/{id}")  
32.     public ResponseEntity<Product> updateProduct(  
33.         @PathVariable Long id,  
34.         @RequestBody @Valid Product product) {  
35.         Product updated = productService.update(id, product);  
36.         return ResponseEntity.ok(updated);  
37.     }  
38.  
39.     // DELETE /api/products/{id}  
40.     @DeleteMapping("/{id}")
```

ANNOTATIONS DE MAPPING HTTP (PARTIE 1/2) [B]

Annotations de mapping REST (java) - Suite

```
41.     public ResponseEntity<Void> deleteProduct(@PathVariable Long id) {
42.         productService.delete(id);
43.         return ResponseEntity.noContent().build();
44.     }
45.
46.     // GET /api/products/search?name=laptop&minPrice=500
47.     @GetMapping("/search")
48.     public List<Product> searchProducts(
49.         @RequestParam String name,
50.         @RequestParam(required = false) Double minPrice) {
51.         return productService.search(name, minPrice);
52.     }
53. }
```

ANNOTATIONS DE MAPPING HTTP (PARTIE 2/2)

Annotations de mapping HTTP :

@GetMapping : Requêtes GET (lecture)

@PostMapping : Requêtes POST (création)

@PutMapping : Requêtes PUT (mise à jour complète)

@PatchMapping : Requêtes PATCH (mise à jour partielle)

@DeleteMapping : Requêtes DELETE (suppression)

@RequestMapping : Générique, supporte tous les verbes HTTP

ANNOTATIONS JPA ET PERSISTANCE (PARTIE 1/2)

Annotations JPA pour entités (java)

```
1. @Entity
2. @Table(name = "users")
3. public class User {
4.
5.     @Id
6.     @GeneratedValue(strategy = GenerationType.IDENTITY)
7.     private Long id;
8.
9.     @Column(nullable = false, unique = true, length = 100)
10.    private String email;
11.
12.    @Column(nullable = false)
13.    private String password;
14.
15.    @Column(name = "first_name", length = 50)
16.    private String firstName;
17.
18.    @Column(name = "last_name", length = 50)
19.    private String lastName;
20.
```

ANNOTATIONS JPA ET PERSISTANCE (PARTIE 1/2) [A]

Annotations JPA pour entités (java) - Suite

```
21.     @Enumerated(EnumType.STRING)
22.     private UserRole role;
23.
24.     @Temporal(TemporalType.TIMESTAMP)
25.     @Column(name = "created_at")
26.     private Date createdAt;
27.
28.     @OneToMany(mappedBy = "user", cascade = CascadeType.ALL)
29.     private List<Order> orders = new ArrayList<>();
30.
31.     @ManyToOne(fetch = FetchType.LAZY)
32.     @JoinColumn(name = "department_id")
33.     private Department department;
34.
35.     @Transient // Non persisté en base
36.     private String fullName;
37.
38.     // Constructeurs, getters, setters
39.
40.     @PrePersist
```

ANNOTATIONS JPA ET PERSISTANCE (PARTIE 1/2) [B]

Annotations JPA pour entités (java) - Suite

```
41.     protected void onCreate() {  
42.         createdAt = new Date();  
43.     }  
44. }
```

ANNOTATIONS JPA ET PERSISTANCE (PARTIE 2/2)

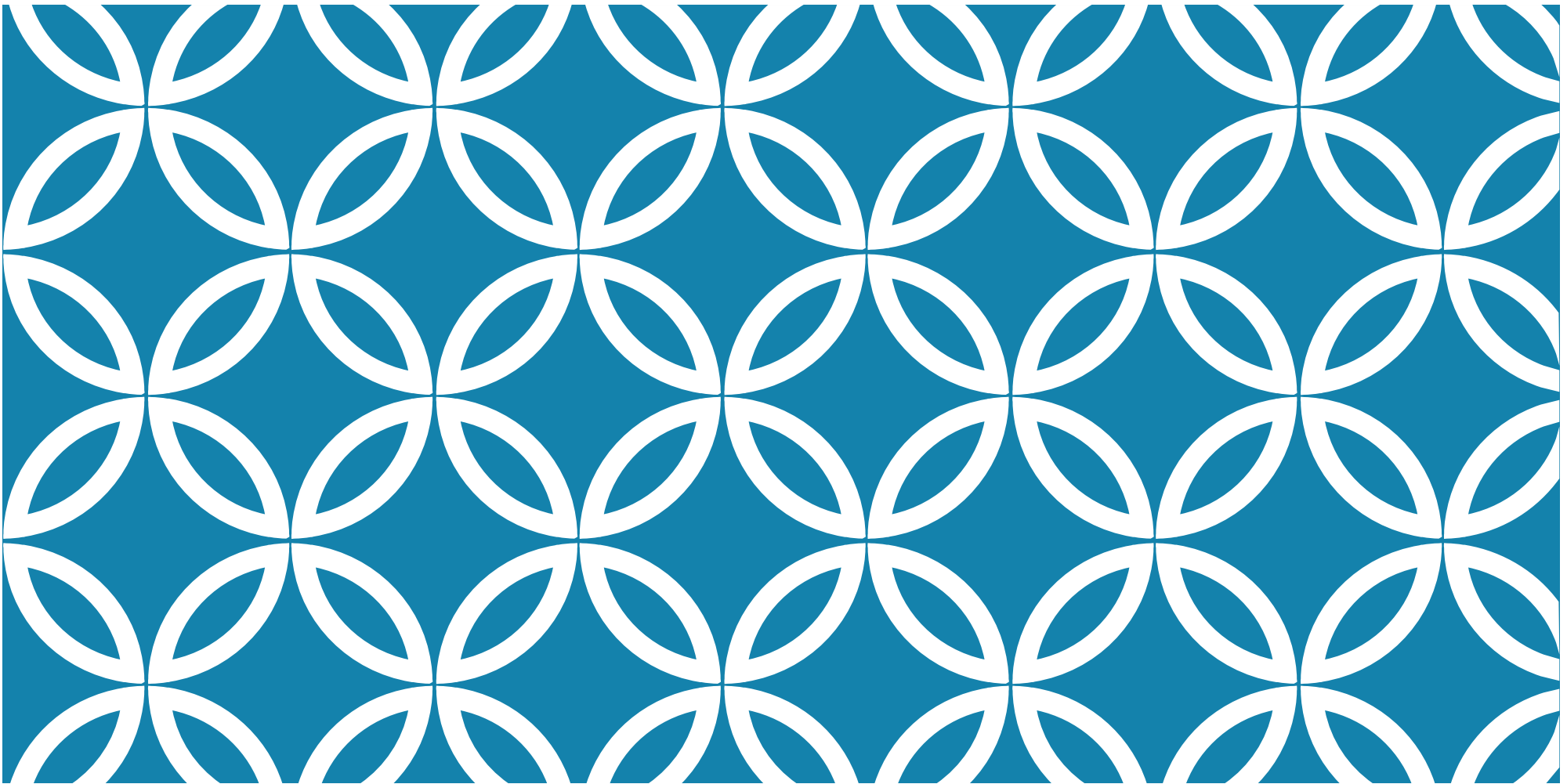
Principales annotations JPA

Annotation	Rôle	Exemple d'usage
@Entity	Marque une classe comme entité JPA	Classe mappée à une table
@Table	Spécifie le nom de la table	@Table(name = "users")
@Id	Définit la clé primaire	Identifiant unique
@GeneratedValue	Stratégie de génération de l'ID	AUTO, IDENTITY, SEQUENCE
@Column	Configure une colonne	nullable, unique, length
@OneToMany	Relation un-à-plusieurs	User → Orders
@ManyToOne	Relation plusieurs-à-un	Orders → User
@ManyToMany	Relation plusieurs-à-plusieurs	Students ↔ Courses
@JoinColumn	Spécifie la colonne de jointure	Clé étrangère
@Transient	Exclut un champ de la persistance	Champs calculés

ANNOTATIONS DE VALIDATION

Exemple de Code (Code)

```
1. import javax.validation.constraints.*;
2. import org.hibernate.validator.constraints.*;
3.
4. @Entity
5. public class UserRegistrationDTO {
6.
7.     @NotNull(message = "L'email ne peut pas être null")
8.     @NotBlank(message = "L'email ne peut pas être vide")
9.     @Email(message = "Format d'email invalide")
10.    @Size(max = 100, message = "L'email ne peut dépasser 100 caractères")
11.    private String email;
12.
13.    @NotBlank(message = "Le mot de passe est obligatoire")
14.    @Size(min = 8, max = 50,
15.        message = "Le mot de passe doit contenir entre 8 et 50 caractères")
16.    @Pattern(regexp = "^(?=.*[A-Z])(?=.*[a-z])(?=.*\\d).*$",
17.        message = "Le mot de passe doit contenir majusc
```



LECTURE 2 : INTRODUCTION À SPRING BOOT ET INJECTION DE DÉPENDANCES

- Présentation de l'écosystème Spring et architecture du framework Spring Boot, - Configuration d'un projet Spring Boot avec Spring Initializr et structure des dépendances Maven/Gradle, - Principe d'I

INTRODUCTION À L'ÉCOSYSTÈME SPRING (PARTIE 1/2)

Spring est un framework Java open-source créé en 2003 pour simplifier le développement d'applications d'entreprise. L'écosystème Spring s'est considérablement enrichi au fil des années pour devenir une plateforme complète offrant des solutions pour tous les aspects du développement moderne : de la gestion des dépendances à la sécurité, en passant par la persistance des données et les microservices.

INTRODUCTION À L'ÉCOSYSTÈME SPRING (PARTIE 2/2)

Spring Framework : Conteneur IoC et fonctionnalités de base (DI, AOP, transactions)

Spring Boot : Configuration automatique et démarrage rapide d'applications

Spring Data : Simplification de l'accès aux données (JPA, MongoDB, Redis)

Spring Security : Gestion de l'authentification et des autorisations

Spring Cloud : Développement de microservices et systèmes distribués

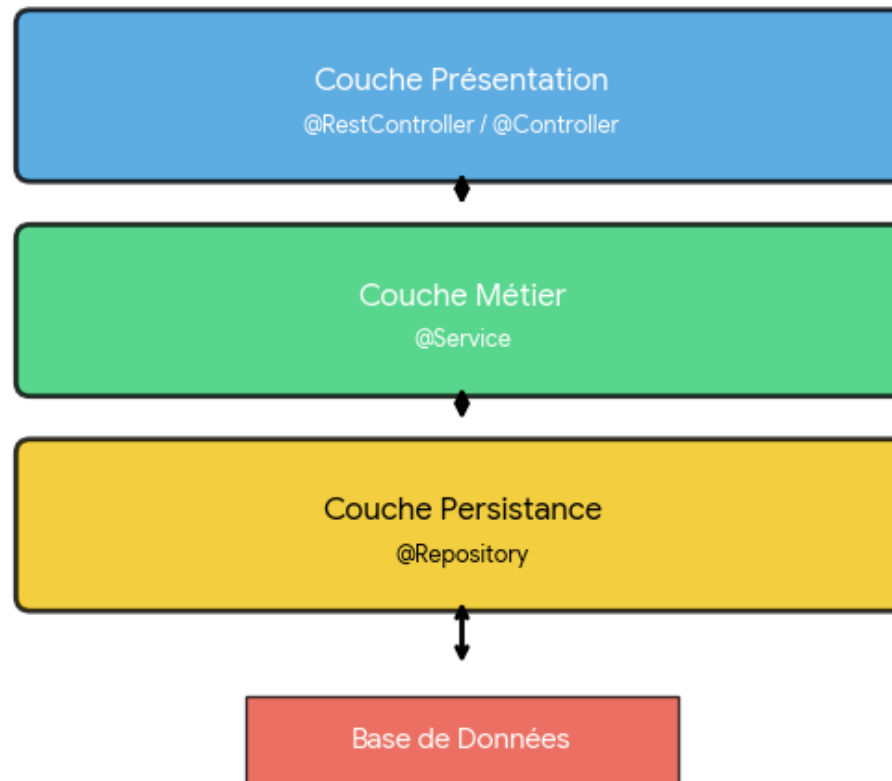
Spring MVC : Framework web pour créer des applications et API REST

ARCHITECTURE DU FRAMEWORK SPRING BOOT (PARTIE 1/3)

Spring Boot repose sur une architecture en couches qui favorise la séparation des responsabilités et facilite la maintenance. Cette architecture suit le pattern MVC (Model-View-Controller) adapté aux applications web modernes et aux API REST.

ARCHITECTURE DU FRAMEWORK SPRING BOOT (PARTIE 2/3)

Architecture en Couches Spring Boot



ARCHITECTURE DU FRAMEWORK SPRING BOOT (PARTIE 3/3)

Couche Controller : Gère les requêtes HTTP et les réponses

- Annotations : @RestController, @Controller, @RequestMapping

Couche Service : Contient la logique métier de l'application

- Annotation : @Service

Couche Repository : Assure la persistance des données

- Annotations : @Repository, interfaces JpaRepository

Couche Model/Entity : Représente les entités métier

- Annotation : @Entity pour les classes JPA

AVANTAGES DE SPRING BOOT

Spring Framework vs Spring Boot

Aspect	Spring Framework	Spring Boot
Configuration	XML ou Java Config manuelle	Auto-configuration intelligente
Démarrage	Configuration serveur externe requise	Serveur embarqué (Tomcat, Jetty)
Dépendances	Gestion manuelle des versions	Starters avec versions compatibles
Temps de setup	Plusieurs heures	Quelques minutes
Fichiers de config	Nombreux fichiers XML/Java	application.properties/yml centralisé
Production-ready	Configuration manuelle	Actuator intégré (métriques, health)

CONFIGURATION AVEC SPRING INITIALIZR (PARTIE 1/3)

Spring Initializr (<https://start.spring.io>) est un générateur de projet web qui permet de créer rapidement la structure de base d'une application Spring Boot avec les dépendances nécessaires. Il génère un projet Maven ou Gradle prêt à l'emploi.

CONFIGURATION AVEC SPRING INITIALIZR (PARTIE 2/3)

Étapes de configuration :

- 1. Choisir le gestionnaire de build (Maven ou Gradle)
- 2. Sélectionner la version de Spring Boot
- 3. Définir les métadonnées du projet (Group, Artifact, Name)
- 4. Choisir la version Java (8, 11, 17, 21)
- 5. Ajouter les dépendances (starters) nécessaires
- 6. Générer et télécharger le projet

Dépendances courantes :

- Spring Web : pour créer des API REST
- Spring Data JPA : pour la persistance
- H2/MySQL/PostgreSQL : bases de données
- Lombok : réduction du code boilerplate
- Spring Boot DevTools : rechargement automatique

CONFIGURATION AVEC SPRING INITIALIZR (PARTIE 3/3)



Project

☒ Gradle - Groovy ☐ Gradle - Kotlin
☐ Maven

Language

☒ Java ☐ Kotlin ☐ Groovy

Spring Boot

☐ 4.1.0 (SNAPSHOT) ☐ 4.1.0 (RC1) ☐ 4.0.7 (SNAPSHOT) ☒ 4.0.6
☐ 3.5.15 (SNAPSHOT) ☐ 3.5.14

Project Metadata

Group

Artifact

Package name

Packaging ☒ Jar ☐ War

Configuration ☒ Properties ☐ YAML

Java ☐ 26 ☐ 25 ☐ 21 ☒ 17

Dependencies

ADD DEPENDENCIES... ⌘ + B

No dependency selected



GENERATE ⌘ + ↵

EXPLORE CTRL + SPACE

...

STRUCTURE D'UN PROJET SPRING BOOT AVEC MAVEN (PARTIE 1/2)

Structure de répertoires d'un projet Spring Boot (text)

```
1. my-spring-boot-app/  
2. |— src/  
3. |   |— main/  
4. |       |— java/  
5. |           |— com/example/demo/  
6. |               |— DemoApplication.java  
7. |               |— controller/  
8. |               |— service/  
9. |               |— repository/  
10. |               |— model/  
11. |           |— resources/  
12. |               |— application.properties  
13. |               |— static/  
14. |               |— templates/  
15. |       |— test/  
16. |           |— java/  
17. |               |— com/example/demo/  
18. |— target/  
19. |— pom.xml  
20. |— mvnw (Maven Wrapper)
```

STRUCTURE D'UN PROJET SPRING BOOT AVEC MAVEN (PARTIE 2/2)

DemoApplication.java : Classe principale avec @SpringBootApplication

src/main/resources : Fichiers de configuration et ressources statiques

application.properties : Configuration centralisée (port, base de données, etc.)

pom.xml : Fichier de configuration Maven avec les dépendances

target/ : Répertoire de compilation (généré automatiquement)

mvnw : Maven Wrapper pour garantir la version Maven utilisée

LE FICHER POM.XML - GESTION DES DÉPENDANCES MAVEN (PARTIE 1/2)

Exemple de fichier pom.xml Spring Boot (xml)

```
1. <?xml version="1.0" encoding="UTF-8"?>
2. <project xmlns="http://maven.apache.org/POM/4.0.0"
3.         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4.         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5.         https://maven.apache.org/xsd/maven-4.0.0.xsd">
6.     <modelVersion>4.0.0</modelVersion>
7.
8.     <!-- Parent Spring Boot pour gérer les versions -->
9.     <parent>
10.         <groupId>org.springframework.boot</groupId>
11.         <artifactId>spring-boot-starter-parent</artifactId>
12.         <version>3.2.0</version>
13.     </parent>
14.
15.     <groupId>com.example</groupId>
16.     <artifactId>demo</artifactId>
17.     <version>0.0.1-SNAPSHOT</version>
18.     <name>demo</name>
19.
20.     <properties>
```

LE FICHIER POM.XML - GESTION DES DÉPENDANCES MAVEN (PARTIE 1/2) [A]

Exemple de fichier pom.xml Spring Boot (xml) - Suite

```
21.         <java.version>17</java.version>
22.     </properties>
23.
24.     <dependencies>
25.         <!-- Starter Web pour API REST -->
26.         <dependency>
27.             <groupId>org.springframework.boot</groupId>
28.             <artifactId>spring-boot-starter-web</artifactId>
29.         </dependency>
30.
31.         <!-- Starter Data JPA pour la persistance -->
32.         <dependency>
33.             <groupId>org.springframework.boot</groupId>
34.             <artifactId>spring-boot-starter-data-jpa</artifactId>
35.         </dependency>
36.     </dependencies>
37. </project>
```

LE FICHER POM.XML - GESTION DES DÉPENDANCES MAVEN (PARTIE 2/2)

Le parent `spring-boot-starter-parent` gère automatiquement les versions compatibles de toutes les dépendances Spring. Les starters sont des dépendances préconfigurées qui incluent tout le nécessaire pour une fonctionnalité spécifique.

GRADLE COMME ALTERNATIVE À MAVEN (PARTIE 1/2)

Comparaison Maven vs Gradle

Critère	Maven	Gradle
Fichier de config	pom.xml (XML)	build.gradle (Groovy/Kotlin DSL)
Verbosité	Plus verbeux	Plus concis
Performance	Build incrémental limité	Build incrémental optimisé
Courbe d'apprentissage	Plus simple pour débutants	Plus complexe mais flexible
Cache	Cache local basique	Cache distribué avancé
Adoption Spring	Standard historique	Recommandé pour nouveaux projets

GRADLE COMME ALTERNATIVE À MAVEN (PARTIE 2/2)

Exemple de fichier build.gradle (gradle)

```
1. plugins {
2.     id 'java'
3.     id 'org.springframework.boot' version '3.2.0'
4.     id 'io.spring.dependency-management' version '1.1.4'
5. }
6.
7. group = 'com.example'
8. version = '0.0.1-SNAPSHOT'
9. sourceCompatibility = '17'
10.
11. repositories {
12.     mavenCentral()
13. }
14.
15. dependencies {
16.     implementation 'org.springframework.boot:spring-boot-starter-web'
17.     implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
18.     testImplementation 'org.springframework.boot:spring-boot-starter-test'
19. }
```

PRINCIPE D'INVERSION DE CONTRÔLE (IOC) (PARTIE 1/3)

L'Inversion de Contrôle (IoC) est un principe de conception où le contrôle du flux d'exécution et de la création des objets est inversé par rapport à la programmation traditionnelle. Au lieu que votre code crée et gère ses dépendances, c'est un conteneur externe (le conteneur Spring) qui s'en charge.

PRINCIPE D'INVERSION DE CONTRÔLE (IOC) (PARTIE 2/3)

Approche Traditionnelle vs IoC

PRINCIPE D'INVERSION DE CONTRÔLE (IOC) (PARTIE 3/3)

Comparaison : Approche traditionnelle vs IoC (java)

```
1. // APPROCHE TRADITIONNELLE - Couplage fort
2. public class UserService {
3.     private UserRepository repository;
4.
5.     public UserService() {
6.         // La classe crée elle-même sa dépendance
7.         this.repository = new UserRepositoryImpl();
8.     }
9.
10.    public User findUser(Long id) {
11.        return repository.findById(id);
12.    }
13. }
14.
15. // APPROCHE IoC - Couplage faible
16. public class UserService {
17.     private UserRepository repository;
18.
19.     // Le conteneur IoC injecte la dépendance
20.     public UserService(UserRepository repository) {
```

PRINCIPE D'INVERSION DE CONTRÔLE (IOC) (PARTIE 3/3) [A]

Comparaison : Approche traditionnelle vs IoC (java) - Suite

```
21.         this.repository = repository;
22.     }
23.
24.     public User findUser(Long id) {
25.         return repository.findById(id);
26.     }
27. }
```

AVANTAGES DE L'INVERSION DE CONTRÔLE (PARTIE 1/2)

Couplage faible : Les classes dépendent d'abstractions, pas d'implémentations concrètes

Testabilité accrue : Facilite l'injection de mocks et stubs pour les tests unitaires

Flexibilité : Changement d'implémentation sans modifier le code client

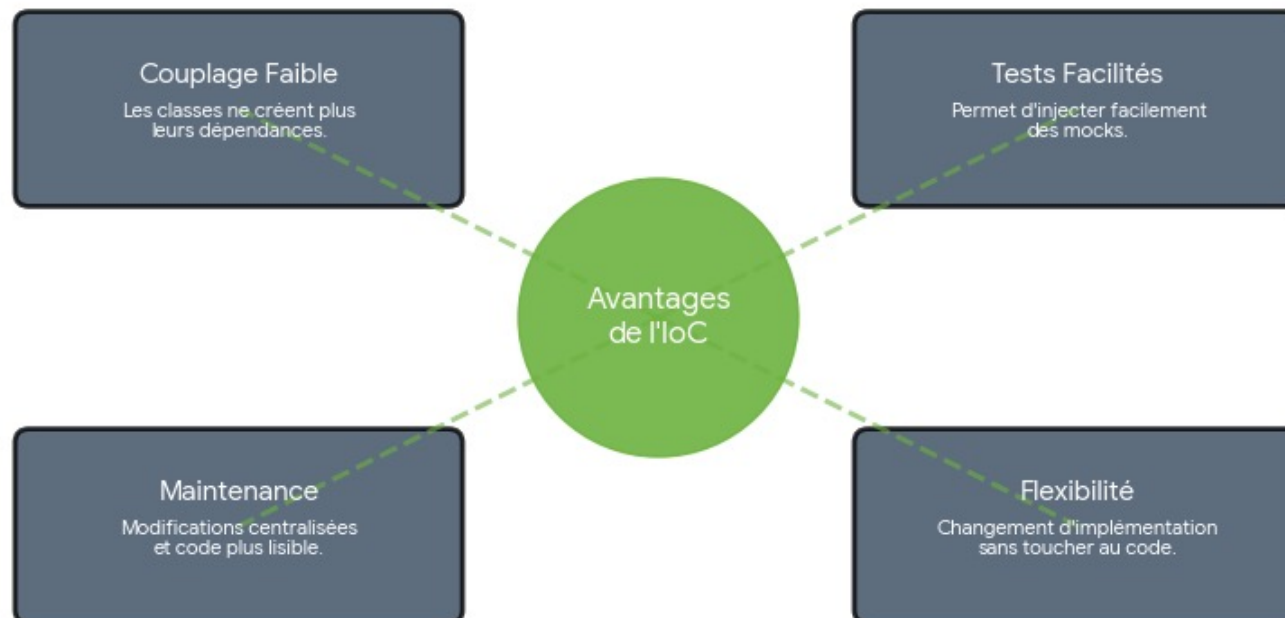
Réutilisabilité : Les composants sont plus modulaires et réutilisables

Maintenabilité : Séparation claire des responsabilités

Configuration centralisée : Gestion des dépendances en un seul endroit

AVANTAGES DE L'INVERSION DE CONTRÔLE (PARTIE 2/2)

Avantages de l'Inversion de Contrôle (IoC)



LE CONTENEUR IOC DE SPRING (PARTIE 1/2)

Le conteneur IoC Spring (ApplicationContext) est le cœur du framework. Il est responsable de l'instanciation, de la configuration et de l'assemblage des beans (objets gérés par Spring). Le conteneur lit les métadonnées de configuration (annotations, XML ou Java Config) pour savoir quels objets créer et comment les connecter.

LE CONTENEUR IOC DE SPRING (PARTIE 2/2)

Responsabilités du conteneur IoC :

- Création et gestion du cycle de vie des beans
- Injection des dépendances entre beans
- Configuration des beans selon les métadonnées
- Gestion des scopes (singleton, prototype, request, session)
- Application de la programmation orientée aspect (AOP)

Types de conteneurs Spring :

- BeanFactory : Conteneur basique (rarement utilisé directement)
- ApplicationContext : Conteneur avancé (recommandé)
 - • ClassPathXmlApplicationContext
 - • AnnotationConfigApplicationContext
 - • WebApplicationContext (pour applications web)

MÉCANISME D'INJECTION DE DÉPENDANCES (PARTIE 1/2)

L'Injection de Dépendances (DI) est l'implémentation concrète du principe IoC. C'est le processus par lequel Spring fournit aux objets leurs dépendances au lieu que ces objets les créent eux-mêmes. Spring supporte trois types d'injection de dépendances.

MÉCANISME D'INJECTION DE DÉPENDANCES (PARTIE 2/2)

Types d'Injection de Dépendances

Type	Syntaxe	Avantages	Inconvénients	Recommandation
Injection par Constructeur	@Autowired sur constructeur	Immutabilité, dépendances obligatoires, facilite tests	Verbeux si nombreuses dépendances	✓ RECOMMANDÉ
Injection par Setter	@Autowired sur setter	Dépendances optionnelles, reconfiguration possible	Mutabilité, peut créer états incohérents	Usage limité
Injection par Champ	@Autowired sur attribut	Code concis, moins verbeux	Difficile à tester, viole encapsulation	✗ À ÉVITER

INJECTION PAR CONSTRUCTEUR - EXEMPLE PRATIQUE (PARTIE 1/2)

Injection par Constructeur - Pattern Recommandé (java)

```
1. // Interface Repository
2. public interface UserRepository {
3.     User findById(Long id);
4.     void save(User user);
5. }
6.
7. // Implémentation Repository
8. @Repository
9. public class UserRepositoryImpl implements UserRepository {
10.     @Override
11.     public User findById(Long id) {
12.         // Logique d'accès base de données
13.         return new User(id, "John Doe");
14.     }
15.
16.     @Override
17.     public void save(User user) {
18.         // Logique de sauvegarde
19.     }
20. }
```

INJECTION PAR CONSTRUCTEUR - EXEMPLE PRATIQUE (PARTIE 1/2) [A]

Injection par Constructeur - Pattern Recommandé (java) - Suite

```
21.  
22. // Service avec injection par constructeur  
23. @Service  
24. public class UserService {  
25.     private final UserRepository userRepository;  
26.  
27.     // @Autowired optionnel si un seul constructeur  
28.     public UserService(UserRepository userRepository) {  
29.         this.userRepository = userRepository;  
30.     }  
31.  
32.     public User getUserById(Long id) {  
33.         return userRepository.findById(id);  
34.     }  
35. }
```

INJECTION PAR CONSTRUCTEUR - EXEMPLE PRATIQUE (PARTIE 2/2)

Avantages de cet exemple :

- UserService est immutable (final sur la dépendance)
- La dépendance est obligatoire (constructeur)
- Facilite les tests unitaires (injection manuelle possible)
- Respect du principe de responsabilité unique
- Spring détecte et injecte automatiquement UserRepositoryImpl

INJECTION PAR SETTER ET PAR CHAMP (PARTIE 1/2)

Injection par Setter et par Champ (java)

```
1. // INJECTION PAR SETTER - Pour dépendances optionnelles
2. @Service
3. public class EmailService {
4.     private EmailValidator validator;
5.
6.     @Autowired(required = false) // Dépendance optionnelle
7.     public void setValidator(EmailValidator validator) {
8.         this.validator = validator;
9.     }
10.
11.     public void sendEmail(String email) {
12.         if (validator != null) {
13.             validator.validate(email);
14.         }
15.         // Logique d'envoi
16.     }
17. }
18.
19. // INJECTION PAR CHAMP - À éviter en production
20. @Service
```


INJECTION PAR SETTER ET PAR CHAMP (PARTIE 1/2) [A]

Injection par Setter et par Champ (java) - Suite

```
21. public class NotificationService {
22.     @Autowired
23.     private EmailService emailService; // Injection directe
24.
25.     @Autowired
26.     private SmsService smsService;
27.
28.     public void notifyUser(User user) {
29.         emailService.sendEmail(user.getEmail());
30.         smsService.sendSms(user.getPhone());
31.     }
32. }
```

INJECTION PAR SETTER ET PAR CHAMP (PARTIE 2/2)

L'injection par champ est tentante car concise, mais elle rend les tests difficiles (nécessite réflexion ou framework de test) et cache les dépendances de la classe. L'injection par setter est utile pour les dépendances optionnelles ou la reconfiguration dynamique, mais doit être utilisée avec précaution.

ANNOTATIONS DE STÉRÉOTYPES SPRING (PARTIE 1/3)

Spring utilise des annotations de stéréotype pour identifier et enregistrer automatiquement les beans dans le conteneur IoC. Ces annotations sont détectées lors du component scanning et permettent une configuration par convention.

ANNOTATIONS DE STÉRÉOTYPES SPRING

(PARTIE 2/3)

Annotations de Stéréotypes Spring

Annotation	Couche	Usage	Fonctionnalités Spécifiques
@Component	Générique	Bean Spring générique	Détection automatique par component scan
@Service	Service/Métier	Logique métier	Identique à @Component, sémantique métier
@Repository	Persistence	Accès aux données	Traduction automatique exceptions JDBC/JPA
@Controller	Présentation	Contrôleur MVC	Gestion des vues (Thymeleaf, JSP)
@RestController	API REST	Contrôleur REST	@Controller + @ResponseBody automatique
@Configuration	Configuration	Classe de config Java	Définition de beans avec @Bean

ANNOTATIONS DE STÉRÉOTYPES SPRING (PARTIE 3/3)

Utilisation des annotations de stéréotypes (java)

```
1. @Service // Bean géré par Spring
2. public class ProductService {
3.     // ...
4. }
5.
6. @Repository // Bean avec gestion d'exceptions
7. public class ProductRepository {
8.     // ...
9. }
10.
11. @RestController // Contrôleur REST
12. @RequestMapping("/api/products")
13. public class ProductController {
14.     // ...
15. }
```

CONFIGURATION DU COMPONENT SCANNING

Exemple de Code (Code)

```
1. package com.example.demo;
2.
3. import org.springframework.boot.SpringApplication;
4. import org.springframework.boot.autoconfigure.SpringBootApplication;
5.
6. // @SpringBootApplication combine 3 annotations :
7. // - @Configuration : classe de configuration
8. // - @EnableAutoConfiguration : active l'auto-configuration
9. // - @ComponentScan : scan du package et sous-packages
10. @SpringBootApplication
11. public class DemoApplication {
12.     public static void main(String[] args) {
13.         SpringApplication.run(DemoApplication.
```

ANNOTATIONS SPRING ESSENTIELLES : VUE D'ENSEMBLE (PARTIE 1/3)

Spring Boot utilise un système d'annotations pour identifier et gérer automatiquement les composants de l'application. Ces annotations permettent au conteneur IoC de détecter, instancier et injecter les beans nécessaires. Elles constituent la base de la programmation déclarative dans Spring.

ANNOTATIONS SPRING ESSENTIELLES : VUE D'ENSEMBLE (PARTIE 2/3)

`@Component` : Annotation générique pour tout composant Spring

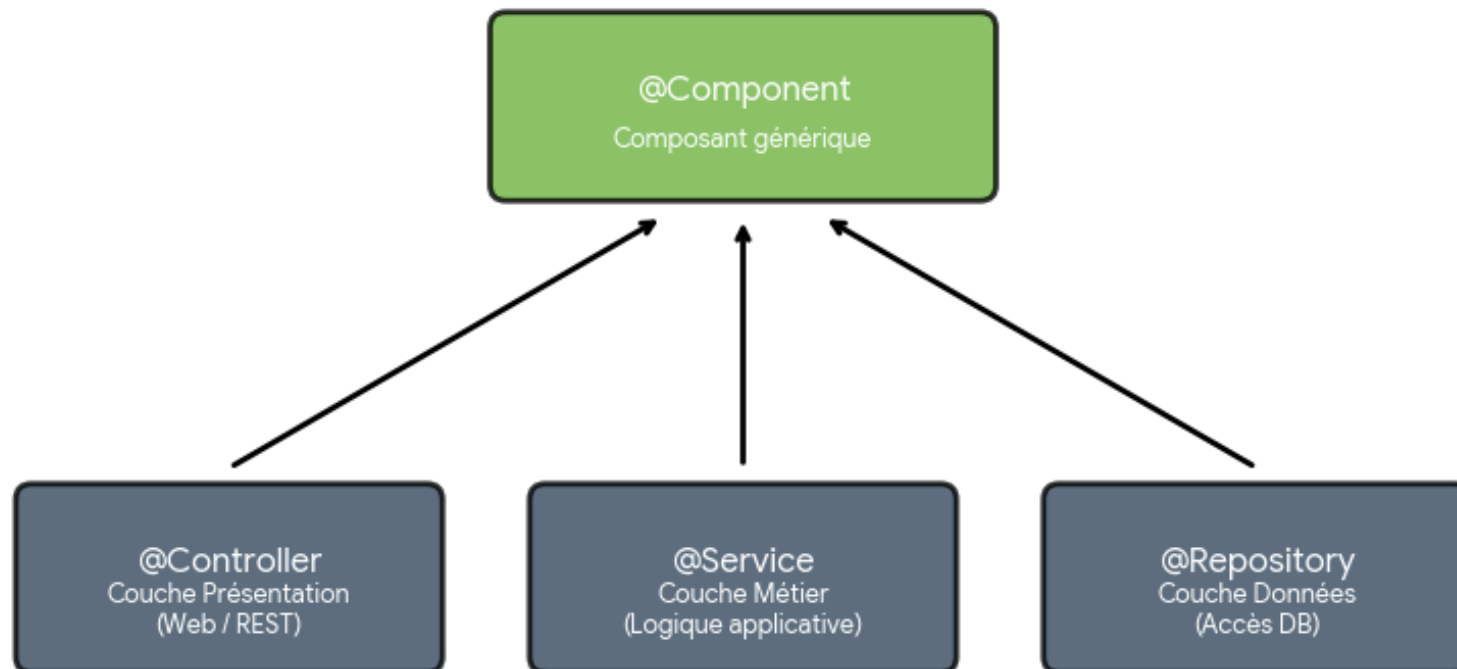
`@Service` : Spécialisation pour la couche métier (logique applicative)

`@Repository` : Spécialisation pour la couche d'accès aux données (DAO)

`@Autowired` : Injection automatique de dépendances par Spring

ANNOTATIONS SPRING ESSENTIELLES : VUE D'ENSEMBLE (PARTIE 3/3)

Hiérarchie des Annotations Stéréotypes Spring



@COMPONENT : COMPOSANT GÉNÉRIQUE (PARTIE 1/3)

@Component est l'annotation de base pour déclarer une classe comme bean Spring. Elle indique au conteneur IoC que cette classe doit être gérée automatiquement. Spring détecte ces classes lors du component scanning et les instancie comme beans singleton par défaut.

@COMPONENT : COMPOSANT GÉNÉRIQUE (PARTIE 2/3)

Exemple d'utilisation de @Component (java)

```
1. package com.example.demo.utils;
2.
3. import org.springframework.stereotype.Component;
4.
5. @Component
6. public class EmailValidator {
7.
8.     public boolean isValid(String email) {
9.         if (email == null || email.isEmpty()) {
10.             return false;
11.         }
12.         return email.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$");
13.     }
14.
15.     public String normalize(String email) {
16.         return email != null ? email.trim().toLowerCase() : "";
17.     }
18. }
```

@COMPONENT : COMPOSANT GÉNÉRIQUE (PARTIE 3/3)

Utilisé pour les classes utilitaires, helpers ou composants génériques

Peut spécifier un nom de bean :
`@Component("monValideur")`

Détecté automatiquement si le package est scanné
(`@ComponentScan`)

Instancié en singleton par défaut (une seule instance partagée)

@SERVICE : COUCHE MÉTIER (PARTIE 1/3)

@Service est une spécialisation de @Component dédiée à la couche métier (business logic). Elle identifie les classes qui contiennent la logique applicative, les règles métier et orchestrent les opérations entre le contrôleur et la couche d'accès aux données.

@SERVICE : COUCHE MÉTIER (PARTIE 2/3)

Exemple de classe Service avec logique métier (java)

```
1. package com.example.demo.service;
2.
3. import org.springframework.stereotype.Service;
4. import org.springframework.beans.factory.annotation.Autowired;
5. import com.example.demo.repository.UserRepository;
6. import com.example.demo.model.User;
7. import java.util.List;
8.
9. @Service
10. public class UserService {
11.
12.     @Autowired
13.     private UserRepository userRepository;
14.
15.     @Autowired
16.     private EmailValidator emailValidator;
17.
18.     public User createUser(String email, String name) {
19.         // Logique métier : validation
20.         if (!emailValidator.isValid(email)) {
```

@SERVICE : COUCHE MÉTIER (PARTIE 2/3) [A]

Exemple de classe Service avec logique métier (java) - Suite

```
21.         throw new IllegalArgumentException("Email invalide");
22.     }
23.
24.     // Logique métier : vérification unicité
25.     if (userRepository.existsByEmail(email)) {
26.         throw new IllegalStateException("Email déjà utilisé");
27.     }
28.
29.     // Création et sauvegarde
30.     User user = new User();
31.     user.setEmail(emailValidator.normalize(email));
32.     user.setName(name);
33.
34.     return userRepository.save(user);
35. }
36.
37. public List<User> getAllActiveUsers() {
38.     return userRepository.findByActiveTrue();
39. }
40. }
```

@SERVICE : COUCHE MÉTIER (PARTIE 3/3)

Contient la logique métier et les règles applicatives

Orchestre les appels entre contrôleurs et repositories

Gère les transactions et les validations métier

Favorise la séparation des responsabilités (SRP)

Facilite les tests unitaires avec des mocks

@REPOSITORY : COUCHE D'ACCÈS AUX DONNÉES (PARTIE 1/3)

@Repository est une spécialisation de @Component pour la couche d'accès aux données (DAO - Data Access Object). Elle marque les classes responsables de l'interaction avec la base de données et active la traduction automatique des exceptions de persistance en exceptions Spring.

@REPOSITORY : COUCHE D'ACCÈS AUX DONNÉES (PARTIE 2/3)

Exemple de Repository avec Spring Data JPA (java)

```
1. package com.example.demo.repository;
2.
3. import org.springframework.data.jpa.repository.JpaRepository;
4. import org.springframework.stereotype.Repository;
5. import com.example.demo.model.User;
6. import java.util.List;
7. import java.util.Optional;
8.
9. @Repository
10. public interface UserRepository extends JpaRepository<User, Long> {
11.
12.     // Méthodes de requête dérivées automatiquement
13.     Optional<User> findByEmail(String email);
14.
15.     boolean existsByEmail(String email);
16.
17.     List<User> findByActiveTrue();
18.
19.     List<User> findByNameContainingIgnoreCase(String name);
20.
```

@REPOSITORY : COUCHE D'ACCÈS AUX DONNÉES (PARTIE 2/3) [A]

Exemple de Repository avec Spring Data JPA (java) - Suite

```
21.      // Requête personnalisée avec @Query (optionnel)
22.      @Query("SELECT u FROM User u WHERE u.createdDate > :date")
23.      List<User> findRecentUsers(@Param("date") LocalDateTime date);
24. }
```

@REPOSITORY : COUCHE D'ACCÈS AUX DONNÉES (PARTIE 3/3)

Encapsule la logique d'accès aux données (CRUD et requêtes)

Traduction automatique des exceptions de persistance (SQLException → DataAccessException)

Avec Spring Data JPA : pas besoin d'implémentation, juste l'interface

Supporte les méthodes de requête dérivées du nom

Permet l'utilisation de @Query pour des requêtes personnalisées

COMPARAISON DES ANNOTATIONS STÉRÉOTYPES (PARTIE 1/2)

Annotations stéréotypes Spring : caractéristiques et usages

Annotation	Couche	Usage principal	Fonctionnalités spécifiques
@Component	Générique	Composants utilitaires, helpers, classes génériques	Détection automatique par component scan
@Service	Métier (Business)	Logique applicative, règles métier, orchestration	Clarté sémantique, support transactionnel
@Repository	Accès données (DAO)	Interaction avec la base de données, requêtes	Traduction d'exceptions, intégration Spring Data
@Controller	Présentation (Web)	Gestion des requêtes HTTP, endpoints MVC	Mapping de routes, gestion de vues (vu plus tard)

COMPARAISON DES ANNOTATIONS STÉRÉOTYPES (PARTIE 2/2)

Bien que techniquement interchangeables (toutes héritent de `@Component`), ces annotations apportent une clarté architecturale essentielle. Elles permettent de structurer l'application selon le pattern en couches et facilitent la maintenance en identifiant clairement le rôle de chaque classe.

| @AUTOWIRED : INJECTION DE DÉPENDANCES (PARTIE 1/3)

@Autowired est l'annotation centrale pour l'injection de dépendances dans Spring. Elle indique à Spring d'injecter automatiquement les beans nécessaires, éliminant ainsi le besoin de créer manuellement les instances. Spring résout les dépendances par type (by type) lors de l'initialisation du contexte.

@AUTOWIRED : INJECTION DE DÉPENDANCES (PARTIE 2/3)

Trois modes d'injection avec @Autowired

@AUTOWIRED : INJECTION DE DÉPENDANCES (PARTIE 3/3)

Les trois types d'injection @Autowired (java)

```
1. // 1. INJECTION PAR CONSTRUCTEUR (RECOMMANDÉE)
2. @Service
3. public class OrderService {
4.     private final ProductRepository productRepository;
5.     private final PaymentService paymentService;
6.
7.     @Autowired // Optionnel si un seul constructeur
8.     public OrderService(ProductRepository productRepository,
9.         PaymentService paymentService) {
10.         this.productRepository = productRepository;
11.         this.paymentService = paymentService;
12.     }
13. }
14.
15. // 2. INJECTION PAR SETTER
16. @Service
17. public class NotificationService {
18.     private EmailService emailService;
19.
20.     @Autowired
```

@AUTOWIRED : INJECTION DE DÉPENDANCES (PARTIE 3/3) [A]

Les trois types d'injection @Autowired (java) - Suite

```
21.     public void setEmailService(EmailService emailService) {
22.         this.emailService = emailService;
23.     }
24. }
25.
26. // 3. INJECTION PAR CHAMP (À ÉVITER EN PRODUCTION)
27. @Service
28. public class ReportService {
29.     @Autowired
30.     private DataAnalyzer dataAnalyzer; // Difficile à tester
31. }
```

@AUTOWIRED : GESTION DES CAS PARTICULIERS (PARTIE 1/2)

Cas avancés d'injection avec @Autowired (java)

```
1. // Dépendance optionnelle
2. @Service
3. public class CacheService {
4.     private final RedisClient redisClient;
5.
6.     @Autowired(required = false) // Ne lève pas d'erreur si absent
7.     public CacheService(RedisClient redisClient) {
8.         this.redisClient = redisClient;
9.     }
10. }
11.
12. // Résolution d'ambiguïté avec @Qualifier
13. @Service
14. public class PaymentProcessor {
15.     private final PaymentGateway gateway;
16.
17.     @Autowired
18.     public PaymentProcessor(@Qualifier("stripeGateway") PaymentGateway gateway) {
19.         this.gateway = gateway;
20.     }
```

@AUTOWIRED : GESTION DES CAS PARTICULIERS (PARTIE 1/2) [A]

Cas avancés d'injection avec @Autowired (java) - Suite

```
21. }
22.
23. // Injection de multiples beans du même type
24. @Service
25. public class NotificationDispatcher {
26.     private final List<NotificationChannel> channels;
27.
28.     @Autowired
29.     public NotificationDispatcher(List<NotificationChannel> channels) {
30.         this.channels = channels; // Injecte TOUS les beans NotificationChannel
31.     }
32. }
```

@AUTOWIRED : GESTION DES CAS PARTICULIERS (PARTIE 2/2)

`required = false` : rend la dépendance optionnelle (null si bean absent)

`@Qualifier` : résout l'ambiguïté quand plusieurs beans du même type existent

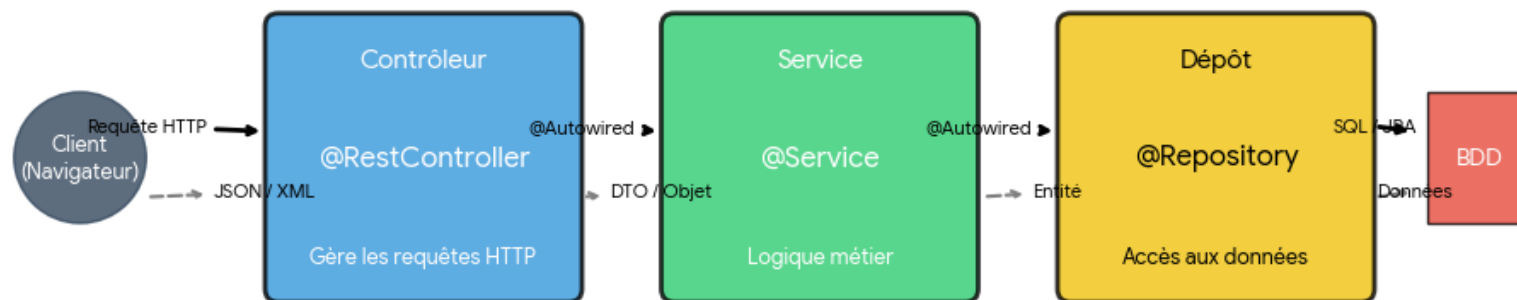
Injection de `List/Map` : récupère tous les beans d'un type donné

`@Primary` : désigne le bean par défaut en cas de multiples candidats

`Optional<T>` : alternative moderne à `required = false`

ARCHITECTURE COMPLÈTE AVEC ANNOTATIONS SPRING (PARTIE 1/2)

Architecture Complète et Flux d'Annotations Spring Boot



ARCHITECTURE COMPLÈTE AVEC ANNOTATIONS SPRING (PARTIE 2/2)

Architecture complète avec toutes les annotations (java)

```
1. // Exemple complet d'architecture
2.
3. @RestController
4. @RequestMapping("/api/users")
5. public class UserController {
6.     private final UserService userService;
7.
8.     @Autowired
9.     public UserController(UserService userService) {
10.         this.userService = userService;
11.     }
12.
13.     @PostMapping
14.     public ResponseEntity<User> createUser(@RequestBody UserDTO dto) {
15.         return ResponseEntity.ok(userService.createUser(dto.getEmail(),
16.             dto.getName()));
17.     }
18.
19. @Service
20. public class UserService {
```

ARCHITECTURE COMPLÈTE AVEC ANNOTATIONS SPRING (PARTIE 2/2) [A]

Architecture complète avec toutes les annotations (java) - Suite

```
21.     private final UserRepository userRepository;
22.     private final EmailValidator emailValidator;
23.
24.     @Autowired
25.     public UserService(UserRepository userRepository, EmailValidator
emailValidator) {
26.         this.userRepository = userRepository;
27.         this.emailValidator = emailValidator;
28.     }
29.
30.     public User createUser(String email, String name) {
31.         if (!emailValidator.isValid(email)) {
32.             throw new IllegalArgumentException("Email invalide");
33.         }
34.         User user = new User(email, name);
35.         return userRepository.save(user);
36.     }
37. }
38.
39. @Repository
40. public interface UserRepository extends JpaRepository<User, Long> {
```


ARCHITECTURE COMPLÈTE AVEC ANNOTATIONS SPRING (PARTIE 2/2) [B]

Architecture complète avec toutes les annotations (java) - Suite

```
41.     Optional<User> findByEmail(String email);
42. }
43.
44. @Component
45. public class EmailValidator {
46.     public boolean isValid(String email) {
47.         return email.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$");
48.     }
49. }
```

CONFIGURATION AVEC APPLICATION.PROPERTIES (PARTIE 1/3)

Le fichier `application.properties` est le fichier de configuration principal de Spring Boot. Situé dans `src/main/resources`, il permet de configurer tous les aspects de l'application : base de données, serveur, logging, propriétés personnalisées, etc. Spring Boot charge automatiquement ce fichier au démarrage.

CONFIGURATION AVEC APPLICATION.PROPERTIES (PARTIE 2/3)

Exemple de fichier application.properties (properties)

```
1. # Configuration du serveur
2. server.port=8080
3. server.servlet.context-path=/api
4.
5. # Configuration de la base de données
6. spring.datasource.url=jdbc:mysql://localhost:3306/mydb
7. spring.datasource.username=root
8. spring.datasource.password=secret
9. spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
10.
11. # Configuration JPA/Hibernate
12. spring.jpa.hibernate.ddl-auto=update
13. spring.jpa.show-sql=true
14. spring.jpa.properties.hibernate.format_sql=true
15. spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
16.
17. # Configuration du logging
18. logging.level.root=INFO
19. logging.level.com.example.demo=DEBUG
20. logging.file.name=logs/application.log
```

CONFIGURATION AVEC APPLICATION.PROPERTIES (PARTIE 2/3) [A]

Exemple de fichier application.properties (properties) - Suite

```
21.  
22. # Propriétés personnalisées  
23. app.name=Mon Application  
24. app.version=1.0.0  
25. app.max-upload-size=10MB  
26. app.email.from=noreply@example.com
```

CONFIGURATION AVEC APPLICATION.PROPERTIES (PARTIE 3/3)

Format clé=valeur simple et lisible

Chargement automatique au démarrage de l'application

Supporte les profils (application-dev.properties, application-prod.properties)

Peut être surchargé par variables d'environnement ou arguments JVM

Autocomplétion disponible dans les IDE modernes

CONFIGURATION AVEC APPLICATION.YML (PARTIE 1/3)

Le fichier `application.yml` (ou `.yaml`) est une alternative moderne à `application.properties`. Il utilise le format YAML (Yet Another Markup Language) qui offre une structure hiérarchique plus lisible et concise, particulièrement appréciée pour les configurations complexes avec de nombreuses propriétés imbriquées.

CONFIGURATION AVEC APPLICATION.YML

(PARTIE 2/3)

Même configuration en format YAML (yaml)

```
1. # Configuration du serveur
2. server:
3.   port: 8080
4.   servlet:
5.     context-path: /api
6.
7. # Configuration de la base de données
8. spring:
9.   datasource:
10.    url: jdbc:mysql://localhost:3306/mydb
11.    username: root
12.    password: secret
13.    driver-class-name: com.mysql.cj.jdbc.Driver
14.
15. # Configuration JPA/Hibernate
16. jpa:
17.   hibernate:
18.     ddl-auto: update
19.     show-sql: true
20.   properties:
```

CONFIGURATION AVEC APPLICATION.YML (PARTIE 2/3) [A]

Même configuration en format YAML (yaml) - Suite

```
21.     hibernate:
22.         format_sql: true
23.         dialect: org.hibernate.dialect.MySQL8Dialect
24.
25. # Configuration du logging
26. logging:
27.     level:
28.         root: INFO
29.         com.example.demo: DEBUG
30.     file:
31.         name: logs/application.log
32.
33. # Propriétés personnalisées
34. app:
35.     name: Mon Application
36.     version: 1.0.0
37.     max-upload-size: 10MB
38.     email:
39.         from: noreply@example.com
40.     smtp:
```


CONFIGURATION AVEC APPLICATION.YML (PARTIE 2/3) [B]

Même configuration en format YAML (yaml) - Suite

```
41.      host: smtp.gmail.com
42.      port: 587
```

CONFIGURATION AVEC APPLICATION.YML

(PARTIE 3/3)

Structure hiérarchique avec indentation (2 espaces recommandés)

Plus lisible pour configurations complexes et imbriquées

Évite la répétition des préfixes (spring.datasource.* devient spring: datasource:)

Supporte les listes et objets complexes nativement

Attention : sensible à l'indentation (pas de tabulations !)

COMPARAISON : PROPERTIES VS YAML (PARTIE 1/2)

Comparaison des formats de configuration Spring Boot

Critère	application.properties	application.yml
Format	Clé=valeur plat	Hiérarchique avec indentation
Lisibilité	Bonne pour configurations simples	Excellente pour configurations complexes
Répétition	Préfixes répétés (spring.datasource.*)	Pas de répétition grâce à la hiérarchie
Sensibilité	Tolérant aux espaces	Très sensible à l'indentation
Support IDE	Excellent	Excellent
Courbe d'apprentissage	Immédiate (format traditionnel Java)	Légère (syntaxe YAML à apprendre)
Structures complexes	Verbeux pour listes/objets	Natif et concis
Popularité	Historique, toujours utilisé	Préféré dans projets modernes

COMPARAISON : PROPERTIES VS YAML (PARTIE 2/2)

Les deux formats sont totalement équivalents en fonctionnalités. Le choix dépend des préférences de l'équipe et de la complexité de la configuration. Spring Boot peut charger les deux simultanément, avec `application.properties` ayant la priorité en cas de conflit.

PROFILS DE CONFIGURATION (PROFILES)

(PARTIE 1/3)

Les profils Spring permettent de définir différentes configurations selon l'environnement d'exécution (développement, test, production). On peut créer des fichiers spécifiques par profil qui surchargent la configuration de base, permettant ainsi d'adapter l'application sans modifier le code.

PROFILS DE CONFIGURATION (PROFILES)

(PARTIE 2/3)

Configuration multi-profils avec YAML (yaml)

```
1. # application.yml (configuration commune)
2. spring:
3.   application:
4.     name: mon-application
5.
6. app:
7.   name: Mon Application
8.
9. ---
10. # application-dev.yml (développement)
11. spring:
12.   config:
13.     activate:
14.       on-profile: dev
15.   datasource:
16.     url: jdbc:h2:mem:testdb
17.     username: sa
18.     password:
19.   jpa:
20.     show-sql: true
```

PROFILS DE CONFIGURATION (PROFILES) (PARTIE 2/3) [A]

Configuration multi-profils avec YAML (yaml) - Suite

```
21.  
22. logging:  
23.   level:  
24.     com.example: DEBUG  
25.  
26. ---  
27. # application-prod.yml (production)  
28. spring:  
29.   config:  
30.     activate:  
31.       on-profile: prod  
32.   datasource:  
33.     url: jdbc:mysql://prod-server:3306/proddb  
34.     username: ${DB_USER}  
35.     password: ${DB_PASSWORD}  
36.   jpa:  
37.     show-sql: false  
38.  
39. logging:  
40.   level:
```

PROFILS DE CONFIGURATION (PROFILES) (PARTIE 2/3) [B]

Configuration multi-profils avec YAML (yaml) - Suite

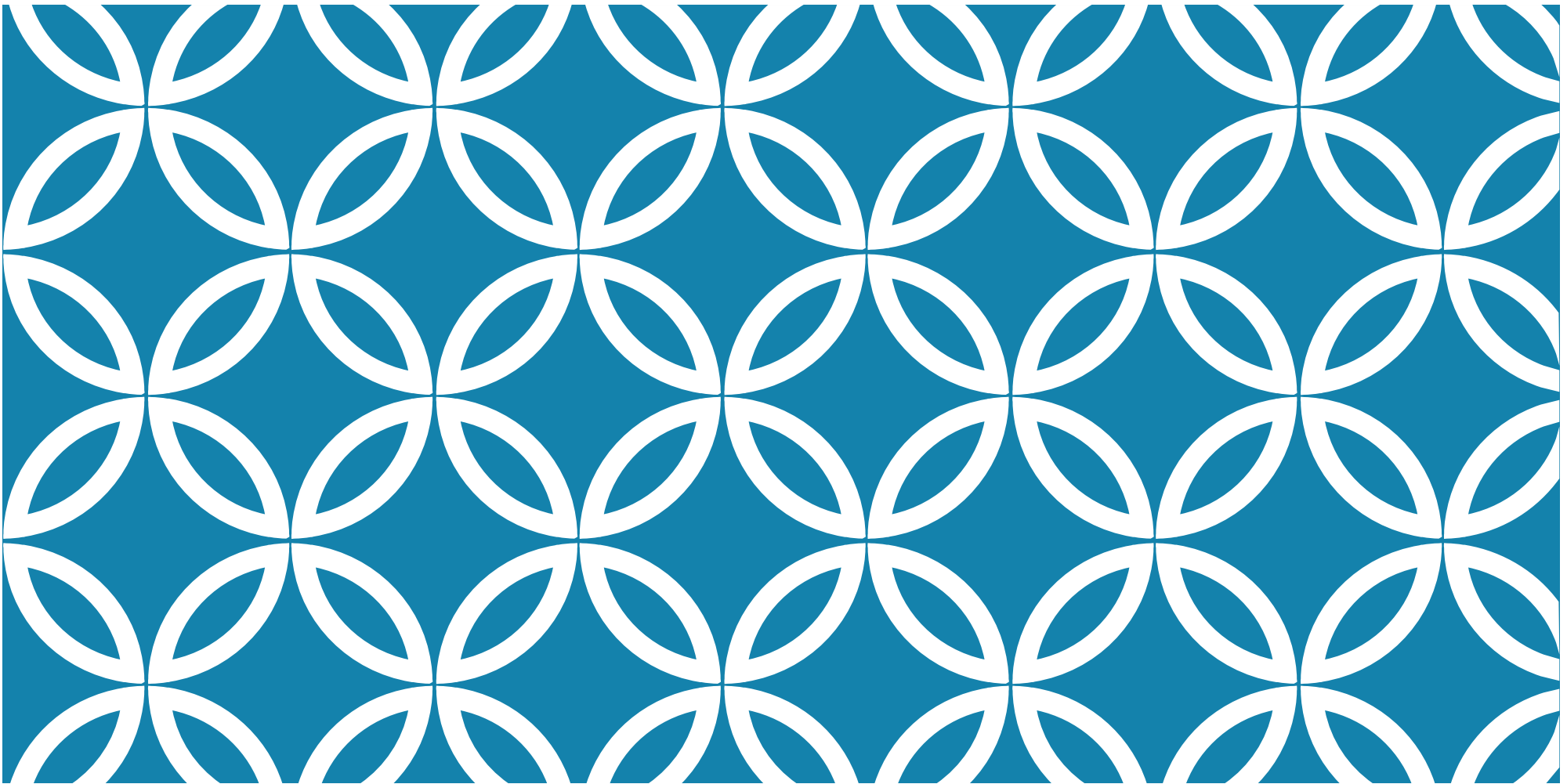
```
41.      com.example: WARN
```


PROFILS DE CONFIGURATION (PROFILES)

(PARTIE 3/3)

Exemple de Code (Code)

```
1. # Activer un profil au démarrage
2.
3. # 1. Dans application.properties/yml
4. spring.profiles.active=dev
5.
6. # 2. Via argument JVM
7. java -jar app.jar -Dspring.profiles.active=prod
8.
9. # 3. Via variable d'environnement
10. export SPRING_PROFILES_ACTIVE=prod
11. java -jar app.jar
12.
13. # 4. Via argument de programme
14. java -jar app.jar --spring.profiles.active=prod
```



LECTURE 3 : ARCHITECTURE MVC ET DÉVELOPPEMENT DE CONTRÔLEURS SPRING

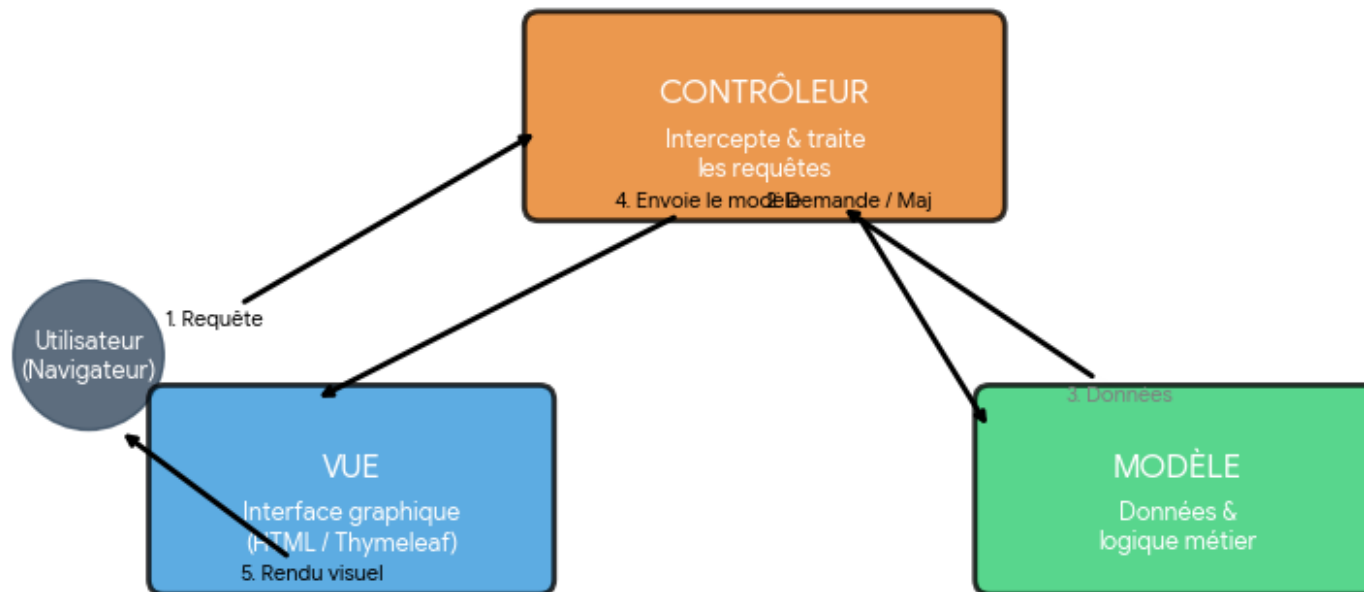
- Compréhension du pattern
Model-View-Controller (MVC)
dans le contexte Spring, -
Création de contrôleurs avec
`@Controller` et
`@RestController` pour gérer les
requêtes HTTP, - Mapping des
routes avec `@R`

LE PATTERN MVC : VUE D'ENSEMBLE (PARTIE 1/2)

Le pattern Model-View-Controller (MVC) est un patron d'architecture logicielle qui sépare la logique applicative en trois composants distincts et interconnectés. Cette séparation des responsabilités facilite la maintenance, les tests et l'évolution des applications web.

LE PATTERN MVC : VUE D'ENSEMBLE (PARTIE 2/2)

Le Pattern MVC : Vue d'ensemble



LES TROIS COMPOSANTS DU PATTERN MVC

Model (Modèle) : Représente les données et la logique métier

- Entités JPA (@Entity)
- Services métier (@Service)
- Repositories (@Repository)
- Gestion de la persistance et validation des données

View (Vue) : Interface utilisateur et présentation

- Templates Thymeleaf, JSP ou autres moteurs de rendu
- Affichage des données fournies par le contrôleur
- Aucune logique métier, uniquement présentation

Controller (Contrôleur) : Gestion des requêtes et coordination

- Réception des requêtes HTTP (@Controller, @RestController)
- Appel aux services métier appropriés
- Sélection de la vue et préparation des données à afficher
- Gestion des redirections et des codes de statut HTTP

MVC DANS LE CONTEXTE SPRING BOOT (PARTIE 1/2)

Spring MVC est l'implémentation du pattern MVC par le framework Spring. Il utilise le `DispatcherServlet` comme contrôleur frontal qui intercepte toutes les requêtes HTTP entrantes et les route vers les contrôleurs appropriés selon les mappings définis.

MVC DANS LE CONTEXTE SPRING BOOT (PARTIE 2/2)

Flux de Traitement d'une Requête Spring MVC

Étape	Composant	Action
1	DispatcherServlet	Réception de la requête HTTP
2	HandlerMapping	Identification du contrôleur approprié
3	Controller	Exécution de la logique métier via Services
4	Model	Récupération/modification des données
5	Controller	Ajout des données au Model et retour du nom de vue
6	ViewResolver	Résolution du nom de vue en template réel
7	View	Rendu du template avec les données du Model
8	DispatcherServlet	Envoi de la réponse HTTP au client

@CONTROLLER : CONTRÔLEUR POUR APPLICATIONS WEB CLASSIQUES (PARTIE 1/3)

L'annotation `@Controller` marque une classe comme contrôleur Spring MVC destiné à gérer des requêtes web et retourner des vues (pages HTML). Elle combine `@Component` pour la détection automatique et des capacités spécifiques de gestion des requêtes HTTP.

@CONTROLLER : CONTRÔLEUR POUR APPLICATIONS WEB CLASSIQUES (PARTIE 2/3)

Exemple de Contrôleur Web avec @Controller (java)

```
1. package com.example.demo.controller;
2.
3. import org.springframework.stereotype.Controller;
4. import org.springframework.ui.Model;
5. import org.springframework.web.bind.annotation.GetMapping;
6.
7. @Controller
8. public class AccueilController {
9.
10.     @GetMapping("/")
11.     public String afficherAccueil(Model model) {
12.         model.addAttribute("message", "Bienvenue sur notre application");
13.         model.addAttribute("titre", "Page d'accueil");
14.
15.         // Retourne le nom de la vue (sans extension)
16.         // Spring recherchera templates/accueil.html
17.         return "accueil";
18.     }
19.
20.     @GetMapping("/a-propos")
```

@CONTROLLER : CONTRÔLEUR POUR APPLICATIONS WEB CLASSIQUES (PARTIE 2/3)

[A]

Exemple de Contrôleur Web avec @Controller (java) - Suite

```
21.     public String afficherAPropos(Model model) {  
22.         model.addAttribute("version", "1.0.0");  
23.         return "a-propos";  
24.     }  
25. }
```

@CONTROLLER : CONTRÔLEUR POUR APPLICATIONS WEB CLASSIQUES (PARTIE 3/3)

Caractéristiques clés de @Controller :**

- Retourne des noms de vues (String) qui seront résolus en templates
- Utilise l'objet Model pour passer des données à la vue
- Adapté pour les applications avec rendu côté serveur (Thymeleaf, JSP)
- Peut gérer les redirections avec le préfixe "redirect:"
- Peut effectuer des forwards avec le préfixe "forward:"

@RestController : CONTRÔLEUR POUR API REST (PARTIE 1/2)

@RestController est une annotation spécialisée qui combine @Controller et @ResponseBody. Elle est conçue pour créer des services web RESTful qui retournent directement des données (JSON, XML) plutôt que des vues HTML.

@RestController : CONTRÔLEUR POUR API REST (PARTIE 2/2)

Exemple de Contrôleur REST avec @RestController (java)

```
1. package com.example.demo.controller;
2.
3. import org.springframework.web.bind.annotation.*;
4. import java.util.List;
5. import java.util.ArrayList;
6.
7. @RestController
8. @RequestMapping("/api/produits")
9. public class ProduitRestController {
10.
11.     @GetMapping
12.     public List<Produit> listerProduits() {
13.         // Retourne directement une liste d'objets
14.         // Spring la convertit automatiquement en JSON
15.         List<Produit> produits = new ArrayList<>();
16.         produits.add(new Produit(1L, "Laptop", 999.99));
17.         produits.add(new Produit(2L, "Souris", 29.99));
18.         return produits;
19.     }
20.
```

@RestController : CONTRÔLEUR POUR API REST (PARTIE 2/2) [A]

Exemple de Contrôleur REST avec @RestController (java) - Suite

```
21.     @GetMapping("/{id}")
22.     public Produit obtenirProduit(@PathVariable Long id) {
23.         // Retourne un objet unique converti en JSON
24.         return new Produit(id, "Laptop", 999.99);
25.     }
26.
27.     @PostMapping
28.     public Produit creerProduit(@RequestBody Produit produit) {
29.         // Reçoit et retourne du JSON
30.         // @RequestBody désérialise le JSON en objet Java
31.         return produit;
32.     }
33. }
```

COMPARAISON @CONTROLLER VS @RestController (PARTIE 1/2)

Différences entre @Controller et @RestController

Critère	@Controller	@RestController
Composition	@Component	@Controller + @ResponseBody
Type de retour	Nom de vue (String)	Données (objets Java)
Sérialisation	Non automatique	JSON/XML automatique
Usage principal	Applications web avec HTML	API REST
Objet Model	Nécessaire pour passer données	Non utilisé
Rendu	Côté serveur (templates)	Côté client (JSON)
Content-Type	text/html	application/json
@ResponseBody	Requis par méthode si JSON	Implicite sur toutes méthodes

COMPARAISON @CONTROLLER VS @RESTCONTROLLER (PARTIE 2/2)

Le choix entre @Controller et @RestController dépend de l'architecture de votre application : utilisez @Controller pour les applications monolithiques avec rendu serveur, et @RestController pour les architectures découplées (SPA, applications mobiles) où le backend fournit uniquement des données via API.

MAPPING DE ROUTES :

@RequestMapping (PARTIE 1/2)

@RequestMapping est l'annotation générique pour mapper des requêtes HTTP à des méthodes de contrôleur. Elle peut être utilisée au niveau de la classe (préfixe commun) et au niveau des méthodes (routes spécifiques). Elle supporte tous les verbes HTTP et offre de nombreuses options de configuration.

MAPPING DE ROUTES : @RequestMapping (PARTIE 2/2)

Utilisation de @RequestMapping avec différentes configurations (java)

```
1. package com.example.demo.controller;
2.
3. import org.springframework.web.bind.annotation.*;
4.
5. @RestController
6. @RequestMapping("/api/utilisateurs") // Préfixe pour toutes les méthodes
7. public class UtilisateurController {
8.
9.     // GET /api/utilisateurs
10.    @RequestMapping(method = RequestMethod.GET)
11.    public List<Utilisateur> lister() {
12.        return utilisateurService.findAll();
13.    }
14.
15.    // POST /api/utilisateurs
16.    @RequestMapping(method = RequestMethod.POST,
17.                    consumes = "application/json",
18.                    produces = "application/json")
19.    public Utilisateur creer(@RequestBody Utilisateur user) {
20.        return utilisateurService.save(user);
21.    }
22. }
```

MAPPING DE ROUTES :

@RequestMapping (PARTIE 2/2) [A]

Utilisation de @RequestMapping avec différentes configurations (java) - Suite

```
21.     }
22.
23.     // GET /api/utilisateurs/recherche?nom=Dupont
24.     @RequestMapping(value = "/recherche",
25.                     method = RequestMethod.GET,
26.                     params = "nom")
27.     public List<Utilisateur> rechercherParNom(
28.         @RequestParam String nom) {
29.         return utilisateurService.findByNom(nom);
30.     }
31.
32.     // PUT /api/utilisateurs/5
33.     @RequestMapping(value =("/{id}",
34.                     method = RequestMethod.PUT)
35.     public Utilisateur modifier(
36.         @PathVariable Long id,
37.         @RequestBody Utilisateur user) {
38.         return utilisateurService.update(id, user);
39.     }
40. }
```

ANNOTATIONS DE MAPPING SPÉCIALISÉES (PARTIE 1/3)

Spring fournit des annotations raccourcies pour les verbes HTTP les plus courants, rendant le code plus lisible et expressif. Ces annotations sont des spécialisations de `@RequestMapping` avec le verbe HTTP pré-configuré.

ANNOTATIONS DE MAPPING SPÉCIALISÉES (PARTIE 2/3)

Annotations de Mapping HTTP

Annotation	Verbe HTTP	Usage typique	Équivalent @RequestMapping
@GetMapping	GET	Récupération de ressources	@RequestMapping(method = GET)
@PostMapping	POST	Création de ressources	@RequestMapping(method = POST)
@PutMapping	PUT	Mise à jour complète	@RequestMapping(method = PUT)
@PatchMapping	PATCH	Mise à jour partielle	@RequestMapping(method = PATCH)
@DeleteMapping	DELETE	Suppression de ressources	@RequestMapping(method = DELETE)

ANNOTATIONS DE MAPPING SPÉCIALISÉES (PARTIE 3/3)

Avantages des annotations spécialisées : **

- Code plus concis et lisible
- Intention claire du point de terminaison
- Conformité aux principes REST
- Réduction des erreurs de configuration
- Meilleure documentation auto-générée
(Swagger/OpenAPI)

@GETMAPPING : RÉCUPÉRATION DE DONNÉES (PARTIE 1/2)

@GetMapping est utilisé pour mapper les requêtes HTTP GET, typiquement pour récupérer des données sans modifier l'état du serveur. C'est l'opération la plus courante dans les API REST et les applications web.

@GETMAPPING : RÉCUPÉRATION DE DONNÉES (PARTIE 2/2)

Exemples variés d'utilisation de @GetMapping (java)

```
1. package com.example.demo.controller;
2.
3. import org.springframework.web.bind.annotation.*;
4. import org.springframework.beans.factory.annotation.Autowired;
5.
6. @RestController
7. @RequestMapping("/api/articles")
8. public class ArticleController {
9.
10.     @Autowired
11.     private ArticleService articleService;
12.
13.     // GET /api/articles - Liste tous les articles
14.     @GetMapping
15.     public List<Article> listerTous() {
16.         return articleService.findAll();
17.     }
18.
19.     // GET /api/articles/5 - Article spécifique par ID
20.     @GetMapping("/{id}")
```


@GETMAPPING : RÉCUPÉRATION DE DONNÉES (PARTIE 2/2) [A]

Exemples variés d'utilisation de @GetMapping (java) - Suite

```
21.     public Article obtenirParId(@PathVariable Long id) {
22.         return articleService.findById(id)
23.             .orElseThrow(() -> new ResourceNotFoundException(
24.                 "Article non trouvé"));
25.     }
26.     // GET /api/articles/categorie/tech - Filtrage par catégorie
27.     @GetMapping("/categorie/{categorie}")
28.     public List<Article> parCategorie(@PathVariable String categorie) {
29.         return articleService.findByCategorie(categorie);
30.     }
31.
32.     // GET /api/articles/recherche?titre=Spring&auteur=Martin
33.     @GetMapping("/recherche")
34.     public List<Article> rechercher(
35.         @RequestParam(required = false) String titre,
36.         @RequestParam(required = false) String auteur) {
37.         return articleService.search(titre, auteur);
38.     }
39.
40.     // GET /api/articles?page=0&size=10&sort=date,desc
```

@GETMAPPING : RÉCUPÉRATION DE DONNÉES (PARTIE 2/2) [B]

Exemples variés d'utilisation de @GetMapping (java) - Suite

```
41.     @GetMapping(params = {"page", "size"})
42.     public Page<Article> listerAvecPagination(
43.         @RequestParam int page,
44.         @RequestParam int size,
45.         @RequestParam(defaultValue = "id") String sort) {
46.         Pageable pageable = PageRequest.of(page, size, Sort.by(sort));
47.         return articleService.findAll(pageable);
48.     }
49. }
```

@POSTMAPPING : CRÉATION DE RESSOURCES (PARTIE 1/3)

@PostMapping gère les requêtes HTTP POST, utilisées principalement pour créer de nouvelles ressources. Les données sont généralement envoyées dans le corps de la requête au format JSON et désérialisées automatiquement par Spring.

@POSTMAPPING : CRÉATION DE RESSOURCES (PARTIE 2/3)

Utilisation de `@PostMapping` pour création de ressources (java)

```
1. package com.example.demo.controller;
2.
3. import org.springframework.http.HttpStatus;
4. import org.springframework.http.ResponseEntity;
5. import org.springframework.web.bind.annotation.*;
6. import org.springframework.validation.annotation.Validated;
7. import javax.validation.Valid;
8.
9. @RestController
10. @RequestMapping("/api/commandes")
11. public class CommandeController {
12.
13.     @Autowired
14.     private CommandeService commandeService;
15.
16.     // POST /api/commandes - Création simple
17.     @PostMapping
18.     public Commande creer(@RequestBody @Valid Commande commande) {
19.         return commandeService.save(commande);
20.     }
```

@POSTMAPPING : CRÉATION DE RESSOURCES (PARTIE 2/3) [A]

Utilisation de @PostMapping pour création de ressources (java) - Suite

```
21.  
22.     // POST /api/commandes - Avec ResponseEntity pour contrôle HTTP  
23.     @PostMapping("/v2")  
24.     public ResponseEntity<Commande> creerAvecStatus(  
25.         @RequestBody @Valid Commande commande) {  
26.         Commande nouvelleCmde = commandeService.save(commande);  
27.  
28.         // Retourne 201 Created avec l'URI de la ressource créée  
29.         return ResponseEntity  
30.             .status(HttpStatus.CREATED)  
31.             .header("Location", "/api/commandes/" + nouvelleCmde.getId())  
32.             .body(nouvelleCmde);  
33.     }  
34.  
35.     // POST /api/commandes/5/valider - Action sur ressource existante  
36.     @PostMapping("/{id}/valider")  
37.     public ResponseEntity<Void> valider(@PathVariable Long id) {  
38.         commandeService.valider(id);  
39.         return ResponseEntity.ok().build();  
40.     }
```

@PostMapping : CRÉATION DE RESSOURCES (PARTIE 2/3) [B]

Utilisation de @PostMapping pour création de ressources (java) - Suite

```
41.  
42.    // POST /api/commandes/batch - Création multiple  
43.    @PostMapping("/batch")  
44.    public List<Commande> creerPlusieurs(  
45.        @RequestBody @Valid List<Commande> commandes) {  
46.        return commandeService.saveAll(commandes);  
47.    }  
48.  
49.    // POST avec paramètres d'URL et corps  
50.    @PostMapping("/client/{clientId}")  
51.    public Commande creerPourClient(  
52.        @PathVariable Long clientId,  
53.        @RequestBody @Valid Commande commande) {  
54.        commande.setClientId(clientId);  
55.        return commandeService.save(commande);  
56.    }  
57. }
```

@POSTMAPPING : CRÉATION DE RESSOURCES (PARTIE 3/3)

Bonnes pratiques avec `@PostMapping` :**

- Utiliser `@Valid` pour valider automatiquement les données entrantes
- Retourner HTTP 201 (Created) plutôt que 200 (OK) pour les créations
- Inclure l'URI de la ressource créée dans le header Location
- Utiliser `@RequestBody` pour désérialiser le JSON en objet Java
- Gérer les exceptions de validation avec `@ExceptionHandler`
- Retourner la ressource créée avec son ID généré

@PUTMAPPING ET @PATCHMAPPING : MISE À JOUR (PARTIE 1/2)

Différence entre @PutMapping et @PatchMapping (java)

```
1. package com.example.demo.controller;
2.
3. import org.springframework.web.bind.annotation.*;
4. import org.springframework.http.ResponseEntity;
5.
6. @RestController
7. @RequestMapping("/api/produits")
8. public class ProduitController {
9.
10.     @Autowired
11.     private ProduitService produitService;
12.
13.     // PUT /api/produits/5 - Remplacement complet de la ressource
14.     @PutMapping("/{id}")
15.     public ResponseEntity<Produit> mettreAJourComplet(
16.         @PathVariable Long id,
17.         @RequestBody @Valid Produit produit) {
18.
19.         // Vérifie que la ressource existe
20.         if (!produitService.existsById(id)) {
```


@PUTMAPPING ET @PATCHMAPPING : MISE À JOUR (PARTIE 1/2) [A]

Différence entre @PutMapping et @PatchMapping (java) - Suite

```
21.         return ResponseEntity.notFound().build();
22.     }
23.
24.     // Remplace complètement la ressource
25.     produit.setId(id);
26.     Produit produitMaj = produitService.save(produit);
27.
28.     return ResponseEntity.ok(produitMaj);
29. }
30.
31. // PATCH /api/produits/5 - Modification partielle
32. @PatchMapping("/{id}")
33. public ResponseEntity<Produit> mettreAJourPartiel(
34.     @PathVariable Long id,
35.     @RequestBody Map<String, Object> updates) {
36.
37.     Produit produit = produitService.findById(id)
38.         .orElseThrow(() -> new ResourceNotFoundException("Produit non
trouvé"));
39.
40.     // Applique uniquement les champs fournis
```

@PUTMAPPING ET @PATCHMAPPING : MISE À JOUR (PARTIE 1/2) [B]

Différence entre @PutMapping et @PatchMapping (java) - Suite

```
41.         updates.forEach((key, value) -> {
42.             switch (key) {
43.                 case "nom":
44.                     produit.setNom((String) value);
45.                     break;
46.                 case "prix":
47.                     produit.setPrix((Double) value);
48.                     break;
49.                 case "stock":
50.                     produit.setStock((Integer) value);
51.                     break;
52.             }
53.         });
54.
55.         Produit produitMaj = produitService.save(produit);
56.         return ResponseEntity.ok(produitMaj);
57.     }
58.
59.     // PATCH avec DTO spécifique pour mise à jour partielle
60.     @PatchMapping("/{id}/prix")
```

@PUTMAPPING ET @PATCHMAPPING : MISE À JOUR (PARTIE 1/2) [C]

Différence entre @PutMapping et @PatchMapping (java) - Suite

```
61.     public ResponseEntity<Produit> modifierPrix(  
62.         @PathVariable Long id,  
63.         @RequestBody PrixUpdateDTO prixDTO) {  
64.  
65.         Produit produit = produitService.updatePrix(id,  
prixDTO.getNouveau_prix());  
66.         return ResponseEntity.ok(produit);  
67.     }  
68. }
```

@PUTMAPPING ET @PATCHMAPPING : MISE À JOUR (PARTIE 2/2)

PUT vs PATCH : Comparaison

Aspect	@PutMapping (PUT)	@PatchMapping (PATCH)
Sémantique	Remplacement complet	Modification partielle
Idempotence	Oui	Oui
Données requises	Toutes les propriétés	Uniquement les champs à modifier
Champs omis	Réinitialisés à null/défaut	Conservent leur valeur actuelle
Complexité	Simple à implémenter	Plus complexe (logique conditionnelle)
Bande passante	Plus élevée	Optimisée
Usage typique	Formulaire complet	Modification ciblée (ex: statut)

@DELETEMAPPING : SUPPRESSION DE RESSOURCES (PARTIE 1/2)

@DeleteMapping gère les requêtes HTTP DELETE pour supprimer des ressources. Cette opération est idempotente : supprimer plusieurs fois la même ressource produit le même résultat (la ressource n'existe plus).

@DELETEMAPPING : SUPPRESSION DE RESSOURCES (PARTIE 2/2)

Exemple de Code (Code)

```
1. package com.example.demo.controller;
2.
3. import org.springframework.web.bind.annotation.*;
4. import org.springframework.http.ResponseEntity;
5. import org.springframework.http.HttpStatus;
6.
7. @RestController
8. @RequestMapping("/api/clients")
9. public class ClientController {
10.
11.     @Autowired
12.     private ClientService clientService;
13.
14.     // DELETE /api/clients/5 - Suppression simple
15.     @DeleteMapping("/{id}")
16.     public ResponseEntity<Void> supprimer(@PathVariable Long id) {
17.         clientService.deleteById(id);
18.         return ResponseEntity.noContent().build(); // 204 No Content
19.     }
20.
```

@DELETEMAPPING : SUPPRESSION DE RESSOURCES (PARTIE 2/2) [A]

Exemple de Code (Code) - Suite

```
21.    // DELETE avec vérification d'existence
22.    @DeleteMapping("/v2/{id}")
23.    public ResponseEntity<Void> supprimerAvecVerification(@PathVariable
Long id) {
24.        if (!clientService.existsById(id)) {
25.            return ResponseEntity.notFound().build(); // 404 Not Found
26.        }
27.
28.        clientService.deleteById(id);
29.        return ResponseEntity.noContent().build(); // 204 No Content
30.    }
31.
32.    // DELETE avec message de confirmation
33.    @DeleteMapping("/v3/{id}")
34.    public ResponseEntity<MessageDTO> supprimerAvecMessage(@PathVariable Long id)
35.        Client client = clientService.findById(id)
36.            .orElseThrow(() -> new ResourceNotFoundException("Client non trouvé"))
37.
38.        clientService.delete(client);
39.
40.        MessageDTO message = new MessageDTO(
```

@DELETEMAPPING : SUPPRESSION DE RESSOURCES (PARTIE 2/2) [B]

Exemple de Code (Code) - Suite

```
41.         "Client " + client.getNom() + " supprimé avec succès"
42.     );
43.     return ResponseEntity.ok(message); // 200 OK avec corps
44. }
45.
46. // DELETE /api/clients?ids
```


GESTION DES PARAMÈTRES DE REQUÊTE (@RequestParam) (PARTIE 1/3)

Les paramètres de requête (query parameters) sont des données transmises dans l'URL après le symbole '?'. Spring Boot utilise l'annotation `@RequestParam` pour extraire et injecter automatiquement ces valeurs dans les méthodes des contrôleurs.

GESTION DES PARAMÈTRES DE REQUÊTE (@RequestParam) (PARTIE 2/3)

Exemple d'utilisation de @RequestParam (java)

```
1. @GetMapping("/recherche")
2. public String rechercherProduits(
3.     @RequestParam(name = "categorie") String categorie,
4.     @RequestParam(name = "prix", required = false, defaultValue = "0")
Double prixMax,
5.     @RequestParam(name = "page", defaultValue = "1") int page,
6.     Model model) {
7.
8.     List<Produit> produits = produitService.rechercher(categorie,
prixMax, page);
9.     model.addAttribute("resultats", produits);
10.    return "liste-produits";
11. }
12. // URL exemple: /recherche?categorie=electronique&prix=500&page=2
```

GESTION DES PARAMÈTRES DE REQUÊTE (@RequestParam) (PARTIE 3/3)

Attributs principaux de @RequestParam :

- name/value : nom du paramètre dans l'URL
- required : indique si le paramètre est obligatoire (true par défaut)
- defaultValue : valeur par défaut si le paramètre est absent

Conversion automatique vers les types primitifs (int, double, boolean)

Gestion des paramètres multiples avec List<String> ou tableau

VARIABLES DE CHEMIN (@PATHVARIABLE) (PARTIE 1/3)

Les variables de chemin (path variables) sont des segments dynamiques intégrés directement dans le chemin de l'URL. Elles sont essentielles pour les API REST et permettent d'identifier des ressources spécifiques de manière élégante et conforme aux principes RESTful.

VARIABLES DE CHEMIN (@PATHVARIABLE) (PARTIE 2/3)

Utilisation de @PathVariable avec chemins simples et multiples (java)

```
1. @GetMapping("/utilisateurs/{id}")
2. public String afficherUtilisateur(
3.     @PathVariable("id") Long utilisateurId,
4.     Model model) {
5.
6.     Utilisateur user = utilisateurService.findById(utilisateurId);
7.     model.addAttribute("utilisateur", user);
8.     return "profil-utilisateur";
9. }
10.
11. @GetMapping("/commandes/{commandeId}/articles/{articleId}")
12. public String detailArticle(
13.     @PathVariable Long commandeId,
14.     @PathVariable Long articleId,
15.     Model model) {
16.
17.     Article article = commandeService.getArticle(commandeId, articleId);
18.     model.addAttribute("article", article);
19.     return "detail-article";
20. }
```

VARIABLES DE CHEMIN

(@PATHVARIABLE) (PARTIE 2/3) [A]

Utilisation de `@PathVariable` avec chemins simples et multiples (java) - Suite

```
21. // URL exemple: /commandes/123/articles/456
```

VARIABLES DE CHEMIN (@PATHVARIABLE) (PARTIE 3/3)

Comparaison @RequestParam vs @PathVariable

Critère	@RequestParam	@PathVariable
Position dans URL	Après '?' (query string)	Dans le chemin de l'URL
Syntaxe URL	/recherche?id=123	/utilisateurs/123
Optionalité	Peut être optionnel (required=false)	Généralement obligatoire
Cas d'usage	Filtres, pagination, tri	Identification de ressources
Style REST	Moins RESTful	Conforme REST
Exemple typique	?page=2&sort=nom	/produits/42/avis/7

GESTION DU CORPS DE REQUÊTE (@REQUESTBODY) (PARTIE 1/3)

L'annotation `@RequestBody` permet de récupérer et désérialiser automatiquement le contenu du corps d'une requête HTTP (généralement en JSON) vers un objet Java. Cette fonctionnalité est essentielle pour les API REST qui reçoivent des données complexes via POST, PUT ou PATCH.

GESTION DU CORPS DE REQUÊTE (@RequestBody) (PARTIE 2/3)

Utilisation de @RequestBody pour créer et modifier des ressources (java)

```
1. // Classe DTO (Data Transfer Object)
2. public class UtilisateurDTO {
3.     private String nom;
4.     private String email;
5.     private String motDePasse;
6.     private List<String> roles;
7.
8.     // Getters et setters...
9. }
10.
11. @PostMapping("/utilisateurs")
12. public ResponseEntity<Utilisateur> creerUtilisateur(
13.     @RequestBody @Valid UtilisateurDTO utilisateurDTO) {
14.
15.     Utilisateur nouvelUtilisateur = utilisateurService.creer(utilisateurDTO);
16.     return ResponseEntity
17.         .status(HttpStatus.CREATED)
18.         .body(nouvelUtilisateur);
19. }
20.
```

GESTION DU CORPS DE REQUÊTE (@RequestBody) (PARTIE 2/3) [A]

Utilisation de @RequestBody pour créer et modifier des ressources (java) - Suite

```
21. @PutMapping("/utilisateurs/{id}")
22. public ResponseEntity<Utilisateur> modifierUtilisateur(
23.     @PathVariable Long id,
24.     @RequestBody UtilisateurDTO utilisateurDTO) {
25.
26.     Utilisateur utilisateurModifie = utilisateurService.update(id,
    utilisateurDTO);
27.     return ResponseEntity.ok(utilisateurModifie);
28. }
```

GESTION DU CORPS DE REQUÊTE (@RequestBody) (PARTIE 3/3)

Caractéristiques de @RequestBody :

- Désérialisation automatique JSON → Objet Java (via Jackson)
- Fonctionne avec POST, PUT, PATCH (méthodes avec corps)
- Combinable avec @Valid pour validation automatique
- Peut gérer des structures complexes (objets imbriqués, collections)

Bonnes pratiques :

- Utiliser des DTO plutôt que des entités JPA directement
- Valider les données avec Bean Validation (@NotNull, @Email, etc.)
- Gérer les erreurs de désérialisation avec @ExceptionHandler

COMBINAISON DES ANNOTATIONS DE PARAMÈTRES (PARTIE 1/3)

Dans les applications réelles, il est fréquent de combiner plusieurs types de paramètres dans une même méthode de contrôleur pour gérer des cas d'usage complexes. Spring Boot permet cette combinaison de manière fluide et intuitive.

COMBINAISON DES ANNOTATIONS DE PARAMÈTRES (PARTIE 2/3)

Exemple complet combinant `@PathVariable`, `@RequestBody` et `@RequestParam` (java)

```
1. @PostMapping("/projets/{projetId}/taches")
2. public ResponseEntity<Tache> ajouterTache(
3.     @PathVariable Long projetId,
4.     @RequestBody @Valid TacheDTO tacheDTO,
5.     @RequestParam(name = "notifier", defaultValue = "true")
boolean envoyerNotifcation,
6.     @RequestParam(name = "priorite", required = false) String priorite,
7.     @RequestHeader("X-User-Id") Long utilisateurId) {
8.
9.     // Validation du projet
10.    Projet projet = projetService.findById(projetId);
11.
12.    // Création de la tâche avec les données du corps
13.    Tache nouvelleTache = tacheService.creer(projet, tacheDTO);
14.
15.    // Application de la priorité si spécifiée
16.    if (priorite != null) {
17.        nouvelleTache.setPriorite(Priorite.valueOf(priorite));
18.    }
19.
20.    // Notifcation conditionnelle
```

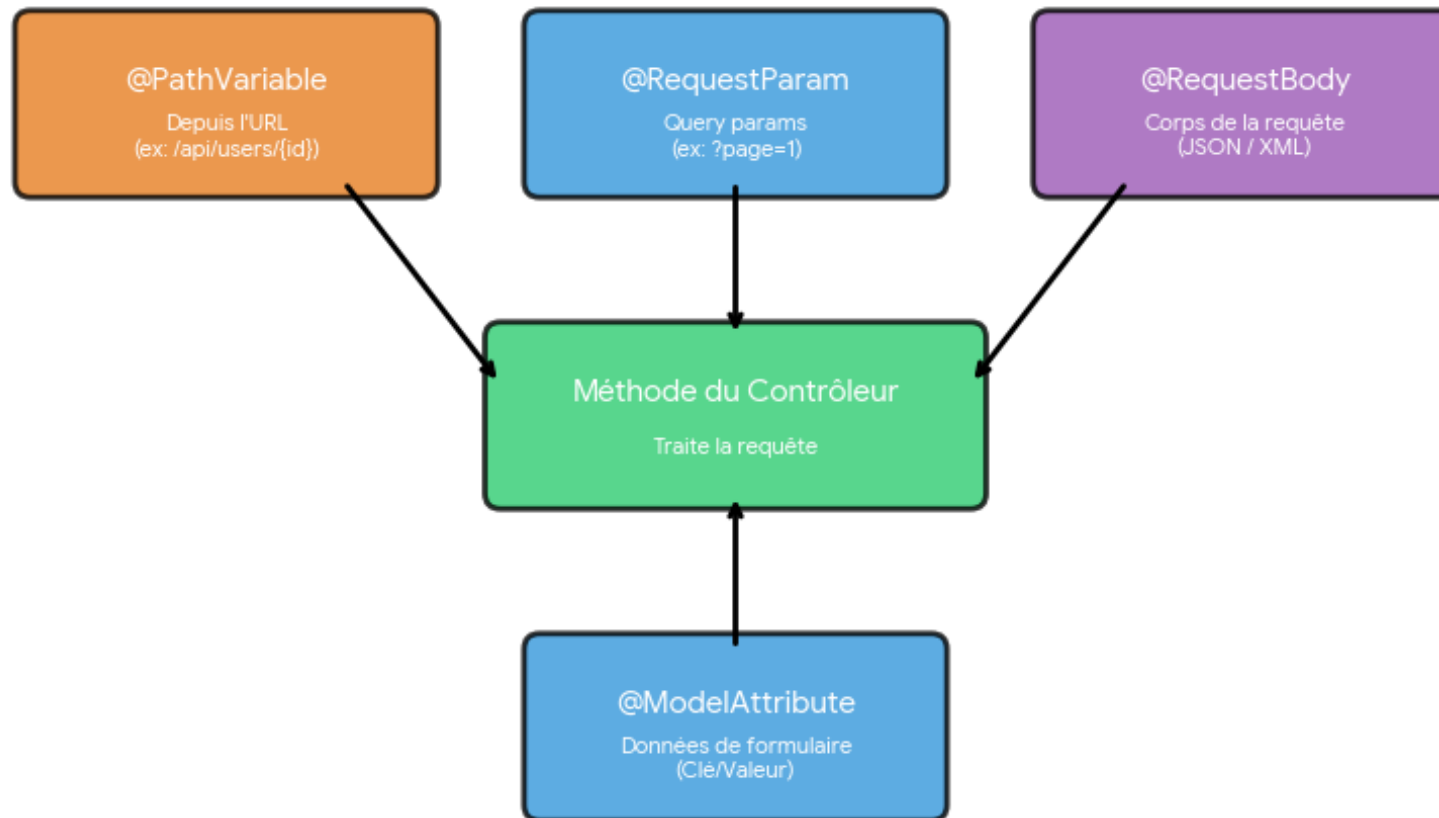
COMBINAISON DES ANNOTATIONS DE PARAMÈTRES (PARTIE 2/3) [A]

Exemple complet combinant `@PathVariable`, `@RequestBody` et `@RequestParam` (java) - Su

```
21.     if (envoyerNotifcation) {
22.         notificationService.notifierEquipe(projet, nouvelleTache,
utilisateurId);
23.     }
24.
25.     return ResponseEntity.status(HttpStatus.CREATED).body(nouvelleTache);
26. }
27. // URL exemple: POST /projets/42/taches?notifier=true&priorite=HAUTE
28. // Corps JSON: {"titre": "Nouvelle tâche", "description": "...",
"echeance": "2024-12-31"}
```

COMBINAISON DES ANNOTATIONS DE PARAMÈTRES (PARTIE 3/3)

Combinaison des Annotations de Paramètres dans Spring MVC



INTRODUCTION À MODELANDVIEW (PARTIE 1/3)

ModelAndView est une classe Spring qui combine à la fois le modèle de données (Model) et le nom de la vue à afficher (View). Elle offre une alternative à l'utilisation séparée de Model et du retour String, permettant de configurer les deux aspects en un seul objet.

INTRODUCTION À MODELANDVIEW (PARTIE 2/3)

Utilisation de ModelAndView dans les contrôleurs (java)

```
1. @GetMapping("/dashboard")
2. public ModelAndView afficherDashboard() {
3.     ModelAndView mav = new ModelAndView();
4.
5.     // Définir le nom de la vue
6.     mav.setViewName("dashboard");
7.
8.     // Ajouter des données au modèle
9.     mav.addObject("utilisateur", utilisateurConnecte);
10.    mav.addObject("statistiques", statistiqueService.getStats());
11.    mav.addObject("notifications", notificationService.getNonLues());
12.
13.    return mav;
14. }
15.
16. // Alternative avec constructeur
17. @GetMapping("/produits/{id}")
18. public ModelAndView detailProduit(@PathVariable Long id) {
19.     Produit produit = produitService.findById(id);
20.
```

INTRODUCTION À MODELANDVIEW (PARTIE 2/3) [A]

Utilisation de ModelAndView dans les contrôleurs (java) - Suite

```
21.     // Constructeur : new ModelAndView(nomVue, nomAttribut, valeur)
22.     ModelAndView mav = new ModelAndView("detail-produit", "produit", produit);
23.     mav.addObject("avisClients", avisService.getAvisByProduit(id));
24.
25.     return mav;
26. }
```

INTRODUCTION À MODELANDVIEW (PARTIE 3/3)

Avantages de ModelAndView :

- Encapsulation du modèle et de la vue dans un seul objet
- Utile pour les méthodes complexes avec logique conditionnelle de vue
- Permet de définir le statut HTTP et les en-têtes de réponse
- Facilite la réutilisation et les tests unitaires

Méthodes principales :

- `setViewName(String)` : définit la vue à afficher
- `addObject(String, Object)` : ajoute un attribut au modèle
- `addAllObjects(Map)` : ajoute plusieurs attributs d'un coup
- `setStatus(HttpStatus)` : définit le code de statut HTTP

TRANSMISSION DE DONNÉES AU FRONT-END AVEC MODEL (PARTIE 1/3)

Le Model est l'interface Spring permettant de transférer des données du contrôleur vers la vue (template Thymeleaf, JSP, etc.). Les attributs ajoutés au Model deviennent accessibles dans les templates pour l'affichage dynamique du contenu.

TRANSMISSION DE DONNÉES AU FRONT-END AVEC MODEL (PARTIE 2/3)

Transmission de données complexes via Model (java)

```
1. @GetMapping("/commandes")
2. public String listerCommandes(
3.     @RequestParam(defaultValue = "0") int page,
4.     @RequestParam(defaultValue = "10") int taille,
5.     Model model) {
6.
7.     // Récupération des données paginées
8.     Page<Commande> pageCommandes = commandeService.findAll(
9.         PageRequest.of(page, taille, Sort.by("dateCreation").descending())
10.    );
11.
12.    // Transmission de multiples objets au template
13.    model.addAttribute("commandes", pageCommandes.getContent());
14.    model.addAttribute("pageActuelle", page);
15.    model.addAttribute("totalPages", pageCommandes.getTotalPages());
16.    model.addAttribute("totalElements", pageCommandes.getTotalElements());
17.    model.addAttribute("titre", "Gestion des Commandes");
18.
19.    // Ajout d'objets complexes
20.    model.addAttribute("statistiques", Map.of(
```

TRANSMISSION DE DONNÉES AU FRONT-END AVEC MODEL (PARTIE 2/3) [A]

Transmission de données complexes via Model (java) - Suite

```
21.         "commandesEnCours", commandeService.countEnCours(),
22.         "commandesLivrees", commandeService.countLivrees(),
23.         "chiffreAffaires", commandeService.calculerCA()
24.     ));
25.
26.     return "liste-commandes";
27. }
```

TRANSMISSION DE DONNÉES AU FRONT-END AVEC MODEL (PARTIE 3/3)

Utilisation des données du Model dans un template Thymeleaf (html)

```
1. <!-- Template Thymeleaf : liste-commandes.html -->
2. <!DOCTYPE html>
3. <html xmlns:th="http://www.thymeleaf.org">
4. <head>
5.     <title th:text="${titre}">Commandes</title>
6. </head>
7. <body>
8.     <h1 th:text="${titre}">Liste des Commandes</h1>
9.
10.    <!-- Affichage des statistiques -->
11.    <div class="stats">
12.        <p>En cours: <span th:text="${statistiques.commandesEnCours}">0</span></p>
13.        <p>Livrées: <span th:text="${statistiques.commandesLivrees}">0</span></p>
14.        <p>CA: <span th:text="${statistiques.chiffreAffaires}">0</span> €</p>
15.    </div>
16.
17.    <!-- Itération sur la liste de commandes -->
18.    <table>
19.        <tr th:each="commande : ${commandes}">
20.            <td th:text="${commande.id}">1</td>
```

TRANSMISSION DE DONNÉES AU FRONT-END AVEC MODEL (PARTIE 3/3) [A]

Utilisation des données du Model dans un template Thymeleaf (html) - Suite

```
21.         <td th:text="${commande.client.nom}">Client</td>
22.         <td th:text="${#temporals.format(commande.dateCreation,
'dd/MM/yyyy')}">Date</td>
23.         <td th:text="${commande.montantTotal}">100.00</td>
24.     </tr>
25. </table>
26.
27.     <!-- Pagination -->
28.     <div class="pagination">
29.         <span th:text="'Page ' + ${pageActuelle + 1} + ' sur '
+ ${totalPages}"></span>
30.         <span th:text="'Total: ' + ${totalElements} + ' commandes'"></span>
31.     </div>
32. </body>
33. </html>
```


TECHNIQUES AVANCÉES DE TRANSMISSION DE DONNÉES (PARTIE 1/3)

Méthodes de transmission de données vers le Front-End

Approche	Cas d'usage	Syntaxe	Avantages
<code>Model.addAttribute()</code>	Données pour templates	<code>model.addAttribute("key", value)</code>	Simple, intuitif, type-safe
<code>ModelAndView.addObject()</code>	Vue + données combinées	<code>mav.addObject("key", value)</code>	Encapsulation complète
<code>@ModelAttribute</code> (méthode)	Données communes à toutes vues	<code>@ModelAttribute("user")</code> <code>User getUser()</code>	Évite la répétition de code
<code>RedirectAttributes</code>	Données pour redirection	<code>redirectAttributes.addFlashAttribute()</code>	Survit à la redirection
<code>HttpSession</code>	Données de session	<code>session.setAttribute("cart", cart)</code>	Persistence entre requêtes
<code>ResponseEntity<T></code>	API REST (JSON)	<code>return</code> <code>ResponseEntity.ok(data)</code>	Contrôle total de la réponse

TECHNIQUES AVANCÉES DE TRANSMISSION DE DONNÉES (PARTIE 2/3)

Techniques avancées : `@ModelAttribute` global et `RedirectAttributes` (java)

```
1. // Méthode @ModelAttribute - exécutée avant chaque requête du contrôleur
2. @ModelAttribute("categoriesMenu")
3. public List<Categorie> ajouterCategoriesMenu() {
4.     // Ces données seront disponibles dans TOUTES les vues de ce contrôleur
5.     return categorieService.findAll();
6. }
7.
8. @GetMapping("/produits")
9. public String afficherProduits(Model model) {
10.     model.addAttribute("produits", produitService.findAll());
11.     // "categoriesMenu" est automatiquement disponible sans
    l'ajouter explicitement
12.     return "catalogue";
13. }
14.
15. // RedirectAttributes pour les messages flash
16. @PostMapping("/produits")
17. public String creerProduit(@ModelAttribute ProduitDTO produitDTO,
18.                             RedirectAttributes redirectAttributes) {
19.     Produit nouveau = produitService.save(produitDTO);
20.
```

TECHNIQUES AVANCÉES DE TRANSMISSION DE DONNÉES (PARTIE 2/3) [A]

Techniques avancées : `@ModelAttribute` global et `RedirectAttributes` (java) - Suite

```
21.     // Message qui survit à la redirection
22.     redirectAttributes.addFlashAttribute("message", "Produit créé avec succès!");
23.     redirectAttributes.addFlashAttribute("type", "success");
24.
25.     return "redirect:/produits/" + nouveau.getId();
26. }
```

TECHNIQUES AVANCÉES DE TRANSMISSION DE DONNÉES (PARTIE 3/3)

Bonnes pratiques de transmission de données :

- Utiliser des DTO pour éviter d'exposer les entités JPA directement
- Préférer `Model.addAttribute()` pour les cas simples
- Utiliser `@ModelAttribute` au niveau méthode pour les données communes (menu, utilisateur connecté)
- `RedirectAttributes` pour les messages après POST/PUT/DELETE (pattern PRG)
- Limiter la quantité de données transmises (pagination, lazy loading)
- Valider et nettoyer les données avant transmission au Front-End

GESTION DES ERREURS ET VALIDATION DES DONNÉES (PARTIE 1/4)

La validation des données entrantes et la gestion appropriée des erreurs sont essentielles pour créer des applications robustes. Spring Boot intègre Bean Validation (JSR-303) et fournit des mécanismes élégants pour traiter les erreurs de validation et les transmettre au Front-End.

GESTION DES ERREURS ET VALIDATION DES DONNÉES (PARTIE 2/4)

Validation avec Bean Validation et gestion des erreurs (java)

```
1. // DTO avec annotations de validation
2. public class InscriptionDTO {
3.     @NotBlank(message = "Le nom est obligatoire")
4.     @Size(min = 2, max = 50, message = "Le nom doit contenir entre 2
et 50 caractères")
5.     private String nom;
6.
7.     @NotBlank(message = "L'email est obligatoire")
8.     @Email(message = "Format d'email invalide")
9.     private String email;
10.
11.     @NotNull(message = "Le mot de passe est obligatoire")
12.     @Pattern(regexp = "^(?=.*[A-Z])(?=.*[0-9])(?=.*[!@#$%]).{8,}$",
13.             message = "Le mot de passe doit contenir au moins 8
caractères, une majuscule, un chiffre et un caractère spécial")
14.     private String motDePasse;
15.
16.     @Min(value = 18, message = "Vous devez avoir au moins 18 ans")
17.     private Integer age;
18.
19.     // Getters et setters...
20. }
```

GESTION DES ERREURS ET VALIDATION DES DONNÉES (PARTIE 2/4) [A]

Validation avec Bean Validation et gestion des erreurs (java) - Suite

```
21.  
22. @PostMapping("/inscription")  
23. public String inscrire(@Valid @ModelAttribute InscriptionDTO inscriptionDTO,  
24.                        BindingResult bindingResult,  
25.                        Model model) {  
26.  
27.     // Vérification des erreurs de validation  
28.     if (bindingResult.hasErrors()) {  
29.         // Les erreurs sont automatiquement disponibles dans le template  
30.         model.addAttribute("messageErreur", "Veuillez corriger les  
erreurs du formulaire");  
31.         return "formulaire-inscription"; // Retour au formulaire avec erreurs  
32.     }  
33.  
34.     // Validation métier personnalisée  
35.     if (utilisateurService.emailExiste(inscriptionDTO.getEmail())) {  
36.         bindingResult.rejectValue("email", "email.existe", "Cet email  
est déjà utilisé");  
37.         return "formulaire-inscription";  
38.     }  
39.  
40.     // Traitement si tout est valide
```

GESTION DES ERREURS ET VALIDATION DES DONNÉES (PARTIE 2/4) [B]

Validation avec Bean Validation et gestion des erreurs (java) - Suite

```
41.     utilisateurService.inscrire(inscriptionDTO);  
42.     return "redirect:/connexion?inscription=success";  
43. }
```


GESTION DES ERREURS ET VALIDATION DES DONNÉES (PARTIE 3/4)

Affichage des erreurs de validation dans Thymeleaf (html)

```
1. <!-- Template Thymeleaf avec affichage des erreurs -->
2. <form th:action="@{/inscription}" th:object="${inscriptionDTO}" method="post">
3.
4.     <div class="form-group">
5.         <label for="nom">Nom:</label>
6.         <input type="text" id="nom" th:field="*{nom}"
7.             th:classappend="${#fields.hasErrors('nom')} ? 'error' : ''" />
8.         <span th:if="${#fields.hasErrors('nom')}"
9.             th:errors="*{nom}"
10.            class="error-message"></span>
11.     </div>
12.
13.     <div class="form-group">
14.         <label for="email">Email:</label>
15.         <input type="email" id="email" th:field="*{email}"
16.             th:classappend="${#fields.hasErrors('email')} ? 'error' : ''" />
17.         <span th:if="${#fields.hasErrors('email')}"
18.             th:errors="*{email}"
19.            class="error-message"></span>
20.     </div>
```

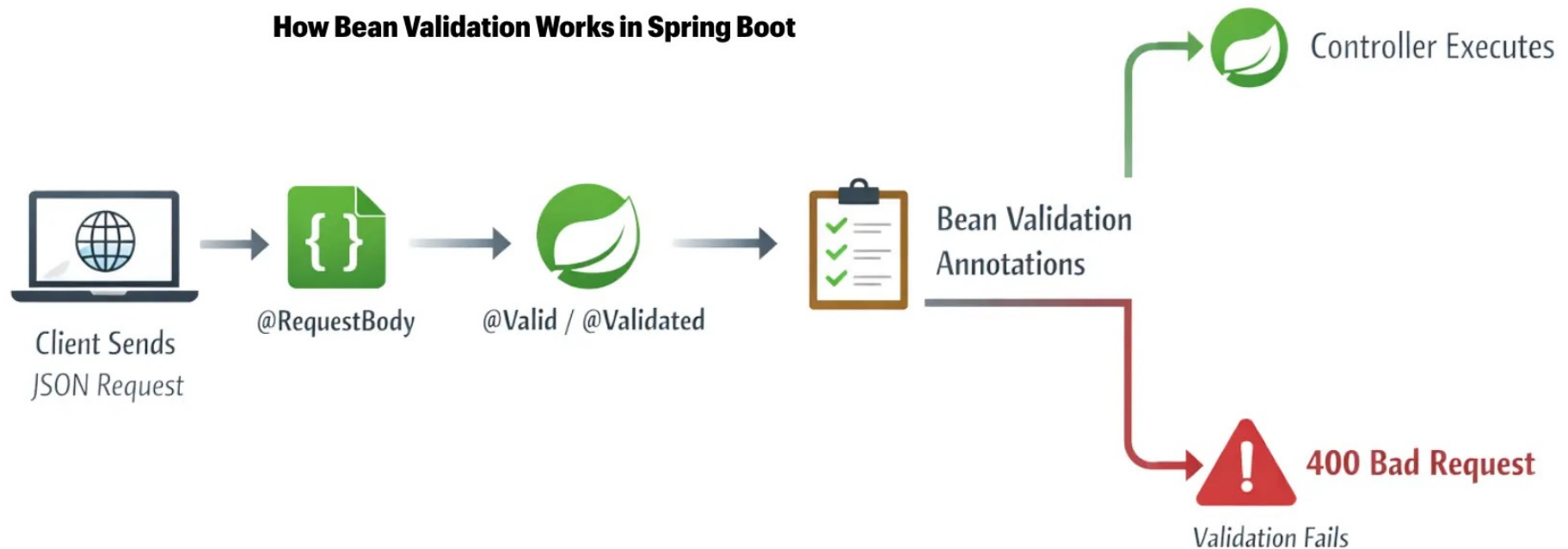
GESTION DES ERREURS ET VALIDATION DES DONNÉES (PARTIE 3/4) [A]

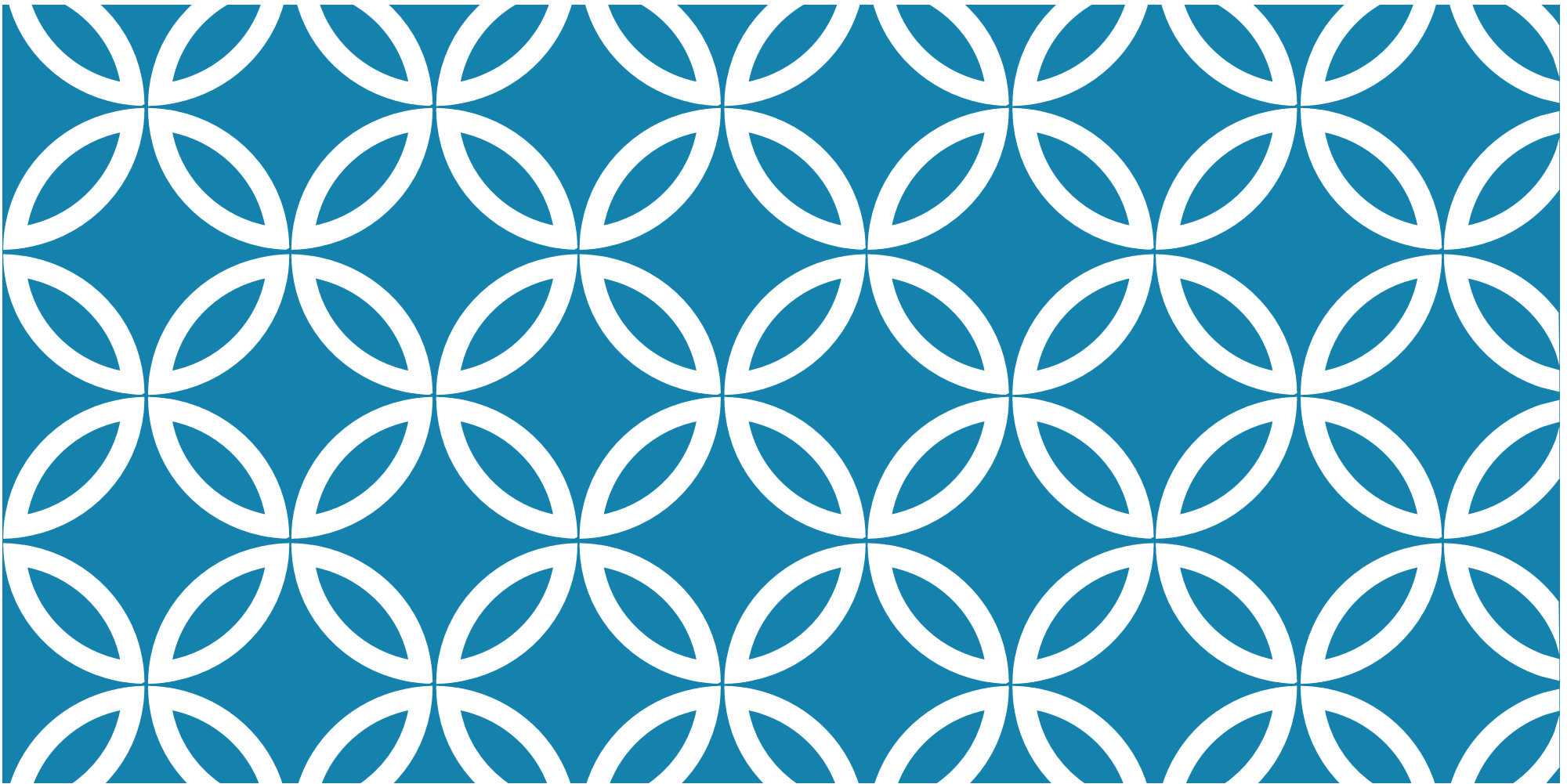
Affichage des erreurs de validation dans Thymeleaf (html) - Suite

```
21.
22.     <div class="form-group">
23.         <label for="motDePasse">Mot de passe:</label>
24.         <input type="password" id="motDePasse" th:field="*{motDePasse}"
25.             th:classappend="${#fields.hasErrors('motDePasse')}"
? 'error' : '' />
26.         <span th:if="${#fields.hasErrors('motDePasse')}"
27.             th:errors="*{motDePasse}"
28.             class="error-message"></span>
29.     </div>
30.
31.     <div th:if="${messageErreur}" class="alert alert-danger"
th:text="${messageErreur}"></div>
32.
33.     <button type="submit">S'inscrire</button>
34. </form>
```

GESTION DES ERREURS ET VALIDATION DES DONNÉES (PARTIE 4/4)

How Bean Validation Works in Spring Boot





LECTURE 5 : PERSISTENCE DES DONNÉES AVEC SPRING DATA JPA

- Introduction à JPA (Java Persistence API) et à l'ORM (Object-Relational Mapping), - Configuration de la connexion à la base de données et propriétés Hibernate, - Création d'entités JPA avec annotations

INTRODUCTION À JPA ET ORM (PARTIE 1/2)

JPA (Java Persistence API) est une spécification Java standard qui définit comment gérer la persistance des données relationnelles dans les applications Java. Elle propose une abstraction de haut niveau permettant de manipuler des objets Java tout en les stockant dans une base de données relationnelle, sans écrire de SQL manuellement.

INTRODUCTION À JPA ET ORM (PARTIE 2/2)

Spécification standardisée (JSR 338) indépendante du fournisseur

Abstraction de la couche de persistance pour réduire le code boilerplate

Mapping automatique entre objets Java et tables relationnelles

Gestion transparente des transactions et du cycle de vie des entités

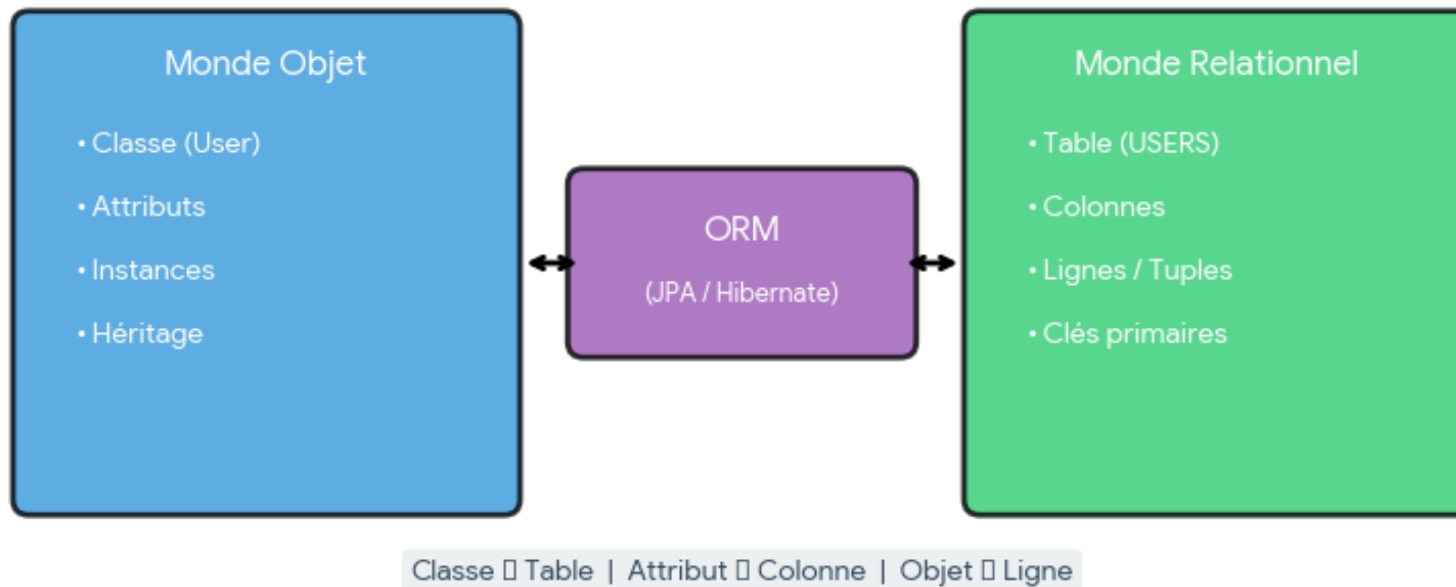
Langage de requêtes orienté objet (JPQL) au lieu de SQL natif

CONCEPT DE L'ORM (OBJECT-RELATIONAL MAPPING) (PARTIE 1/3)

L'ORM (Object-Relational Mapping) est un paradigme de programmation qui permet de convertir automatiquement les données entre le système de types orienté objet (classes Java) et le système de types relationnel (tables SQL). Il résout le problème d'incompatibilité entre ces deux mondes, appelé 'impedance mismatch'.

CONCEPT DE L'ORM (OBJECT-RELATIONAL MAPPING) (PARTIE 2/3)

Le Concept de l'ORM (Object-Relational Mapping)



CONCEPT DE L'ORM (OBJECT-RELATIONAL MAPPING) (PARTIE 3/3)

Mapping Objet-Relationnel : Correspondances

Concept Orienté Objet	Concept Relationnel	Annotation JPA
Classe	Table	@Entity
Attribut	Colonne	@Column
Instance d'objet	Ligne (enregistrement)	-
Référence d'objet	Clé étrangère	@ManyToOne, @OneToMany
Collection d'objets	Table de jointure	@ManyToMany

AVANTAGES ET IMPLÉMENTATIONS JPA (PARTIE 1/3)

Avantages de l'utilisation de JPA/ORM

AVANTAGES ET IMPLÉMENTATIONS JPA (PARTIE 2/3)

Productivité accrue : moins de code SQL manuel à écrire et maintenir

Portabilité : changement de base de données facilité (MySQL → PostgreSQL)

Maintenance simplifiée : modifications du schéma centralisées dans les entités

Sécurité renforcée : protection native contre les injections SQL

Cache de premier et second niveau pour optimiser les performances

Gestion automatique des relations complexes entre entités

AVANTAGES ET IMPLÉMENTATIONS JPA (PARTIE 3/3)

Principales Implémentations JPA

Implémentation	Fournisseur	Utilisation	Points forts
Hibernate	Red Hat	Par défaut dans Spring Boot	Mature, riche en fonctionnalités, grande communauté
EclipseLink	Eclipse Foundation	Implémentation de référence	Conforme strictement à la spec JPA
OpenJPA	Apache	Projets Apache	Léger, orienté performance
DataNucleus	DataNucleus	Projets spécifiques	Support NoSQL en plus du relationnel

CONFIGURATION DE LA CONNEXION À LA BASE DE DONNÉES (PARTIE 1/3)

Spring Boot simplifie drastiquement la configuration de la connexion à la base de données grâce à l'auto-configuration. Il suffit de définir les propriétés de connexion dans le fichier `application.properties` ou `application.yml`, et Spring Boot configure automatiquement le `DataSource`, l'`EntityManager` et le `TransactionManager`.

CONFIGURATION DE LA CONNEXION À LA BASE DE DONNÉES (PARTIE 2/3)

Configuration de connexion MySQL (properties)

```
1. # Configuration MySQL dans application.properties
2. spring.datasource.url=jdbc:mysql://localhost:3306/ma_base_donnees
3. spring.datasource.username=root
4. spring.datasource.password=motdepasse
5. spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
6.
7. # Configuration du pool de connexions HikariCP (par défaut)
8. spring.datasource.hikari.maximum-pool-size=10
9. spring.datasource.hikari.minimum-idle=5
10. spring.datasource.hikari.connection-timeout=30000
```

CONFIGURATION DE LA CONNEXION À LA BASE DE DONNÉES (PARTIE 3/3)

Configurations alternatives (PostgreSQL et H2) (properties)

```
1. # Configuration PostgreSQL
2. spring.datasource.url=jdbc:postgresql://localhost:5432/ma_base_donnees
3. spring.datasource.username=postgres
4. spring.datasource.password=motdepasse
5. spring.datasource.driver-class-name=org.postgresql.Driver
6.
7. # Configuration H2 (base en mémoire pour tests)
8. spring.datasource.url=jdbc:h2:mem:testdb
9. spring.datasource.driver-class-name=org.h2.Driver
10. spring.h2.console.enabled=true
```

PROPRIÉTÉS HIBERNATE ESSENTIELLES (PARTIE 1/3)

Hibernate, en tant qu'implémentation JPA utilisée par Spring Boot, offre de nombreuses propriétés de configuration pour contrôler son comportement : génération du schéma, dialecte SQL, affichage des requêtes, formatage, stratégies de naming, etc. Ces propriétés sont préfixées par 'spring.jpa' dans Spring Boot.

PROPRIÉTÉS HIBERNATE ESSENTIELLES

(PARTIE 2/3)

Propriétés Hibernate dans application.properties (properties)

```
1. # Propriétés JPA/Hibernate essentielles
2.
3. # Dialecte SQL (auto-déTECTÉ gÉNÉralement)
4. spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect
5.
6. # Stratégie de gÉNÉration du schéma
7. spring.jpa.hibernate.ddl-auto=update
8. # Options: none, validate, update, create, create-drop
9.
10. # Affichage et formatage des requêtes SQL
11. spring.jpa.show-sql=true
12. spring.jpa.properties.hibernate.format_sql=true
13. logging.level.org.hibernate.SQL=DEBUG
14. logging.level.org.hibernate.type.descriptor.sql.BasicBinder=TRACE
15.
16. # Stratégie de naming (physique et implicite)
17. spring.jpa.hibernate.naming.physical-strategy=
org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl
18. spring.jpa.hibernate.naming.implicit-strategy=
org.hibernate.boot.model.naming.ImplicitNamingStrategyJpaCompliantImpl
```

PROPRIÉTÉS HIBERNATE ESSENTIELLES (PARTIE 3/3)

Valeurs de `spring.jpa.hibernate.ddl-auto`

Valeur	Comportement	Usage recommandé
none	Aucune action sur le schéma	Production
validate	Valide le schéma, erreur si différence	Production (avec Flyway/Liquibase)
update	Met à jour le schéma (ajoute colonnes/tables)	Développement
create	Supprime et recrée le schéma à chaque démarrage	Tests automatisés
create-drop	Crée au démarrage, supprime à l'arrêt	Tests unitaires

CRÉATION D'ENTITÉS JPA : LES BASES (PARTIE 1/2)

Une entité JPA est une classe Java standard (POJO) annotée avec `@Entity` qui représente une table dans la base de données. Chaque instance de cette classe correspond à une ligne de la table. Les entités doivent respecter certaines règles : avoir un constructeur sans arguments, ne pas être final, et posséder un identifiant unique annoté avec `@Id`.

CRÉATION D'ENTITÉS JPA : LES BASES (PARTIE 2/2)

Exemple d'entité JPA basique - Utilisateur (java)

```
1. package com.example.demo.model;
2.
3. import jakarta.persistence.*;
4. import java.time.LocalDateTime;
5.
6. @Entity
7. @Table(name = "utilisateurs")
8. public class Utilisateur {
9.
10.     @Id
11.     @GeneratedValue(strategy = GenerationType.IDENTITY)
12.     private Long id;
13.
14.     @Column(name = "nom", nullable = false, length = 100)
15.     private String nom;
16.
17.     @Column(name = "email", unique = true, nullable = false)
18.     private String email;
19.
20.     @Column(name = "date_creation")
```

CRÉATION D'ENTITÉS JPA : LES BASES (PARTIE 2/2) [A]

Exemple d'entité JPA basique - Utilisateur (java) - Suite

```
21.     private LocalDateTime dateCreation;
22.
23.     // Constructeur sans arguments (obligatoire pour JPA)
24.     public Utilisateur() {
25.     }
26.
27.     // Constructeur avec paramètres
28.     public Utilisateur(String nom, String email) {
29.         this.nom = nom;
30.         this.email = email;
31.         this.dateCreation = LocalDateTime.now();
32.     }
33.
34.     // Getters et Setters
35.     public Long getId() { return id; }
36.     public void setId(Long id) { this.id = id; }
37.
38.     public String getNom() { return nom; }
39.     public void setNom(String nom) { this.nom = nom; }
```

40.

CRÉATION D'ENTITÉS JPA : LES BASES (PARTIE 2/2) [B]

Exemple d'entité JPA basique - Utilisateur (java) - Suite

```
41.     public String getEmail() { return email; }
42.     public void setEmail(String email) { this.email = email; }
43.
44.     public LocalDateTime getDateCreation() { return dateCreation; }
45.     public void setDateCreation(LocalDateTime dateCreation) {
46.         this.dateCreation = dateCreation;
47.     }
48. }
```

ANNOTATIONS @ENTITY, @ID ET @GENERATEDVALUE (PARTIE 1/4)

Annotations fondamentales des entités JPA

Annotation	Rôle	Attributs importants	Exemple
@Entity	Marque une classe comme entité JPA	name (nom de l'entité pour JPQL)	@Entity(name="User")
@Table	Personnalise la table associée	name, schema, uniqueConstraints, indexes	@Table(name="users")
@Id	Définit la clé primaire	-	@Id
@GeneratedValue	Stratégie de génération de l'ID	strategy (AUTO, IDENTITY, SEQUENCE, TABLE)	@GeneratedValue(strategy=IDENTITY)
@Column	Personnalise une colonne	name, nullable, unique, length, precision	@Column(nullable=false)

ANNOTATIONS @ENTITY, @ID ET @GENERATEDVALUE (PARTIE 2/4)

Stratégies de génération d'identifiants

ANNOTATIONS @ENTITY, @ID ET @GENERATEDVALUE (PARTIE 3/4)

GenerationType.IDENTITY : Auto-incrémentation de la base de données (MySQL, PostgreSQL)

- Avantage : Simple et performant
- Inconvénient : Requier un accès DB pour obtenir l'ID

GenerationType.SEQUENCE : Utilise une séquence DB (PostgreSQL, Oracle)

- Avantage : Optimisable avec allocationSize
- Nécessite : @SequenceGenerator

GenerationType.TABLE : Table dédiée pour les IDs (portable)

- Avantage : Fonctionne partout
- Inconvénient : Moins performant

GenerationType.AUTO : JPA choisit automatiquement selon la DB

- Recommandé pour la portabilité

ANNOTATIONS @ENTITY, @ID ET @GENERATEDVALUE (PARTIE 4/4)

Stratégies avancées de génération d'ID (java)

```
1. // Exemple avec SEQUENCE (PostgreSQL)
2. @Entity
3. public class Produit {
4.
5.     @Id
6.     @GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "produit_seq"
7.     @SequenceGenerator(name = "produit_seq", sequenceName = "produit_sequence",
8.         allocationSize = 50)
9.     private Long id;
10.
11.     private String nom;
12.     private Double prix;
13.
14.     // Constructeurs, getters, setters...
15. }
16.
17. // Exemple avec UUID (pour systèmes distribués)
18. @Entity
19. public class Commande {
20.
```

ANNOTATIONS @ENTITY, @ID ET @GENERATEDVALUE (PARTIE 4/4) [A]

Stratégies avancées de génération d'ID (java) - Suite

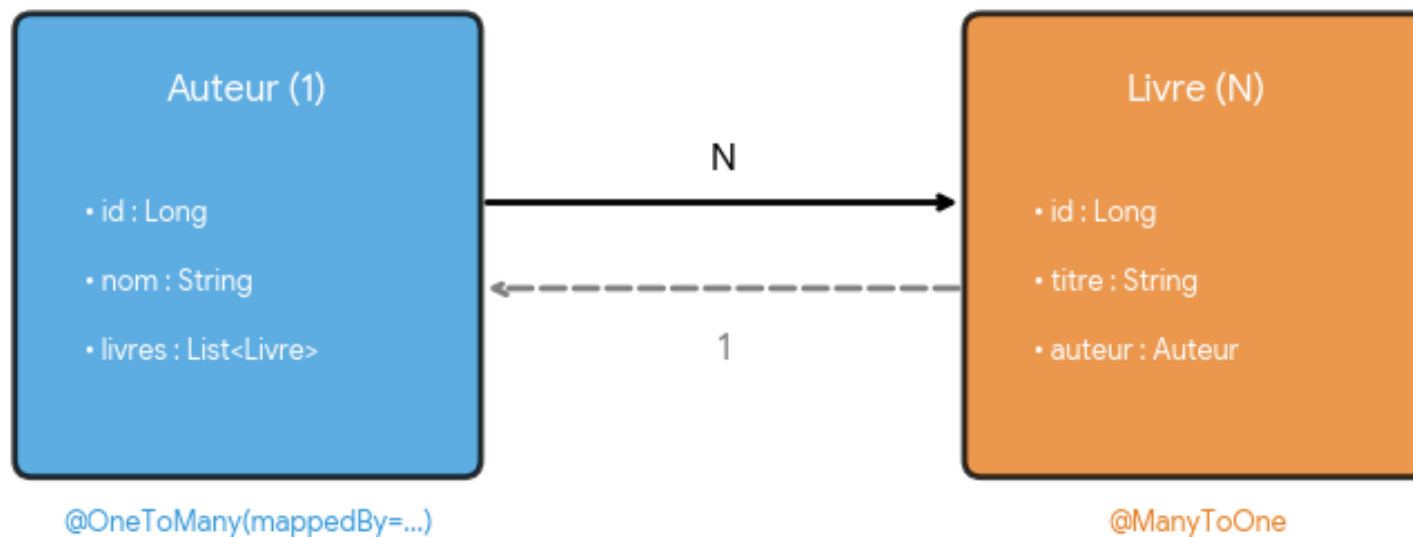
```
21.     @Id
22.     @GeneratedValue(strategy = GenerationType.UUID)
23.     private UUID id;
24.
25.     private LocalDateTime dateCommande;
26.
27.     // Constructeurs, getters, setters...
28. }
```

RELATIONS JPA : @ONETOMANY ET @MANYTOONE (PARTIE 1/4)

Les relations entre entités permettent de modéliser les associations entre tables. La relation @OneToMany/@ManyToOne est la plus courante : elle représente une relation un-à-plusieurs bidirectionnelle. Par exemple, un Auteur peut avoir plusieurs Livres, et chaque Livre appartient à un seul Auteur. Le côté @ManyToOne possède la clé étrangère.

RELATIONS JPA : @ONETOMANY ET @MANYTOONE (PARTIE 2/4)

Relations JPA : @OneToMany et @ManyToOne



RELATIONS JPA : @ONETOMANY ET @MANYTOONE (PARTIE 3/4)

Côté @OneToMany - Entité Auteur (java)

```
1. // Côté "One" - Auteur (côté inverse)
2. @Entity
3. public class Auteur {
4.
5.     @Id
6.     @GeneratedValue(strategy = GenerationType.IDENTITY)
7.     private Long id;
8.
9.     private String nom;
10.
11.     // mappedBy indique que Livre.auteur est le propriétaire de la relation
12.     // cascade permet de propager les opérations (persist, remove, etc.)
13.     @OneToMany(mappedBy = "auteur", cascade = CascadeType.ALL,
14.                orphanRemoval = true)
15.     private List<Livre> livres = new ArrayList<>();
16.
17.     // Méthodes utilitaires pour maintenir la cohérence bidirectionnelle
18.     public void ajouterLivre(Livre livre) {
19.         livres.add(livre);
20.         livre.setAuteur(this);
```

RELATIONS JPA : @ONETOMANY ET @MANYTOONE (PARTIE 3/4) [A]

Côté @OneToMany - Entité Auteur (java) - Suite

```
21.     }
22.
23.     public void retirerLivre(Livre livre) {
24.         livres.remove(livre);
25.         livre.setAuteur(null);
26.     }
27.
28.     // Constructeurs, getters, setters...
29. }
```

RELATIONS JPA : @ONETOMANY ET @MANYTOONE (PARTIE 4/4)

Côté @ManyToOne - Entité Livre (java)

```
1. // Côté "Many" - Livre (côté propriétaire)
2. @Entity
3. public class Livre {
4.
5.     @Id
6.     @GeneratedValue(strategy = GenerationType.IDENTITY)
7.     private Long id;
8.
9.     private String titre;
10.
11.     private String isbn;
12.
13.     // Le côté @ManyToOne possède la clé étrangère
14.     // JoinColumn permet de personnaliser le nom de la colonne FK
15.     @ManyToOne(fetch = FetchType.LAZY)
16.     @JoinColumn(name = "auteur_id", nullable = false)
17.     private Auteur auteur;
18.
19.     // Constructeurs
20.     public Livre() {}
```


RELATIONS JPA : @ONETOMANY ET @MANYTOONE (PARTIE 4/4) [A]

Côté @ManyToOne - Entité Livre (java) - Suite

```
21.  
22.     public Livre(String titre, String isbn) {  
23.         this.titre = titre;  
24.         this.isbn = isbn;  
25.     }  
26.  
27.     // Getters et setters  
28.     public Long getId() { return id; }  
29.     public void setId(Long id) { this.id = id; }  
30.  
31.     public String getTitre() { return titre; }  
32.     public void setTitre(String titre) { this.titre = titre; }  
33.  
34.     public String getIsbn() { return isbn; }  
35.     public void setIsbn(String isbn) { this.isbn = isbn; }  
36.  
37.     public Auteur getAuteur() { return auteur; }  
38.     public void setAuteur(Auteur auteur) { this.auteur = auteur; }  
39. }
```

TYPES DE FETCHTYPE ET CASCADE (PARTIE 1/3)

FetchType : Stratégies de chargement des relations

Type	Comportement	Quand utiliser	Performance
EAGER	Charge immédiatement la relation	Relations petites et toujours nécessaires	Peut causer N+1 queries
LAZY (défaut)	Charge la relation à la demande	Relations volumineuses ou optionnelles	Optimale si bien utilisé
LAZY + JOIN FETCH	Charge explicitement avec JPQL	Contrôle fin du chargement	Meilleure approche

TYPES DE FETCHTYPE ET CASCADE (PARTIE 2/3)

Types de Cascade : Propagation des opérations

CascadeType	Opération propagée	Exemple d'usage
PERSIST	Sauvegarde en cascade	Sauvegarder auteur sauvegarde ses livres
MERGE	Mise à jour en cascade	Mettre à jour auteur met à jour ses livres
REMOVE	Suppression en cascade	Supprimer auteur supprime ses livres
REFRESH	Rafrâichissement en cascade	Recharger l'état depuis la DB
DETACH	Détachement en cascade	Détacher du contexte de persistance
ALL	Toutes les opérations	Entités fortement liées (composition)

TYPES DE FETCHTYPE ET CASCADE (PARTIE 3/3)

Optimisation avec JOIN FETCH et utilisation de Cascade (java)

```
1. // Exemple de requête JPQL avec JOIN FETCH pour éviter N+1
2. @Repository
3. public interface AuteurRepository extends JpaRepository<Auteur, Long> {
4.
5.     // Sans JOIN FETCH : 1 requête pour les auteurs + N requêtes pour les livres
6.     // Avec JOIN FETCH : 1 seule requête avec jointure
7.     @Query("SELECT a FROM Auteur a LEFT JOIN FETCH a.livres WHERE a.id = :id")
8.     Optional<Auteur> findByIdWithLivres(@Param("id") Long id);
9.
10.    @Query("SELECT DISTINCT a FROM Auteur a LEFT JOIN FETCH a.livres")
11.    List<Auteur> findAllWithLivres();
12. }
13.
14. // Utilisation du Cascade
15. @Service
16. public class AuteurService {
17.
18.     @Autowired
19.     private AuteurRepository auteurRepository;
20.
```

TYPES DE FETCHTYPE ET CASCADE (PARTIE 3/3) [A]

Optimisation avec JOIN FETCH et utilisation de Cascade (java) - Suite

```
21.     public void creerAuteurAvecLivres() {
22.         Auteur auteur = new Auteur("Victor Hugo");
23.
24.         Livre livre1 = new Livre("Les Misérables", "978-2070409228");
25.         Livre livre2 = new Livre("Notre-Dame de Paris", "978-2070413089");
26.
27.         // Grâce à cascade = ALL, les livres seront sauvegardés automatiquement
28.         auteur.ajouterLivre(livre1);
29.         auteur.ajouterLivre(livre2);
30.
31.         auteurRepository.save(auteur); // Sauvegarde auteur ET livres
32.     }
33. }
```

AUTRES TYPES DE RELATIONS JPA (PARTIE 1/2)

Relations @OneToOne et @ManyToMany

AUTRES TYPES DE RELATIONS JPA (PARTIE 2/2)

Relation @OneToOne bidirectionnelle (java)

```
1. // Relation @OneToOne : Un utilisateur a un seul profil
2. @Entity
3. public class Utilisateur {
4.     @Id
5.     @GeneratedValue(strategy = GenerationType.IDENTITY)
6.     private Long id;
7.
8.     private String username;
9.
10.    // Relation bidirectionnelle OneToOne
11.    @OneToOne(mappedBy = "utilisateur", cascade = CascadeType.ALL)
12.    private Profil profil;
13. }
14.
15. @Entity
16. public class Profil {
17.     @Id
18.     @GeneratedValue(strategy = GenerationType.IDENTITY)
19.     private Long id;
20.
```

AUTRES TYPES DE RELATIONS JPA (PARTIE 2/2) [A]

Relation @OneToOne bidirectionnelle (java) - Suite

```
21.     private String bio;
22.     private String avatar;
23.
24.     // Côté propriétaire de la relation (possède la FK)
25.     @OneToOne
26.     @JoinColumn(name = "utilisateur_id", unique = true)
27.     private Utilisateur utilisateur;
28. }
```


INTRODUCTION AUX REPOSITORIES

SPRING DATA JPA (PARTIE 1/3)

Spring Data JPA simplifie drastiquement l'accès aux données en fournissant une couche d'abstraction au-dessus de JPA. Les repositories constituent l'interface principale pour interagir avec la base de données, éliminant le besoin d'écrire du code boilerplate pour les opérations courantes.

INTRODUCTION AUX REPOSITORIES SPRING DATA JPA (PARTIE 2/3)

Abstraction complète de la couche de persistance

Réduction significative du code répétitif (boilerplate)

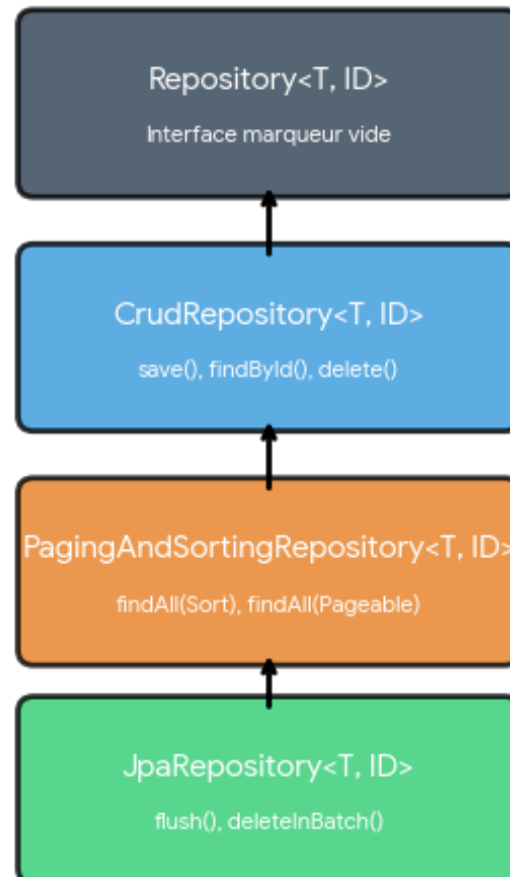
Implémentation automatique des méthodes CRUD

Support des requêtes dérivées à partir des noms de méthodes

Intégration transparente avec le contexte Spring

INTRODUCTION AUX REPOSITORIES SPRING DATA JPA (PARTIE 3/3)

Hiérarchie des Repositories Spring Data JPA



HIÉRARCHIE DES INTERFACES REPOSITORY (PARTIE 1/2)

Interfaces Repository et leurs fonctionnalités

Interface	Méthodes fournies	Cas d'usage
Repository<T, ID>	Interface marker (vide)	Base pour typage personnalisé
CrudRepository<T, ID>	save(), findById(), findAll(), delete(), count()	Opérations CRUD basiques
PagingAndSortingRepository<T, ID>	Hérite CRUD + findAll(Pageable), findAll(Sort)	Pagination et tri des résultats
JpaRepository<T, ID>	Hérite tout + flush(), saveAndFlush(), deleteInBatch()	Fonctionnalités JPA complètes (recommandé)

HIÉRARCHIE DES INTERFACES REPOSITORY (PARTIE 2/2)

En pratique, `JpaRepository` est l'interface la plus utilisée car elle combine toutes les fonctionnalités des interfaces parentes et ajoute des méthodes spécifiques à JPA comme le flushing et les opérations par batch.

CRÉATION D'UN REPOSITORY - EXEMPLE PRATIQUE (PARTIE 1/3)

Repository basique pour l'entité Utilisateur (java)

```
1. package com.example.repository;
2.
3. import com.example.model.Utilisateur;
4. import org.springframework.data.jpa.repository.JpaRepository;
5. import org.springframework.stereotype.Repository;
6.
7. @Repository
8. public interface UtilisateurRepository extends JpaRepository<Utilisateur, Long> {
9.     // Spring Data JPA génère automatiquement l'implémentation
10.    // Aucune méthode à implémenter pour les opérations CRUD de base
11. }
```

CRÉATION D'UN REPOSITORY - EXEMPLE PRATIQUE (PARTIE 2/3)

L'annotation `@Repository` est optionnelle mais recommandée pour la clarté

`JpaRepository<Utilisateur, Long>` : `Utilisateur` = type d'entité, `Long` = type de la clé primaire

Aucune implémentation nécessaire : Spring génère le code automatiquement

Injection par constructeur dans les services via `@Autowired` ou injection directe

CRÉATION D'UN REPOSITORY - EXEMPLE PRATIQUE (PARTIE 3/3)

Injection et utilisation du Repository (java)

```
1. // Utilisation dans un Service
2. @Service
3. public class UtilisateurService {
4.
5.     private final UtilisateurRepository utilisateurRepository;
6.
7.     public UtilisateurService(UtilisateurRepository utilisateurRepository) {
8.         this.utilisateurRepository = utilisateurRepository;
9.     }
10.
11.     public List<Utilisateur> obtenirTousLesUtilisateurs() {
12.         return utilisateurRepository.findAll();
13.     }
14. }
```


MÉTHODES CRUD PRÉDÉFINIES (PARTIE 1/3)

Opérations CRUD Complètes

MÉTHODES CRUD PRÉDÉFINIES (PARTIE 2/3)

Exemples d'opérations CRUD complètes (java)

```
1. // CREATE - Sauvegarder une entité
2. Utilisateur nouvelUtilisateur = new Utilisateur("Jean", "jean@email.com");
3. utilisateurRepository.save(nouvelUtilisateur);
4.
5. // READ - Récupérer par ID
6. Optional<Utilisateur> utilisateur = utilisateurRepository.findById(1L);
7.
8. // READ - Récupérer tous
9. List<Utilisateur> tousLesUtilisateurs = utilisateurRepository.findAll();
10.
11. // UPDATE - Modifier (même méthode save)
12. Utilisateur existant = utilisateurRepository.findById(1L).orElseThrow();
13. existant.setEmail("nouveau@email.com");
14. utilisateurRepository.save(existant);
15.
16. // DELETE - Supprimer
17. utilisateurRepository.deleteById(1L);
18. // ou
19. utilisateurRepository.delete(utilisateur);
```

MÉTHODES CRUD PRÉDÉFINIES (PARTIE 3/3)

`save()` : Insertion ET mise à jour (upsert automatique)

`findById()` : Retourne `Optional<T>` pour gérer l'absence de résultat

`findAll()` : Récupère toutes les entités (attention aux performances)

`existsById()` : Vérifie l'existence sans charger l'entité

`count()` : Compte le nombre total d'entités

`deleteAll()` : Supprime toutes les entités (à utiliser avec précaution)

MÉTHODES DE REQUÊTES DÉRIVÉES - PRINCIPES (PARTIE 1/2)

Spring Data JPA permet de créer des requêtes automatiquement en analysant le nom des méthodes. Le framework parse le nom de la méthode et génère la requête SQL/JPQL correspondante sans nécessiter d'implémentation.

MÉTHODES DE REQUÊTES DÉRIVÉES - PRINCIPES (PARTIE 2/2)

Mots-clés pour les requêtes dérivées

Mot-clé	Exemple de méthode	Requête générée (équivalent)
findBy	findByNom(String nom)	WHERE nom = ?
findBy...And...	findByNomAndPrenom(String n, String p)	WHERE nom = ? AND prenom = ?
findBy...Or...	findByNomOrEmail(String n, String e)	WHERE nom = ? OR email = ?
findBy...OrderBy...	findByNomOrderByAgeDesc(String nom)	WHERE nom = ? ORDER BY age DESC
countBy	countByAge(int age)	SELECT COUNT(*) WHERE age = ?
deleteBy	deleteByNom(String nom)	DELETE WHERE nom = ?

REQUÊTES DÉRIVÉES - EXEMPLES AVANCÉS

Repository avec requêtes dérivées variées (java)

```
1. @Repository
2. public interface UtilisateurRepository extends JpaRepository<Utilisateur, Long> {
3.
4.     // Recherche simple par propriété
5.     List<Utilisateur> findByNom(String nom);
6.
7.     // Recherche avec plusieurs critères
8.     List<Utilisateur> findByNomAndPrenom(String nom, String prenom);
9.
10.    // Recherche avec opérateur LIKE
11.    List<Utilisateur> findByEmailContaining(String email);
12.
13.    // Recherche avec comparaison
14.    List<Utilisateur> findByAgeGreaterThan(int age);
15.    List<Utilisateur> findByAgeBetween(int ageMin, int ageMax);
16.
17.    // Recherche avec tri
18.    List<Utilisateur> findByNomOrderByAgeDesc(String nom);
19.
20.    // Recherche avec limitation de résultats
```

REQUÊTES DÉRIVÉES - EXEMPLES AVANCÉS

[A]

Repository avec requêtes dérivées variées (java) - Suite

```
21.     Utilisateur findFirstByNomOrderByDateCreationDesc(String nom);
22.     List<Utilisateur> findTop5ByOrderByAgeDesc();
23.
24.     // Vérification d'existence
25.     boolean existsByEmail(String email);
26.
27.     // Comptage
28.     long countByAge(int age);
29. }
```

OPÉRATEURS DE REQUÊTES DÉRIVÉES (PARTIE 1/2)

Opérateurs disponibles pour les requêtes dérivées

Opérateur	Exemple	Description
Containing/Contains	<code>findByNomContaining(String s)</code>	LIKE '%s%' - contient la chaîne
StartingWith	<code>findByNomStartingWith(String s)</code>	LIKE 's%' - commence par
EndingWith	<code>findByNomEndingWith(String s)</code>	LIKE '%s' - se termine par
IgnoreCase	<code>findByNomIgnoreCase(String s)</code>	Comparaison insensible à la casse
Between	<code>findByAgeBetween(int min, int max)</code>	WHERE age BETWEEN min AND max
LessThan/GreaterThan	<code>findByAgeLessThan(int age)</code>	WHERE age < ?
IsNull/IsNotNull	<code>findByEmailsNull()</code>	WHERE email IS NULL
In	<code>findByNomIn(Collection<String> noms)</code>	WHERE nom IN (...)
True/False	<code>findByActifTrue()</code>	WHERE actif = true

OPÉRATEURS DE REQUÊTES DÉRIVÉES (PARTIE 2/2)

Les opérateurs peuvent être combinés avec And/Or

L'ordre des paramètres doit correspondre à l'ordre dans le nom de la méthode

Utiliser IgnoreCase pour les recherches insensibles à la casse

Les requêtes dérivées sont limitées en complexité (préférer @Query pour les cas complexes)

INTRODUCTION AUX REQUÊTES PERSONNALISÉES AVEC @QUERY (PARTIE 1/3)

Lorsque les requêtes dérivées deviennent trop complexes ou illisibles, Spring Data JPA offre l'annotation @Query pour définir des requêtes JPQL (Java Persistence Query Language) ou SQL natives. Cette approche offre un contrôle total sur la requête tout en bénéficiant de l'abstraction Spring Data.

INTRODUCTION AUX REQUÊTES PERSONNALISÉES AVEC @QUERY (PARTIE 2/3)

JPQL : Langage de requête orienté objet (travaille avec les entités)

SQL natif : Requêtes SQL brutes pour des cas spécifiques

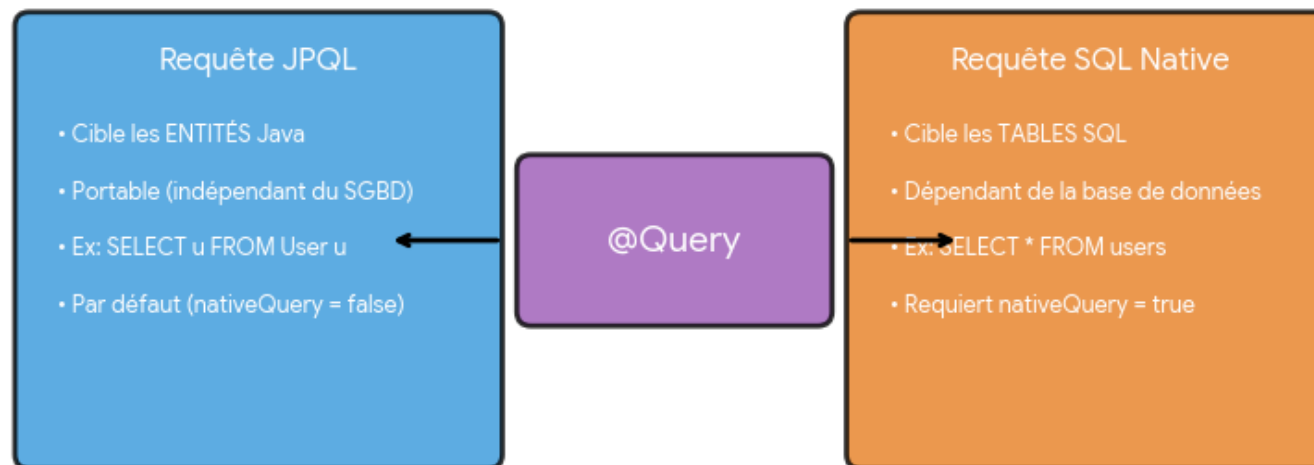
Paramètres nommés (@Param) ou positionnels (?1, ?2)

Support des projections et DTO personnalisés

Requêtes de modification avec @Modifying

INTRODUCTION AUX REQUÊTES PERSONNALISÉES AVEC @QUERY (PARTIE 3/3)

Comparaison des requêtes avec @Query



REQUÊTES JPQL AVEC @QUERY (PARTIE 1/2)

Exemples de requêtes JPQL avec @Query (java)

```
1. @Repository
2. public interface UtilisateurRepository extends JpaRepository<Utilisateur, Long> {
3.
4.     // JPQL avec paramètre nommé
5.     @Query("SELECT u FROM Utilisateur u WHERE u.nom = :nom")
6.     List<Utilisateur> rechercherParNom(@Param("nom") String nom);
7.
8.     // JPQL avec plusieurs paramètres
9.     @Query("SELECT u FROM Utilisateur u WHERE u.nom = :nom AND u.age > :age")
10.    List<Utilisateur> rechercherParNomEtAge(
11.        @Param("nom") String nom,
12.        @Param("age") int age
13.    );
14.
15.    // JPQL avec paramètres positionnels
16.    @Query("SELECT u FROM Utilisateur u WHERE u.email LIKE %?1%")
17.    List<Utilisateur> rechercherParEmail(String email);
18.
19.    // JPQL avec JOIN
20.    @Query("SELECT u FROM Utilisateur u JOIN u.commandes c WHERE
c.montant > :montant")
```

REQUÊTES JPQL AVEC @QUERY (PARTIE 1/2) [A]

Exemples de requêtes JPQL avec @Query (java) - Suite

```
21.      List<Utilisateur> utilisateursAvecCommandesSuperieurA(  
22.      @Param("montant") double montant);  
23.      // JPQL avec fonction d'agrégation  
24.      @Query("SELECT COUNT(u) FROM Utilisateur u WHERE u.actif = true")  
25.      long compterUtilisateursActifs();  
26. }
```

REQUÊTES JPQL AVEC @QUERY (PARTIE 2/2)

JPQL utilise les noms d'entités et d'attributs Java (pas les noms de tables SQL)

@Param lie les paramètres de méthode aux paramètres nommés de la requête

Les paramètres positionnels (?1, ?2) sont moins lisibles mais plus concis

JPQL supporte les JOIN, GROUP BY, ORDER BY comme SQL

Les requêtes JPQL sont vérifiées au démarrage de l'application

REQUÊTES SQL NATIVES (PARTIE 1/2)

Exemples de requêtes SQL natives (java)

```
1. @Repository
2. public interface UtilisateurRepository extends JpaRepository<Utilisateur, Long> {
3.
4.     // SQL natif simple
5.     @Query(value = "SELECT * FROM utilisateurs WHERE nom = :nom", nativeQuery = true)
6.         List<Utilisateur> rechercherParNomSQL(@Param("nom") String nom);
7.
8.     // SQL natif avec fonction spécifique à la base de données
9.     @Query(value = "SELECT * FROM utilisateurs WHERE DATE(date_creation)
10.         = CURRENT_DATE",
11.             nativeQuery = true)
12.         List<Utilisateur> utilisateursCreeAujourd'hui();
13.
14.     // SQL natif avec requête complexe
15.     @Query(value = ""SELECT u.* FROM utilisateurs u
16.         INNER JOIN commandes c ON u.id = c.utilisateur_id
17.         WHERE c.statut = 'COMPLETE'
18.         GROUP BY u.id
19.         HAVING COUNT(c.id) > :nombreCommandes"",
20.             nativeQuery = true)
21.         List<Utilisateur> utilisateursAvecPlusieursCommandes(
```


REQUÊTES SQL NATIVES (PARTIE 1/2) [A]

Exemples de requêtes SQL natives (java) - Suite

```
21.         @Param("nombreCommandes") int nombreCommandes
22.     );
23.
24.     // SQL natif retournant des valeurs scalaires
25.     @Query(value = "SELECT AVG(age) FROM utilisateurs", nativeQuery = true)
26.     Double calculerAgeMoyen();
27. }
```

REQUÊTES SQL NATIVES (PARTIE 2/2)

`nativeQuery = true` active le mode SQL natif

Utilise les noms de tables et colonnes réels de la base de données

Permet d'utiliser des fonctions spécifiques à la base (PostgreSQL, MySQL, etc.)

Utile pour l'optimisation de performances avec des requêtes complexes

Moins portable entre différents SGBD

Pas de vérification de syntaxe au démarrage (erreurs détectées à l'exécution)

REQUÊTES DE MODIFICATION AVEC @MODIFYING (PARTIE 1/3)

Requêtes de modification avec @Modifying (java)

```
1. @Repository
2. public interface UtilisateurRepository extends JpaRepository<Utilisateur, Long> {
3.
4.     // UPDATE avec JPQL
5.     @Modifying
6.     @Query("UPDATE Utilisateur u SET u.actif = :actif WHERE u.id = :id")
7.     int activerDesactiverUtilisateur(@Param("id") Long id, @Param("actif")
boolean actif);
8.
9.     // UPDATE multiple avec JPQL
10.    @Modifying
11.    @Query("UPDATE Utilisateur u SET u.dernierLogin = CURRENT_TIMESTAMP WHERE
u.email = :email")
12.    int mettreAJourDernierLogin(@Param("email") String email);
13.
14.    // DELETE avec JPQL
15.    @Modifying
16.    @Query("DELETE FROM Utilisateur u WHERE u.actif = false AND u.dateCreation
< :date")
17.    int supprimerUtilisateursInactifs(@Param("date") LocalDateTime date);
18.
19.    // UPDATE avec SQL natif
20.    @Modifying
```

REQUÊTES DE MODIFICATION AVEC @MODIFYING (PARTIE 1/3) [A]

Requêtes de modification avec @Modifying (java) - Suite

```
21.     @Query(value = "UPDATE utilisateurs SET tentatives_connexion = 0  
WHERE id = :id",  
22.         nativeQuery = true)  
23.     int reinitialiserTentativesConnexion(@Param("id") Long id);  
24. }
```

REQUÊTES DE MODIFICATION AVEC `@MODIFYING` (PARTIE 2/3)

L'annotation `@Modifying` est obligatoire pour les requêtes UPDATE et DELETE. Elle indique à Spring Data JPA que la requête modifie les données. Le type de retour est généralement `int` (nombre de lignes affectées) ou `void`.

REQUÊTES DE MODIFICATION AVEC @MODIFYING (PARTIE 3/3)

@Modifying obligatoire pour UPDATE/DELETE

Retourne le nombre de lignes affectées (int) ou void

Nécessite @Transactional au niveau du service appelant

clearAutomatically = true pour vider le contexte de persistance après l'exécution

Attention : ne met pas à jour les entités déjà chargées en mémoire

OPTIONS AVANCÉES DE @QUERY (PARTIE 1/3)

Paramètres de configuration de @Query

Paramètre	Valeur par défaut	Description	Exemple d'usage
nativeQuery	false	Active le mode SQL natif	Requêtes SQL spécifiques à la BD
countQuery	null	Requête personnalisée pour la pagination	Optimiser le COUNT pour pagination
name	null	Nom de la named query JPA	Réutilisation de requêtes prédéfinies
value	requis	La requête JPQL ou SQL	Toutes les requêtes @Query

OPTIONS AVANCÉES DE @QUERY (PARTIE 2/3)

Utilisation des options avancées (java)

```
1. // Exemple avec countQuery pour optimiser la pagination
2. @Query(value = "SELECT u FROM Utilisateur u JOIN FETCH u.commandes WHERE
u.actif = true",
3.         countQuery = "SELECT COUNT(u) FROM Utilisateur u WHERE u.actif = true")
4. Page<Utilisateur> trouverUtilisateursActifsAvecCommandes(Pageable pageable);
5.
6. // Exemple avec @Modifying et clearAutomatically
7. @Modifying(clearAutomatically = true)
8. @Query("UPDATE Utilisateur u SET u.nom = :nom WHERE u.id = :id")
9. void mettreAJourNom(@Param("id") Long id, @Param("nom") String nom);
```


OPTIONS AVANCÉES DE @QUERY (PARTIE 3/3)

`countQuery` évite les JOIN inutiles lors du comptage pour la pagination

`clearAutomatically` vide le contexte de persistance après `@Modifying`

Les requêtes peuvent retourner `Page<T>`, `List<T>`, `Optional<T>`, ou types primitifs

Support des projections pour ne récupérer que certains champs

PAGINATION ET TRI AVEC SPRING DATA JPA

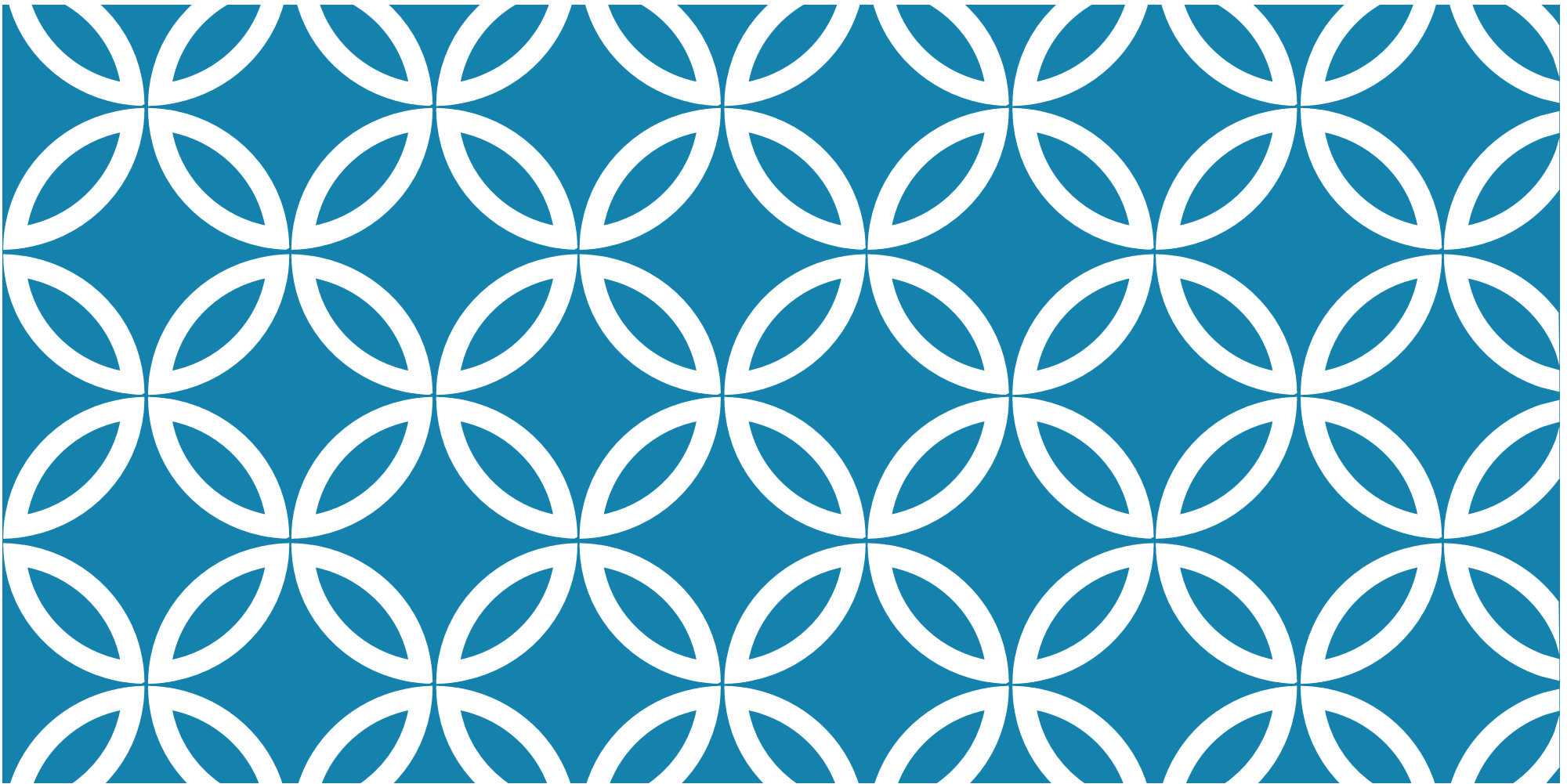
Exemple de Code (Code)

```
1. @Repository
2. public interface UtilisateurRepository extends JpaRepository<Utilisateur, Long> {
3.
4.     // Pagination simple
5.     Page<Utilisateur> findByActif(boolean actif, Pageable pageable);
6.
7.     // Tri simple
8.     List<Utilisateur> findByNom(String nom, Sort sort);
9.
10.    // Pagination + tri combinés
11.    Page<Utilisateur> findByAgeGreaterThan(int age, Pageable pageable);
12.
13.    // Avec @Query
14.    @Query("SELECT u FROM Utilisateur u WHERE u.email LIKE %:domaine%")
15.    Page<Utilisateur> rechercherParDomaine(@Param("domaine") String domaine,
        Pageable pageable);
16. }
17.
18. // Utilisation dans un Service
19. @Service
20. public class UtilisateurService {
```

PAGINATION ET TRI AVEC SPRING DATA JPA [A]

Exemple de Code (Code) - Suite

```
21.  
22.     public Page<Utilisateur> obtenirUtilisateurs(int page, int taille) {  
23.         Pageable pageable = PageRequest.of(page, taille, Sort.by("nom")  
24.         .ascending());  
25.         return utilisateurRepository.findAll(pageable);  
26.     }
```



LECTURE 6 : DÉVELOPPEMENT D'API REST AVEC SPRING BOOT

- Principes de l'architecture REST et conventions des méthodes HTTP (GET, POST, PUT, DELETE), - Création d'endpoints RESTful et structuration des ressources, - Sérialisation/désérialisation JSON avec

INTRODUCTION AUX API REST (PARTIE 1/2)

REST (Representational State Transfer) est un style d'architecture logicielle pour les systèmes distribués, particulièrement adapté au web. Une API REST utilise les protocoles HTTP standards pour permettre la communication entre clients et serveurs de manière stateless, scalable et uniforme. Spring Boot facilite grandement la création d'API REST grâce à ses annotations et sa configuration automatique.